

# Adaptive Security for Constrained PRFs

Kaishuo Cheng<sup>ID</sup> and Joseph Jaeger<sup>ID</sup>

School of Cybersecurity and Privacy  
Georgia Institute of Technology  
Atlanta, Georgia, USA

{kcheng89, josephjaeger}@gatech.edu  
<https://cc.gatech.edu/~kcheng89>  
<https://cc.gatech.edu/~josephjaeger>

**Abstract.** There is a gap between the security of constrained PRFs required in some applications and the security provided by existing definitions. This gap is typically patched by only considering nonadaptive security or manually mixing the CPRF with a random oracle (implicitly constructing a new CPRF) to achieve adaptive security. We fill this gap with a new definition for constrained PRFs with strong adaptive security properties and proofs that it is achieved by practical constructions based on the cascade PRF (which generalizes the GGM construction) and AMAC. We apply the definition for analyzing searchable symmetric encryption and puncturable key wrapping.

## Contents

|     |                                                                         |    |
|-----|-------------------------------------------------------------------------|----|
| 1   | Introduction . . . . .                                                  | 2  |
| 2   | Preliminaries . . . . .                                                 | 4  |
| 3   | (Constrained) PRFs: Old and New Definitions . . . . .                   | 6  |
| 3.1 | Function Families . . . . .                                             | 6  |
| 3.2 | Constrained Function Families . . . . .                                 | 7  |
| 3.3 | Simple Security Claims . . . . .                                        | 9  |
| 4   | Applications of SIM*-AC-CPRF Security . . . . .                         | 10 |
| 4.1 | Searchable Symmetric Encryption . . . . .                               | 10 |
| 4.2 | Puncturable Key Wrapping . . . . .                                      | 11 |
| 5   | Adaptively Secure Constructions . . . . .                               | 12 |
| 5.1 | Hashed Constrained PRF(s) . . . . .                                     | 12 |
| 5.2 | Composed Construction . . . . .                                         | 14 |
| 5.3 | Cascade-based Constructions . . . . .                                   | 16 |
| 5.4 | Augmented MAC . . . . .                                                 | 18 |
| A   | Proof of Theorem 1 (SIM*-AC-CPRF implies IND*-AC-CPRF) . . . . .        | 21 |
| B   | Searchable Symmetric Encryption Application . . . . .                   | 21 |
| C   | Puncturable Key Wrapping Application . . . . .                          | 28 |
| D   | Proofs for Hashed Constrained PRF(s) . . . . .                          | 34 |
| E   | Proof of Theorem 4 (Security of Composed Construction) . . . . .        | 41 |
| F   | Proof for Theorem 5 (Security of Cascade-based Constructions) . . . . . | 47 |
| G   | SIM*-AC-PRF Security With Leakage . . . . .                             | 49 |

## 1 Introduction

Constrained PRFs [9,12,25] are an advanced form of PRF that allows for delegation of the ability to evaluate the PRF on a subset of the domain. They have proven useful in a wide variety of settings including forward secure searchable encryption [10,11,17,25,30,31,33,34], encrypted file storage [2], session resumption for key exchange protocols [1,2], constructions using indistinguishable obfuscation [29], and vector commitments [3,4].

We argue that there is gap between the level of adaptivity provided by the standard security definitions for constrained PRFs and what is required of some applications. In some senses, the attacker is forced to commit to decisions about the nature of its queries before it “should” have to. This restriction hinders the analysis of protocols built based on constrained PRFs that require greater flexibility. We fill this gap with a new, strongly adaptive definition of security for constrained PRFs and show that this definition suffices for analyzing a forward secure searchable encryption scheme (SSE) [11] and a puncturable key wrapping scheme (PKW) [2] whose prior analyses ran into this gap. They bridged the gap, respectively, by manually mixing the constrained PRF with a random oracle or leaving the analysis to future work. We analyze the AMAC [5], cascade [6], and Goldreich-Goldwasser-Micali (GGM) [19] PRFs as constrained PRFs, giving modular proofs that they achieve our definition under appropriate idealization of their low-level components.

### 1.1 Our Contributions

**Constrained PRF definitions.** A normal PRF computes a function  $F(K, x) = y$  so that  $y$  should look random from the perspective of an attacker if  $K$  is uniformly random. A constrained PRF (CPRF) adds an extra functionality  $F.Con(K, R)$  that outputs a constrained key  $K_R$  which can be used to compute  $F(K, x)$  for any  $x \in R$ . Intuitively, security requires that challenge  $F(K, x)$  values be indistinguishable from random even if the attacker learns constrained keys  $K_R$ , as long as  $x \in R$  never holds. The standard security definition explicitly disallows making queries that would break this last condition. It is adaptive if the attacker can arbitrarily interweave its requests for challenge values and constrained keys and nonadaptive if all challenge requests must be made before constrain request. We call these IND\*-AC-CPRF (adaptive) and IND\*-NC-CPRF (non-adaptive) security respectively.

However, even IND\*-AC-CPRF security is not adaptive enough for some applications. Its strict restriction on overlapping challenge and constrain queries implicitly embeds a measure of non-adaptivity because the first time an adversary wants to access  $F(K, x)$  it must commit to whether this will be a challenge value (accessed by asking for the challenge) or constrained value (accessed by asking for the constrained key  $K_{\{x\}}$ ). In some senses, it philosophically captures that  $F(K, x)$  should look random if  $x$  is never constrained, but does not directly capture that  $F(K, x)$  should look random *until*  $x$  is constrained. This observation is not merely philosophical. This can break simulation-based proofs which require a simulator to commit to  $F(K, x)$ , without knowing what  $x$  is, and then later provide a constrained key for  $x$  after learning  $x$ . It can also break proofs wherein an attacker can somehow require a CPRF reduction to act based on  $F(K, x)$  before the attacker has committed to whether  $x$  will be challenged or constrained.

We aim to bridge this gap with a definition that we call SIM\*-AC-CPRF (simulation under adaptive compromise) security, which directly captures this philosophically appealing property. The adversary can arbitrarily interweaves requests for  $F(K, x)$  and  $K_R$  values with no restrictions. In the real world, it is honestly given these values. In the simulated world, the outputs of  $F(K, x)$  are sampled uniformly at random if a constrained key for  $x$  has not been given previously. A simulated is allowed to pick the constrained key (after being told the random outputs chosen for any previously queried  $x \in R$ ) and picks the output of  $F(K, x)$  if a constrain key for  $x$  has been given previously. This definition is intuitively similar to the “SIM-AC” definitions of Jaeger and Tyagi (e.g., SIM-AC-PRF [23] and SIM\*-AC-PRF [22]).

This definition would be impossible in the standard model by a simple counting argument (mirroring, e.g., proofs in [23,26]). However, it can be avoided in programmable ideal models (random oracle model, ideal cipher model, ...). The SIM-AC definitions of Jaeger and Tyagi allow the simulator to completely control the ideal primitive. We follow the SIM\*-AC definition of Jaeger [22] in which ideal primitive to have an explicit programming interface that *both* the adversary and simulator can query. This modification improves the composability of ideal model definitions [13,22]. Most of our proofs require this composability.

We envision the use of ideal model native definitions as best used as a sort of hedging. Typically one can prove some form of non-adaptive security of a protocol using IND\*-NC-CPRF security of the CPRF and

adaptive security using  $\text{SIM}^*\text{-AC-CPRF}$  security (and partially adaptive security using  $\text{IND}^*\text{-AC-CPRF}$ ). If both analyses come to similar conclusions (perhaps with proofs that “intuitively” use the same “core”), one might infer that this reflects the “true” relationship between the security of the protocol and the CPRF. If the idealization in some way break, we can at least fall back on the non-adaptive assurances of security.

**Applications.** To exhibit the usefulness of our new definition, we apply it to the analysis of a searchable symmetric encryption (SSE) scheme by Bost, Minaud, and Ohrimenko [11] and a puncturable key wrapping (PKW) scheme by Backendal, Günther, and Paterson [2]. Both use constrained PRFs to achieve forward security notions.

The analysis of the SSE scheme runs into the sort of issue we described for simulation-based proofs where a simulator must provide outputs to the CPRF without knowing the inputs, then later provide constrained keys after being told the inputs. Bost, et al. resolved this by hashing the output of the CPRF with a random oracle. This hashed CPRF essentially being used to achieve  $\text{SIM}^*\text{-AC-CPRF}$  security, so we prove security of the construction when the hashed CPRF is replaced with any  $\text{SIM}^*\text{-AC-CPRF}$  secure CPRF.

Backendal, Günther, and Paterson [2] analyze their PKW scheme in a setting where nonces are not allowed to repeat, but also motivate that it may be useful in a setting with nonce reuse. They point out that in the latter setting, the proof would run into the sort of issue we described above where a reduction must provide  $F(K, x)$  without knowing whether  $x$  will be challenged or constrained. Backendal, et al. explicitly note that using ideal models might solve this issue and leave analyzing this as future work. We provide this analysis, using showing security of their scheme in the nonce-reuse setting by assuming  $\text{SIM}^*\text{-AC-CPRF}$  security of the CPRF.

**Constructions.** We study the usefulness of  $\text{SIM}^*\text{-AC-CPRF}$  security from the other direction by proving that natural constructions of CPRFs indeed achieve such security (under appropriate assumptions of the lower-level building blocks).

*Hashed CPRF Constructions.* We first show that our SSE proof indeed modularizes the analysis of Bost, et al. by proving  $\text{SIM}^*\text{-AC-CPRF}$  security of  $H(F(K, \cdot))$  when  $H$  is a random oracle and  $F$  is a non-adaptive secure CPRF. We can construct a simulator that honestly picks a key for  $K$  and honestly simulates all uses of  $F$ . When a constrain query is made, it reprograms  $H$  to make  $H(F(K, x)) = z$  holds for any prior evaluation queries that returned  $z$  on input  $x$  (where  $x$  is in the constraint set). Security holds unless the adversary queries  $H$  on an unconstrained  $F(K, x)$  or two different outputs of  $F$  on different inputs collide. A reduction to  $\text{IND}^*$  security of the CPRF shows this is unlikely

*GGM and AMAC (Cascade Constructions).* The most commonly used CPRF (indeed the one proposed for use in the applications we consider) is based on the GGM pseudorandom function [19], so a natural question is whether it achieves  $\text{SIM}^*\text{-AC-CPRF}$  security assuming appropriate idealization. Then, e.g.,  $H(\text{GGM}(K, \cdot))$  in the SSE scheme could instead be replaced with simply  $\text{GGM}(K, \cdot)$  without the extra hash function. We will show that this is indeed the case, though rather than analyzing it directly, we instead capture it as a special case of the cascade construction of a PRF.

The cascade construction of Bellare, Canetti, and Krawczyk [6] iterates an underlying PRF. In particular,  $\text{CSC}(k_0, (x_0, \dots, x_n))$  computes  $k_{i+1} \leftarrow f(k_i, x_i)$  for  $i = 0, \dots, n$ , then returns  $k_{n+1}$ .<sup>1</sup> Used naively, this is vulnerable to length-extension attacks because the output of CSC on input  $\mathbf{x} = (x_0, \dots, x_n)$  can be used to compute the output of CSC for all inputs that  $\mathbf{x}$  is a prefix of. Bellare, et al. proved CSC is secure as long as its queries are “prefix free”. The vulnerability naturally leads CSC to being usable as a constrained PRF because  $\text{CSC}(k, \mathbf{x})$  can act as a constrained key for inputs that  $\mathbf{x}$  is a prefix of. This was observed in the initial works introducing constrained PRFs [9, 12, 25].  $\text{IND}^*\text{-NC-CPRF}$  security follows by a standard direct hybrid argument, while  $\text{IND}^*\text{-AC-CPRF}$  security has proven more complex; several works have aimed to more precisely understand its  $\text{IND}^*\text{-AC-CPRF}$  security [18, 21, 24]. Hofheinz, Kastner, and Klein [21] were finally able to establish that  $\text{IND}^*\text{-AC-CPRF}$  security follows from the PRF security of  $f$  using a complex rewinding argument, though the concrete bounds are worse than those of the  $\text{IND}^*\text{-NC-CPRF}$  proof.

<sup>1</sup> GGM is equivalent to the special case of this when  $x_i \in \{0, 1\}$ . The prior work on constrained PRFs specifically considered GGM, but to the best of our knowledge the results port over to CSC so we will discuss them as if they considered CSC instead.

We provide a proof that SIM\*-AC-CPRF security of CSC follows from SIM\*-AC-PRF security of  $f$ . The two proof provide analogous concrete security bounds to the IND\*-NC-CPRF proof (from different assumptions) and seem to “intuitively” use the same “core” ideas — though the SIM\*-AC proof is still noticeably more subtle. We view this as philosophical evidence that the challenge in writing IND\*-AC-CPRF proofs and the worsened concrete security is somehow an artifact of our current proof techniques, rather than a reflection of the “true” relationship of the security between CSC and its underlying PRF.

Our proof first considers the generic composition of two CPRFs  $F$  and  $H$ , as a construction  $\text{CMP}[F, H]$  which evaluates as  $F(H(K, x'), x'')$  on input  $(x', x'')$ . It allows direct constrains of  $K$  using  $H$  or indirect constrains that first evaluate  $K' = H(K, x')$  and then constrain  $K'$  using  $F$ . We show that SIM\*-AC-CPRF security of  $H$  and  $F$  imply that of the composition. Then we observe that CSC can be viewed as a repeated application of CMP via  $\text{CSC}_1 = f$  and  $\text{CSC}_{i+1} = \text{CMP}[f, \text{CSC}_i]$ . Thus we can use a hybrid argument to establish security of CSC.

Notably, a simplified version of the CMP proof provides the analogous result for IND\*-NC-CPRF while it is not clear how to write such a proof for IND\*-AC-CPRF security. Thus the simple CMP construction seems to contain at least some of the essential hardness of analyzing GGM’s IND\*-AC-CPRF security. The challenge arises when the adversary asks for the  $F$  key  $K' = H(K, x')$  to be constrained on a set  $R$ . The reductions don’t know whether the adversary will later make a challenge query for some  $(x', x'')$  with  $x'' \notin R$  (in which case  $K'$  needs to be switched to random) or request for  $H$  to constrain  $K$  on a set containing  $x'$  (in which case  $K'$  must not have been switched to random).

Next we consider AMAC. It is defined by  $\text{AMAC}(K, x) = \text{Chop}(H(K||x))$  where  $H$  is a hash function and  $\text{Chop}$  is an unkeyed output transform such as truncation. We focus on the special case whose PRF security was analyzed by Bellare, Bernstein, and Tessaro [5] wherein  $H$  is a MD hash function with compression function  $f(k, x)$  so  $\text{AMAC}(K, (x_0, \dots, x_n))$  computes  $k_1 \leftarrow f(x_0, K)$ , then  $k_{i+1} \leftarrow f(k_i, x_i)$  for  $i = 1, \dots, n$ , then returns  $\text{Chop}(k_{n+1})$ . I.e., it is basically the same as CSC except that the inputs for the first  $f$  call are flipped and  $\text{Chop}$  is applied to the output. Using  $\text{Chop}$  prevents length-extension attacks and maintains security even when inputs are prefixes of each other, but the  $k_i$ ’s can still be used as constrained keys. Bellare, et al. established security of AMAC as a PRF under appropriate assumptions on  $f$ .

We do the same for SIM\*-AC-CPRF of AMAC from SIM\*-AC-PRF security of  $f$ . In fact, we set up our formalism so that is essentially follows from the same analysis as CSC. We defined CMP so that on a “special” subset of  $H$ ’s domain it will merely output  $H(K, x')$  and not use  $F$ . (We prove security of CSC for arbitrarily prefix-free domains rather than a domain of all inputs of a fixed length by exploiting this feature of CMP as well.) Appropriate choice of this special subset captures AMAC as a repeated application of CMP.

## 1.2 Outline of the Paper

Section 2 provides general notation and specifies how we formalize programmable ideal models. Section 3 recalls existing security definitions for PRFs and CPRFs, then provides our new SIM\*-AC-CPRF notion for CPRFs. Section 4 summarizes our analysis of searchable symmetric encryption and puncturable key wrapping using SIM\*-AC-CPRF security. Section 5 proves the SIM\*-AC-CPRF security of specific CPRF constructions.

## 2 Preliminaries

**Sets, Tables, Lists.** The empty set is  $\emptyset$ . The size of set  $S$  is  $|S|$ . A table  $T$  is written via  $T[x] \leftarrow y$  and read via  $y \leftarrow T[x]$ . The empty table is  $[\cdot]$ . By  $T[S]$  or  $T_S$  we denote a new table  $T'$  in which  $T'[x] = T[x]$  if  $x \in S$  and  $T'[x] = \perp$  otherwise. The domain of  $T$ , denoted  $T.D$  is the set of entries  $x$  at which  $T[x] \neq \perp$ .

If  $D$  is a set, we define  $D^0 = \{\varepsilon\}$ ,  $D^i = D^{i-1} \times D$ ,  $D^{\leq i} = \bigcup_{j \leq i} D^j$ ,  $D^{< i} = D^{\leq i-1}$ , and  $D^* = \bigcup_{j \in \mathbb{N}} D^j$ . For  $x, y \in D^*$ , the expression  $x \sqsubseteq y$  denotes that  $x$  is a prefix of  $y$ . The empty list is  $\emptyset$ . If  $M$  is a list, then  $M.\text{add}(x_1, \dots, x_n)$  appends the tuple  $(x_1, \dots, x_n)$  to the end of the list.

**Pseudocode Conventions.** We use a pseudocode formalism inspired by Bellare and Rogaway [8]. If  $A$  is a stateful, then  $y \leftarrow A^O(x : \sigma)$  denotes assigning to  $y$  the output of  $A$  on input  $x$  and state  $\sigma$  with oracle access to  $O$ . The state is passed by reference, so  $A$  may internally update it. This notation is generalized to

multiple outputs, inputs, or oracles in a natural way. If  $A$  is randomized, then  $\leftarrow^*$  is used in place of  $\leftarrow$ . Sets, tables, and lists are initialized empty.

We use an asymptotic formalism with security parameter  $\lambda$  which is passed to algorithms as the unary string  $1^\lambda$ ; it is sometimes left implicit for compactness of notation. The probability game  $G$  outputs true when run on  $\lambda$  is  $\Pr[G(\lambda)]$ . Let  $f : \mathbb{N} \rightarrow \mathbb{R}$ . We say  $f$  is negligible if for all positive polynomials  $p$  there exists an  $n \in \mathbb{N}$  such that  $f(\lambda) \leq 1/p(\lambda)$  for all  $\lambda \geq n$ . We say  $f$  is super-polynomial if  $1/f$  is negligible.

If  $S$  is a set, then  $y \leftarrow^* S$  samples  $y$  uniformly from  $S$ . We often use a delayed sampling notation where  $K_{(\cdot)} \leftarrow^* S$  means that each  $K_u$  is generated by  $K_u \leftarrow^* S$  when first used.

We use  $\perp$  to indicate some form of failure. “Require  $B$ ” is shorthand for “If  $\neg B$  then return  $\perp$ ”, i.e.,  $B$  is required to be true to pass the require statement. Oracles that expect inputs from specific domains are implicitly assumed to immediately return  $\perp$  if given invalid inputs. We sometimes use “Abort  $v$ ” in the code for games and adversary. It means the current running process immediately halts and returns the corresponding value  $v$ .

**Programmable Ideal Models.** We work with Jaeger’s [22] syntax for programmable ideal primitives (random oracles, ideal ciphers, random permutations, etc.). An ideal primitive  $P$  specifies initialization algorithm  $P.\text{Init}$ , lazy sampling algorithms  $P.\text{Ls}$ , and programming algorithm  $P.\text{Prog}$ . Their syntaxes are  $\sigma_P \leftarrow^* P.\text{Init}(1^\lambda)$ ,  $y \leftarrow^* P.\text{Ls}(1^\lambda, x : \sigma_P)$ , and  $P.\text{Prog}(1^\lambda, x, y : \sigma_P)$ .

Security games run  $P.\text{Init}$  at the beginning. Adversaries and simulators are allowed to query  $P.\text{Ls}(1^\lambda, x : \sigma_P)$  or  $P.\text{Prog}(1^\lambda, x, y : \sigma_P)$  for  $x, y$  of their choice. For compactness, we write this as a single oracle  $\text{PPRIM}(\text{Op}, x, y)$  where  $\text{Op} \in \{\text{Ls}, \text{Prog}\}$  which runs  $y \leftarrow^* P.\text{Op}(1^\lambda, x, y : \sigma_P)$  and returns  $y$ . The extra input when  $\text{Op} = \text{Ls}$  is ignored. Adversary access to programming can be important for simulation-based proofs as it allows the adversary to internally run simulators which expect programming access [13, 22]. Cryptographic schemes may expect oracle access to  $P.\text{Ls}$ ; we write this as oracle access to  $P$  for compactness (in reductions, a schemes queries to  $P$  are be forwarded to the reductions  $\text{PPRIM}$  oracle if appropriate).

Jaeger [22] required that  $P.\text{Init}$  and  $P.\text{Ls}$  instantiate the lazy sampling of a function  $f : \mathcal{D}_\lambda \rightarrow \mathcal{R}_\lambda$  according to a distribution  $\mathcal{P}_\lambda$  where  $P.\text{Ls}(1^\lambda, x : \sigma_P)$  returns  $f(x)$ . Then programming function  $P.\text{Prog}(1^\lambda, x, y : \sigma_P)$  defines  $f(x) = y$  as long as that is consistent with the definition of function up to that point. The random oracle model then is defined by  $P_{\text{rom}}$  where  $P_{\text{rom}}.\text{Init}$  outputs an empty table  $\sigma_P = []$  and the other algorithms are as follows.

$$\begin{array}{l|l} \frac{P_{\text{rom}}.\text{Ls}(1^\lambda, x : \sigma_P)}{\text{If } \sigma_P[x] = \perp \text{ then } \sigma_P[x] \leftarrow^* \mathcal{R}_\lambda} & \frac{P_{\text{rom}}.\text{Prog}(1^\lambda, x, y : \sigma_P)}{\text{If } \sigma_P[x] = \perp \text{ then } \sigma_P[x] \leftarrow y} \\ \text{Return } \sigma_P[x] & \text{Return } \perp \end{array}$$

Most of our proofs treat  $P$  as a black box and are agnostic to the particular primitive in use. Only for low-level analysis (e.g., PRF security of compression functions) will our analysis need to directly reason about  $P$ .

Cryptographic schemes may require multiple ideal primitives. Let  $P_0$  and  $P_1$  be ideal primitives. We define  $P = P_0 \times P_1$  via the following algorithms. In words,  $P$  runs parallel copies of each primitive. Each input includes a bit to specify which primitives should be used.

$$\begin{array}{l|l|l} \frac{P.\text{Init}(1^\lambda)}{\sigma_0 \leftarrow^* P_0.\text{Init}(1^\lambda)} & \frac{P.\text{Ls}(1^\lambda, (d, x) : (\sigma_0, \sigma_1))}{y \leftarrow^* P_d.\text{Ls}(1^\lambda, x : \sigma_d)} & \frac{P.\text{Prog}(1^\lambda, (d, x), y : (\sigma_0, \sigma_1))}{P_d.\text{Prog}(1^\lambda, x, y : \sigma_d)} \\ \sigma_1 \leftarrow^* P_1.\text{Init}(1^\lambda) & \text{Return } y & \text{Return } \perp \\ \text{Return } (\sigma_0, \sigma_1) & & \end{array}$$

We say  $P$  is *consistent* if no combination of running  $P.\text{Ls}$  and  $P.\text{Prog}$  can make  $P.\text{Ls}(1^\lambda, x : \sigma_P)$  output  $y' \neq y$  if it previously output  $y$ . Consistency is useful for knowing that deterministic algorithms which query  $P.\text{Ls}$  will be consistent in their own input-output behavior. The kind of lazy sampling primitives considered by Jaeger are consistent by definition. We say  $P$  is *non-programmable* if  $P.\text{Prog}$  does not update  $\sigma_P$  and the order in which  $P.\text{Ls}$  is run on different inputs does not change its output distributions. The ideal encryption model [32] is an example for which the latter condition fails; this failure can make proofs using it subtly challenging [23].

|                                                                                              |                                                                                                           |
|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Game $G_{F,S,P,\mathcal{A}_{\text{prf}}}^{\text{sim}^*-\text{ac-prf}}(\lambda)$              | $\text{Ev}(u, x)$                                                                                         |
| $\sigma_P \leftarrow \$ P.\text{Init}(1^\lambda)$                                            | If $b = 1$ : $T_u[x] \leftarrow F.\text{Ev}^P(1^\lambda, K_u, x)$                                         |
| $b \leftarrow \$ \{0, 1\}$                                                                   | If $b = 0$ :                                                                                              |
| If $b = 1$ : $K_{(\cdot)} \leftarrow \$ F.K(\lambda)$                                        | If $K_u \neq \perp$ : $T_u[x] \leftarrow \$ S.\text{Ev}^{\text{PPRIM}}(1^\lambda, u, x, T_u[x] : \sigma)$ |
| If $b = 0$ : $\sigma \leftarrow \$ S.\text{Init}(1^\lambda)$                                 | Else if $T_u[x] \neq \perp$ : $T_u[x] \leftarrow T_u[x]$ // No-op                                         |
| $b' \leftarrow \$ \mathcal{A}_{\text{prf}}^{\text{Ev}, \text{EXP}, \text{PPRIM}}(1^\lambda)$ | Else $T_u[x] \leftarrow \$ F.\text{Out}(\lambda)$                                                         |
| Return $(b = b')$                                                                            | Return $T_u[x]$                                                                                           |
| $\text{PPRIM}(\text{Op}, x, y)$                                                              | $\text{EXP}(u)$                                                                                           |
| $y \leftarrow \$ P.\text{Op}(1^\lambda, x, y : \sigma_P)$                                    | If $b = 1$ : $K_u \leftarrow K_u$ // No-op                                                                |
| Return $y$                                                                                   | If $b = 0$ : $K_u \leftarrow \$ S.\text{Exp}^{\text{PPRIM}}(1^\lambda, u, T_u : \sigma)$                  |
|                                                                                              | Return $K_u$                                                                                              |

Fig. 1. Game defining the SIM\*-AC-PRF security of F

### 3 (Constrained) PRFs: Old and New Definitions

In this section we recall existing definitions and introduce new definitions for pseudorandom functions (Section 3.1) and constrained pseudorandom functions (Section 3.2).

#### 3.1 Function Families

A function family  $F$  specifies evaluation algorithm  $F.\text{Ev}$  together with sets  $F.K$ ,  $F.\text{Inp}$ , and  $F.\text{Out}$ . The syntax of its algorithm is  $y \leftarrow F.\text{Ev}^P(1^\lambda, K, x)$  where  $K \in F.K(\lambda)$  is a key,  $x \in F.\text{Inp}(\lambda)$  is an input, and  $y \in F.\text{Out}(\lambda, x)$  is an output. We let  $F.\text{Fcs}$  denote the set of all possible functions  $f$  with domain  $F.\text{Inp}$  for which  $f(x) \in F.\text{Out}(x)$  always holds. We omit the  $x$  in  $F.\text{Out}(x)$  when the range is the same for all  $x$ . For compactness, we occasionally write  $F(K, x)$  in place of  $F.\text{Ev}^P(1^\lambda, K, x)$ .

**SIM\*-AC-PRF Security.** SIM\*-AC-PRF security [22] of  $F$  is captured by the game shown in Fig. 1. In the “real world” ( $b = 1$ ), the attacker can ask for arbitrary PRF evaluations via  $\text{Ev}$  and ask to expose keys via  $\text{EXP}$ . In the “simulated world” ( $b = 0$ ), evaluation responses are random for unexposed users (because  $K_u = \perp$  holds). On exposure, the simulator is given all input-output pairs from prior  $\text{Ev}$  queries, then outputs a key. Its goal is to reprogram  $P$  to make the previous input-output pairs look consistent with  $F$  applied to this key. Intuitively, this definition ensures PRF security holds until a user is exposed. We use the “no-op” operation  $T_u[x] \leftarrow T_u[x]$  here (and similar “no-ops” elsewhere) just to emphasize that in this case we use the value which was already defined.

The advantage of  $\mathcal{A}_{\text{prf}}$  is  $\text{Adv}_{F,S,P,\mathcal{A}_{\text{prf}}}^{\text{sim}^*-\text{ac-prf}}(\lambda) = 2 \Pr[G_{F,S,P,\mathcal{A}_{\text{prf}}}^{\text{sim}^*-\text{ac-prf}}(\lambda)] - 1$ . If there exists a PPT  $S$  such that  $\text{Adv}_{F,S,P,\mathcal{A}_{\text{prf}}}^{\text{sim}^*-\text{ac-prf}}$  is negligible for all PPT  $\mathcal{A}_{\text{prf}}$ , then we say that  $F$  is SIM\*-AC-PRF secure with  $P$ . Random oracles and ideal ciphers are known to be SIM\*-AC-PRF secure [22,23].

By restricting  $\mathcal{A}_{\text{prf}}$  to never query  $\text{EXP}$  we capture the typical notion of PRF security (in the programmable  $P$  model). We call this IND\*-PRF security, use the superscript  $\text{ind}^*-\text{prf}$ , and omit the simulator (which is never called).

This definition is tweaked from the SIM\*-AC-PRF definition of [22] in that when  $b = 1$  the latter will not re-run  $F.\text{Ev}^P(1^\lambda, K_u, x)$  on repeated queries, instead it would output the stored value  $T_u[x]$ . This change is inspired by as of yet unpublished work. Therein we observe that the definitions are equivalent when  $P$  is consistent, but not when  $P$  is inconsistent. We argue that the definition we give here is more preferable for inconsistent  $P$ . Should not be considered to look like a random function if an attacker can cause its outputs to change (for unexposed keys).

**SIM\*-AC-PRF Security With Leakage.** Bellare, Bernstein, and Tessaro [5] used a variant of PRF security for analyzing the augmented cascade and MAC function, wherein the attacker was given leakage  $\text{Chop}(K_u)$  for each user where  $\text{Chop} : F.K \rightarrow \text{Chop.Out}$ . We will similarly need leakage for our SIM\*-AC analysis of these constructions.



We model leakage by adding an oracle  $\text{LEAK}(u)$ . This oracle will return  $\text{Chop}(K_u)$  whenever  $K_u \neq \perp$  (meaning always in the real world and only if the user has been exposed in the simulated world). Otherwise in the simulated world it returns a random value (which is repeated on repeat queries). The simulator is told this random value if  $u$  is later exposed. The code is as follows, where  $\ell_{(\cdot)} \leftarrow^s \text{Chop.Out}(\lambda)$  is added to the beginning of the game when  $b = 0$ .

$$\begin{array}{l|l} \text{LEAK}(u) & \text{EXP}(u) \\ \hline \text{If } b = 1: L \leftarrow \text{Chop}(1^\lambda, K_u) & \text{If } b = 1: K_u \leftarrow K_u \text{ // No-op} \\ \text{If } b = 0: L \leftarrow \ell_u & \text{If } b = 0: K_u \leftarrow^s \text{S.Exp}^{\text{PPRIM}}(1^\lambda, u, T_u, \ell_u : \sigma) \\ \text{Return } L & \text{Return } K_u \end{array}$$

Note that security with respect to this definition is equivalent to requiring  $\text{SIM}^*\text{-AC-PRF}$  security of the function family  $\text{F}_{\text{Chop}}$  which is identical to  $\text{F}$  except that we define  $\text{F}_{\text{Chop}}.\text{Ev}^{\text{P}}(1^\lambda, K, \varepsilon) = \text{Chop}(1^\lambda, K)$  where  $\varepsilon \notin \text{F.Inp}$ . For simplicity, we will directly use security of  $\text{F}_{\text{Chop}}$  in our results, rather than introducing explicit advantage function and security definition for  $\text{SIM}^*\text{-AC-PRF}$  with leakage.

In Appendix G, we port proofs of Bellare, Bernstein, and Tessaro [5] over to the  $\text{SIM}^*\text{-AC}$  setting to show that (under light combinatorial assumptions about  $\text{Chop}$ ) a random oracle achieves security with key leakage, as does the Davies-Meyer construction based on an ideal cipher. Along the way, we identify and fix a bug in their Davies-Meyer proof. Our concrete security bounds mirror those of [5], achieving better than trivial multi-user security which aids the security bounds we will prove for AMAC as a constrained PRF.

### 3.2 Constrained Function Families

Constrained PRFs were introduced independently by Boneh and Waters [9]; Kiayias, Papadopoulos, Triandopoulos, and Zacharias [25] (termed “delegatable PRFs”); and Boyle, Goldwasser, and Ivan [12] (termed “functional PRFs”). Our syntax is closest to Boneh and Waters’s definition. Constrained PRFs can create special “constrained” keys which allow evaluating the PRF on a restricted set of inputs while revealing nothing else.

**Syntax.** A constrained function family  $\text{CF}$  is a function family (with the syntax that follows) which additionally specifies a constrain algorithm  $\text{CF.Con}$ , a constrained evaluation algorithm  $\text{CF.CEv}$ , and a set system  $\text{CF.SS} \subseteq \mathcal{P}(\text{CF.Inp})$ . Here  $\mathcal{P}$  denotes the power set. The syntax of these algorithms is  $K_R \leftarrow \text{CF.Con}^{\text{P}}(1^\lambda, k, R)$  and  $y \leftarrow \text{CF.CEv}^{\text{P}}(1^\lambda, K_R, R, \mathbf{x})$ .

Here  $k \in \text{CF.K}(\lambda)$  is the root key,  $R \in \text{CF.SS}(\lambda)$  is a set,  $K_R$  is a constrained key for  $R$ ,  $\mathbf{x} \in R \subseteq \text{CF.Inp}(\lambda)$  is an input, and  $y \in \text{CF.Out}(\lambda)$  is an output. The desired behavior is that  $\text{CF.Ev}^{\text{P}}(1^\lambda, k, \mathbf{x}) = \text{CF.CEv}^{\text{P}}(1^\lambda, K_R, R, \mathbf{x})$  when  $K_R \leftarrow \text{CF.Con}^{\text{P}}(1^\lambda, k, R)$  for all  $\mathbf{x} \in R \in \text{CF.SS}(\lambda)$ . For technical reasons, we also need a query consistency property that the exact same set of oracle queries would be made on either side of this equality.

We find including  $R$  as an explicit input to constrained evaluation to be notationally convenient so that we do not have to encode it in  $K_R$ . If we cared about whether  $K_R$  hides the set  $R$  (“policy hiding” [25]), then this would be inappropriate. We assume that  $R$  is encoded as a bit string in a reasonable manner, e.g., checking  $\mathbf{x} \in R$  should be efficient given  $R$ .

When useful, we assume  $\text{CF.SS}$  contains the entire domain ( $K_{\text{CF.Inp}} = k$ ) and all singleton sets ( $K_{\{\mathbf{x}\}} = \text{CF.Ev}^{\text{P}}(1^\lambda, k, \mathbf{x})$ ). In security games this allows us to omit explicit oracles for revealing  $k$  or for non-challenge evaluation queries [9,25]. (Our Theorem 2 makes use of the latter.) One can also modify  $\text{CF}$  so that  $\text{CF.SS}$  is closed under unions ( $K_{R_1 \cup R_2} = (K_{R_1}, K_{R_2})$ ).

We say that  $\text{CF}$  is a prefix-constrained function family if  $\text{CF.Inp}$  is a subset of  $\text{D}^*$  for some set  $\text{D}$  and  $\text{CF.SS}$  contains all sets  $R_{x \sqsubseteq} = \{y \in \text{CF.Inp} \mid x \sqsubseteq y\}$  for  $x \in \text{D}^*$ . Recall  $x \sqsubseteq y$  denotes that  $x$  is a prefix of  $y$ . Natural choices are  $\text{CF.Inp} = \text{D}^*$ ,  $\text{CF.Inp} = \text{D}^n$ , or  $\text{CF.Inp} = \text{D}^{\leq n}$  for some integer  $n$ . It is well known [9,12,21,25,18] that the classic Goldreich-Goldwasser-Micali (GGM) construction of a PRF [19] naturally induces a prefix-constrained function family.

**Reconstrainability.** A constrained function family  $\text{CF}$  is *reconstrainable* if it additionally specifies a constrained constrain algorithm  $\text{CF.CCon}$  for placing further constraints on an already constrained key. Its syntax is  $K_{R'} \leftarrow \text{CF.CCon}^P(1^\lambda, K_R, R, R')$ . Here  $R' \subset R$  are sets in  $\text{CF.SS}(\lambda)$ ,  $K_R$  is a constrained key for  $R$ , and  $K_{R'}$  is a constrained key for  $R'$ .

A useful property of a *reconstrainable* constrained function family is *constrain invariance* which, roughly, requires that a constrained key for  $R'$  is the same whether it is output by  $\text{CF.Con}$  or  $\text{CF.CCon}$ .

**Constrained PRF Security.** The standard notion of constrained PRF security for a constrained function family  $\text{CF}$  (with a programmable ideal primitive) is shown by the game  $\text{G}^{\text{ind}^*-\text{ac-cprf}}$  defined in Fig. 2. The game captures that outputs of  $\text{CF}$  should look random, unless the attacker has learned a constrained key from which that output could be evaluated. We have written it in the standard single-bit, multi-user style.

In the game, the constrain oracle  $\text{CON}$  uses  $\text{CF.Con}$  to compute constrained keys. The evaluation (challenge) oracle  $\text{EV}$  uses  $\text{CF.Ev}$  in the “real world” ( $b = 1$ ) and the output of truly random functions  $f_u$  in the “random world” ( $b = 0$ ). The attacker’s goal is to distinguish which of these two cases is happening. To prevent trivial attacks, we use sets  $C_u \subseteq \text{CF.Inp}$  and  $E_u \subseteq \text{CF.Inp}$  for each user. The former tracks all points for which the adversary can compute  $\text{CF.Ev}$  via the constrained keys it has received. The latter tracks all points for which the adversary has received a challenge. Trivial attacks are prevented by maintaining the invariant that  $C_u \cap E_u = \emptyset$ .

The advantage of  $\mathcal{A}_{\text{cprf}}$  is  $\text{Adv}_{\text{CF}, \text{P}, \mathcal{A}_{\text{cprf}}}^{\text{ind}^*-\text{ac-cprf}}(\lambda) = 2 \Pr[\text{G}_{\text{CF}, \text{P}, \mathcal{A}_{\text{cprf}}}^{\text{ind}^*-\text{ac-cprf}}(\lambda)] - 1$ . If  $\text{Adv}_{\text{CF}, \text{P}, \mathcal{A}_{\text{cprf}}}^{\text{ind}^*-\text{ac-cprf}}$  is negligible for all PPT  $\mathcal{A}_{\text{cprf}}$ , then we say that  $\text{PF}$  is  $\text{IND}^*-\text{AC}-\text{CPRF}$  secure with  $\text{P}$ .

The above represents *adaptive* security, as the adversary can arbitrarily interweave constrain and evaluation queries. *Non-adaptive* security would require evaluation queries be made before constrain queries. This is captured by adding the boxed code to the game.<sup>2</sup> We call this  $\text{IND}^*-\text{NC}-\text{CPRF}$  security and use the superscript  $\text{ind}^*-\text{nc-cprf}$ .

**SIM\*-AC Constrained PRF Security.** While  $\text{IND}^*-\text{AC}-\text{CPRF}$  security is an *adaptive* security notion, it still embeds some inherent *non-adaptivity*. If an attacker needs to know  $y = \text{CF.Ev}(1^\lambda, k_u, \mathbf{x})$ , then it must commit to whether this will be a challenge point (by querying  $\text{EV}(u, \mathbf{x})$ ) or a non-challenge point that may be part of later constrain sets (by querying  $\text{CON}(u, \{\mathbf{x}\})$ ). This does not (conveniently) allow capturing the natural property that each  $y$  should look random up until a point of time when it is exposed by a constrain key. To capture this intuitive property, we cast constrained PRF security into the  $\text{SIM}^*-\text{AC}$  setting. Consider the security game  $\text{G}^{\text{sim}^*-\text{ac-cprf}}$  shown in Fig. 2 for constrained function family  $\text{CF}$ . In this code, for a set  $S$  we let  $f_u(S)$  denote some representation (e.g., as a table) of the restricted function  $f' : S \rightarrow \text{CF.Out}$  defined by  $f'(x) = f_u(x)$ .

The “real world” of this security game ( $b = 1$ ) is identical to that of  $\text{IND}^*-\text{AC}-\text{CPRF}$  except that no restrictions are place on what queries the adversary can make. In the “simulated world” ( $b = 0$ ) the outputs of both oracles are simulated, either being defined by the truly random functions  $f_u$  or chosen by a simulator  $\text{S}$ . In particular, whenever the evaluation on  $\mathbf{x}$  should not trivially be known by the adversary (i.e., when  $\mathbf{x} \notin C_u$ ) it is chosen at random. Otherwise the simulator is told the query, so it can simulate the response appropriately. The simulator always controls the output of constrain queries. To prevent trivial attacks, we reveal to the simulator the previously random outputs of evaluation on points which are contained in this constrain query. Intuitively, the definition requires that each  $y = \text{CF.Ev}(1^\lambda, k_u, \mathbf{x})$  looks random until a relevant constrain query is made, at which point the simulator makes it look like  $y$  was derived from the constrain key it chooses.

The advantage of  $\mathcal{A}_{\text{cprf}^*}$  is  $\text{Adv}_{\text{CF}, \text{S}, \text{P}, \mathcal{A}_{\text{cprf}^*}}^{\text{sim}^*-\text{ac-cprf}}(\lambda) = 2 \Pr[\text{G}_{\text{CF}, \text{S}, \text{P}, \mathcal{A}_{\text{cprf}^*}}^{\text{sim}^*-\text{ac-cprf}}(\lambda)] - 1$ . If there exists a PPT  $\text{S}$  such that  $\text{Adv}_{\text{CF}, \text{S}, \text{P}, \mathcal{A}_{\text{cprf}^*}}^{\text{sim}^*-\text{ac-cprf}}$  is negligible for all PPT  $\mathcal{A}_{\text{cprf}^*}$ , then we say that  $\text{CF}$  is  $\text{SIM}^*-\text{AC}-\text{CPRF}$  secure with  $\text{P}$ .

In Section 4, we analyze several applications of constrained function families and show they require full  $\text{SIM}^*-\text{AC}-\text{CPRF}$  security. In Section 5, we give examples of constrained function families that achieve  $\text{SIM}^*-\text{AC}-\text{CPRF}$  security (under appropriate assumptions on their building blocks).

<sup>2</sup> Note that this also allows evaluation queries after constrain queries if the evaluation query was already made previously. For consistent  $\text{P}$  the later query must anyway match the earlier one and for non-programmable  $\text{P}$  the later query could simply have been made earlier. For other  $\text{P}$ , the ability to make more queries potentially provides the adversary with extra powers.



|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Games <math>\mathcal{G}_{\text{CF},\text{P},\mathcal{A}_{\text{cprf}}}^{\text{ind}^*\text{-ac-cprf}}(\lambda), \boxed{\mathcal{G}_{\text{CF},\text{P},\mathcal{A}_{\text{cprf}}}^{\text{ind}^*\text{-nc-cprf}}(\lambda)}</math></p> <p><math>\sigma_{\text{P}} \leftarrow \text{P.Init}(1^\lambda)</math><br/> <math>k_{(\cdot)} \leftarrow \text{CF.K}(\lambda)</math><br/> <math>f_{(\cdot)} \leftarrow \text{CF.Fcs}(\lambda)</math><br/> <math>b \leftarrow \{0, 1\}</math><br/> <math>b' \leftarrow \mathcal{A}_{\text{cprf}}^{\text{CON},\text{Ev},\text{PPRIM}}(1^\lambda)</math><br/> Return <math>(b = b')</math></p> <p><math>\text{PPRIM}(\text{Op}, x, y)</math><br/> Return <math>\text{P.Op}(1^\lambda, x, y : \sigma_{\text{P}})</math></p>             | <p><math>\text{CON}(u, R)</math><br/> Require <math>R \cap E_u = \emptyset</math><br/> <math>K_R \leftarrow \text{CF.Con}^{\text{P}}(1^\lambda, k_u, R)</math><br/> <math>C_u \leftarrow C_u \cup R</math><br/> Return <math>K_R</math></p> <p><math>\text{Ev}(u, \mathbf{x})</math><br/> Require <math>\mathbf{x} \notin C_u</math><br/> <math>\boxed{\text{Require } C_u = \emptyset \text{ or } \mathbf{x} \in E_u}</math><br/> If <math>b = 1</math>: <math>y \leftarrow \text{CF.Ev}^{\text{P}}(1^\lambda, k_u, \mathbf{x})</math><br/> If <math>b = 0</math>: <math>y \leftarrow f_u(\mathbf{x})</math><br/> <math>E_u \leftarrow E_u \cup \{\mathbf{x}\}</math><br/> Return <math>y</math></p>                                                                                                                  |
| <p>Game <math>\mathcal{G}_{\text{CF},\text{S},\text{P},\mathcal{A}_{\text{cprf}^*}}^{\text{sim}^*\text{-ac-cprf}}(\lambda)</math></p> <p><math>\sigma_{\text{P}} \leftarrow \text{P.Init}(1^\lambda)</math><br/> <math>b \leftarrow \{0, 1\}</math><br/> If <math>b = 1</math>: <math>k_{(\cdot)} \leftarrow \text{CF.K}(\lambda)</math><br/> If <math>b = 0</math>: <math>\sigma \leftarrow \text{S.Init}(1^\lambda)</math><br/> <math>f_{(\cdot)} \leftarrow \text{CF.Fcs}(\lambda)</math><br/> <math>b' \leftarrow \mathcal{A}_{\text{cprf}^*}^{\text{CON},\text{Ev},\text{PPRIM}}(1^\lambda)</math><br/> Return <math>(b = b')</math></p> <p><math>\text{PPRIM}(\text{Op}, x, y)</math><br/> Return <math>\text{P.Op}(1^\lambda, x, y : \sigma_{\text{P}})</math></p> | <p><math>\text{CON}(u, R)</math><br/> If <math>b = 1</math>: <math>K_R \leftarrow \text{CF.Con}^{\text{P}}(1^\lambda, k_u, R)</math><br/> If <math>b = 0</math>: <math>K_R \leftarrow \text{S.Con}^{\text{PPRIM}}(1^\lambda, u, R, f_u(R \cap E_u \setminus C_u) : \sigma)</math><br/> <math>C_u \leftarrow C_u \cup R</math><br/> Return <math>K_R</math></p> <p><math>\text{Ev}(u, \mathbf{x})</math><br/> If <math>b = 1</math>: <math>y \leftarrow \text{CF.Ev}^{\text{P}}(1^\lambda, k_u, \mathbf{x})</math><br/> If <math>b = 0</math>: If <math>\mathbf{x} \in C_u</math>: <math>y \leftarrow \text{S.Ev}^{\text{PPRIM}}(1^\lambda, u, \mathbf{x} : \sigma)</math><br/> Else: <math>y \leftarrow f_u(\mathbf{x})</math><br/> <math>E_u \leftarrow E_u \cup \{\mathbf{x}\}</math><br/> Return <math>y</math></p> |

**Fig. 2.** Games defining constrained PRF security and SIM\*-AC-CPRF security of CF. Boxed code is only included for non-adaptive security. If  $S$  is a set,  $f_u(S)$  is the restriction of function  $f_u$  to domain  $S$ .

### 3.3 Simple Security Claims

**SIM\*-AC implies IND\*-AC.** We have described SIM\*-AC-CPRF security as if it is stronger than IND\*-AC-CPRF security. The following theorem formalized this. In the other direction, IND\*-AC-CPRF security will not imply SIM\*-AC-CPRF security as the former is achievable in the standard model [9,12,25] while a natural counting argument will show that the latter is not [23,26].

**Theorem 1.** *If CF is SIM\*-AC-CPRF secure with P, then it is IND\*-AC-CPRF secure with P. For all  $\mathcal{A}_{\text{cprf}}$ , we can build  $\mathcal{A}_{\text{cprf}^*}$  such that for all simulators S*

$$\text{Adv}_{\text{CF},\text{P},\mathcal{A}_{\text{cprf}}}^{\text{ind}^*\text{-ac-cprf}}(\lambda) = 2 \cdot \text{Adv}_{\text{CF},\text{S},\text{P},\mathcal{A}_{\text{cprf}^*}}^{\text{sim}^*\text{-ac-cprf}}(\lambda).$$

The main idea is that if  $\mathcal{A}_{\text{cprf}}$ 's queries are forwarded to  $\mathcal{A}_{\text{cprf}^*}$ 's oracles, then the restrictions placed on  $\mathcal{A}_{\text{cprf}}$ 's queries will mean that the else branch in CON for SIM\*-AC-CPRF will never be triggered so  $y$  is always  $f_u(\mathbf{x})$ , which will not be used elsewhere. The proof is delayed to Appendix A.

**CPRFs from PRFs.** Let  $\mathbf{F}$  be a function family. Then we will sometimes view  $\mathbf{F}$  as a simple constrained function family whose set system is only the entire domain ( $K_{\text{CF.Inp}} = k$ ) and all singleton sets ( $K_{\{\mathbf{x}\}} = \text{CF.Ev}^{\text{P}}(1^\lambda, k, \mathbf{x})$ ), with the natural constrained evaluation algorithm. The proof of the following is straightforward.

**Proposition 1.** *For  $X \in \{\text{IND}^*\text{-NC}, \text{IND}^*\text{-AC}, \text{SIM}^*\text{-AC}\}$ ,  $\mathbf{F}$  is  $X$ -CPRF secure with P if and only if it is  $X$ -PRF secure with P.*

## 4 Applications of SIM\*-AC-CPRF Security

We exhibit the usefulness of our SIM\*-AC-CPRF definition by describing its use in two applications.

### 4.1 Searchable Symmetric Encryption

Searchable symmetric encryption (SSE) aims to store databases of documents in an encrypted form that nonetheless allows search queries to be performed to identify all documents that are associated with a given keyword. The most efficient protocols aim to have search only take a single round trip. The user sends a search token  $\tau_s$  for keyword  $w$  to the server who then uses an encrypted index to recover all document indices  $ind$  associated with  $w$ . The server can then send the document ciphertexts associated with each index back to the user who decrypts the ciphertexts to read their document. Such one-round protocols lead to adaptive security challenges in the standard simulation-based definition introduced by Curtmola et al. [16].

**A Common Design Pattern.** A common design paradigm for SSE schemes involves associating an encryption key  $ek_w$  and a PRF key  $k_w$  with each keyword. The encryption key encrypts the document identifiers and the PRF key determines random-looking places that these ciphertexts will be stored.

A clean example of this is the  $\Pi_{\text{bas}}$  scheme of Cash et al. [14] where (roughly) the  $i$ -th document identifier  $id$  associated with  $w$  is stored in a dictionary via  $T[F(k_w, i)] \leftarrow \text{Enc}(ek_w, id)$ . Given  $ek_w$  and  $k_w$ , the server loops over  $i$ , recovering each ciphertext from  $T$  and decryption. Cash et al. [14] proved non-adaptive security of this construction assuming IND-PRF security and that the encryption provides indistinguishability from random strings and sketched an adaptive proof based on random oracles.

Jaeger and Tyagi [23] identified a number of SSE schemes that used similar techniques and showed that SIM-AC-PRF security of the function family and SIM-AC-KP (a “key private” variant of CPA security in the SIM-AC style) security of encryption would have sufficed in the analysis, allowing simpler and more modular proofs. As an example, they gave a proof for  $\Pi_{\text{bas}}$ .

**Adding Forward Security.** More advanced constructions of SSE aim for strong security properties like forward security. Searching for keyword  $w$  at time  $t$  should not allow the server to use the search token to identify documents associated with the keyword that are added to the database later. Constrained PRFs have shown themselves to be a natural tool for achieving this and other properties for SSE [10,11,17,25,30,31,33,34].

Perhaps the canonical example of constructing a forward-secure SSE scheme from constrained PRFs is the FS-RCPRF scheme of Bost, Minaud, and Ohrimenko [11] which we show in Fig. 3 (with some details omitted or simplified). The construction is based on two hash functions  $H_0$  and  $H_1$  and a CPRF  $\tilde{F}$  with  $\tilde{F}.\text{Inp} = \mathbb{N}$  which can constrain its key on sets  $[c] = \{0, \dots, c\}$ .

The server stores a table EDB mapping from pseudorandom labels  $UT$  to ciphertexts  $e$  which contain the document identifier. The client stores keys  $K_w$  and  $K'_w$  for each  $w$  and a table  $\mathbf{W}$  with a counter of how many documents associated with  $w$  are already stored in the database. The update protocol allows the client to add a new document identifier  $ind$  to keyword  $w$ . It applies  $\tilde{F}$  with  $K_w$  to the next counter value to obtain an intermediate value  $T$  which is hashed together with  $K'_w$  to derive  $UT$  and encrypt  $ind$  into  $e$ . The update token  $\tau_u = (UT, e)$  is then stored by the server in EDB.

To perform a search for keyword  $w$ , the client constrains  $K_w$  to only be valid on the counter values that have been used for  $w$  so far. This, together with  $K'_w$  allows the server to recompute the  $UT$  values, then recover and decrypt the ciphertexts  $e$ .

A natural question is why this construction needs the hash functions at all. Shouldn't the output of  $\tilde{F}$  already look random for our purposes? The answer is that the hashes (modelled as random oracles) are incorporated to allow an adaptive security proof where the simulator commits to a random EDB and then later reprograms these random oracles to make it look consistent with the attacker's queries.

We view the extra hashes as, in essence, a way of lifting from non-adaptive to adaptive (in particular SIM\*-AC-CPRF) security. However, with this perspective it is natural to omit the hashes and view the construction in terms of a single SIM\*-AC-CPRF secure function. We capture this in the construction FS2 shown on the bottom of the figure. We additionally removed  $K'_w$  entirely (it does not seem to add anything and was unused in the security analysis of FS-RCPRF) and replace one-time pad encryption with arbitrary

|                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <u>FS-RCPRF.Update(<math>w, ind</math>)</u><br><i>Client:</i><br>Get $K_w, K'_w$<br>$c \leftarrow \mathbf{W}[w]; \mathbf{W}[w] \leftarrow c + 1$<br>$T \leftarrow \tilde{\mathbf{F}}.\text{Ev}(K_w, c + 1)$<br>$UT \leftarrow H_0(K'_w, T)$<br>$e \leftarrow ind \oplus H_1(K'_w, T)$<br>Send $(UT, e)$ to the server<br><i>Server:</i><br>$\text{EDB}[UT] \leftarrow e$ | <u>FS-RCPRF.Search(<math>w</math>)</u><br><i>Client:</i><br>Get $K_w, K'_w$<br>$c \leftarrow \mathbf{W}[w]$<br>$k_{[c]} \leftarrow \tilde{\mathbf{F}}.\text{Con}(K_w, [c])$<br>Send $(K'_w, k_{[c]}, c)$ to the server<br><i>Server:</i><br>For $i = c$ to 0:<br>$T \leftarrow \tilde{\mathbf{F}}.\text{CEv}(k_{[c]}, [c], i)$<br>$UT \leftarrow H_0(K'_w, T)$<br>$e \leftarrow \text{EDB}[UT]$<br>$ind_i \leftarrow e \oplus H_1(K'_w, T)$<br>Send document $ind_i$ |
| <u>FS2.Update(<math>w, ind</math>)</u><br><i>Client:</i><br>Get $K_w$<br>$c \leftarrow \mathbf{W}[w]; \mathbf{W}[w] \leftarrow c + 1$<br>$(UT, ek) \leftarrow \tilde{\mathbf{F}}.\text{Ev}(K_w, c + 1)$<br>$e \leftarrow \text{SE}.\text{Enc}(ek, ind)$<br>Send $(UT, e)$ to the server<br><i>Server:</i><br>$\text{EDB}[UT] \leftarrow e$                               | <u>FS2.Search(<math>w</math>)</u><br><i>Client:</i><br>Get $K_w$<br>$c \leftarrow \mathbf{W}[w]$<br>$k_{[c]} \leftarrow \tilde{\mathbf{F}}.\text{Con}(K_w, [c])$<br>Send $(k_{[c]}, c)$ to the server<br><i>Server:</i><br>For $i = c$ to 0:<br>$(UT, ek) \leftarrow \tilde{\mathbf{F}}.\text{CEv}(k_{[c]}, [c], i)$<br>$e \leftarrow \text{EDB}[UT]$<br>$ind_i \leftarrow \text{SE}.\text{Dec}(ek, e)$<br>Send document $ind_i$                                     |

**Fig. 3.** SSE scheme FS-RCPRF and our simplification/modularization FS2. Differences to FS-RCPRF are highlighted.

one-time secure encryption scheme. This allows, e.g., swapping in standard encryption schemes that provide authentication.

In Appendix B, we formalize this construction and prove that it is secure assuming  $\tilde{\mathbf{F}}$  is SIM\*-AC-CPRF security and SE is SIM\*-AC-CPA secure (a CPA security notion in the SIM\*-AC style). This has several benefits. The proof is easier than the security proof for FS-RCPRF, in particular avoiding the need to handle random oracle reprogramming analysis in the middle of an already complicated proof. Indeed, there is a minor bug in this step of the Bost, et al. [11] proof. (They did not consider the possibility that two different honestly generated  $T$ 's could collide.)

The result provides modularity; we have more precisely identified exactly what properties are required of the subcomponents so that implementers have more flexibility. To capture the original scheme (without  $K'_w$ ), in Section 5 we formalize and analyze hashing the output of a non-adaptively secure CPRF to obtain an adaptively secure one. This works as long as the adversary's queries are restricted to an a priori known small (polynomial sized) domain. In FS-RCPRF, it is bounded by the number of update operations performed. The primary instantiation proposed by Bost, et al., called Diana, uses  $\tilde{F}$  based on the Goldreich, Goldwasser, Micali (GGM) CPRF [9,12,19,25]. Our separate analysis of this construction in Section 5 (as a special case of the cascade construction) shows that it achieves SIM\*-AC-CPRF security and the extra hashes in Diana were unnecessary.

We believe that our definition would have been applicable to a number of other SSE constructions that use similar techniques [17,30,33,34].

## 4.2 Puncturable Key Wrapping

Next we consider puncturable key wrapping. Backendal, Günther, and Paterson [2] introduced puncturable key wrapping (PKW) as a means to add forward security to key wrapping and then used this to analyze a

variant of the session resumption mechanism in the pre-shared key mode of TLS 1.3 introduced by Aviram, Gellert, and Jager [1] and an encrypted file storage system that they specify.

A puncturable key wrapping scheme PKW provides a key generation algorithm  $\text{PKW.Kg}$ , wrapping (i.e., encryption) algorithm  $\text{PKW.Wrap}$ , unwrapping (decryption) algorithm  $\text{PKW.Unwrap}$ , and puncturing algorithm  $\text{PKW.Punc}$ . The syntaxes of these algorithms are  $K \leftarrow \text{PKW.Kg}(1^\lambda)$ ,  $c \leftarrow \text{PKW.Wrap}^P(1^\lambda, K, t, h, m)$ ,  $m \leftarrow \text{PKW.Unwrap}^P(1^\lambda, K, t, h, c)$ , and  $K' \leftarrow \text{PKW.Punc}^P(K, t)$ . Here  $t$  is a tag (nonce),  $h$  is a header to be authenticated (associated data), and  $m$  is a message (which is expected to be a key, though the formalism works equally well for the encryption of arbitrary data) to be wrapped. The unwrapping algorithm recovers the message from the ciphertext. The puncturing algorithm creates an updated key which is no longer able to encrypt or decrypt with tag  $t$  and can be applied multiple times.

Backendal, et al. introduced several integrity and confidentiality definition of PKW schemes. We focus on their strongest confidentiality definition  $\text{FIND\$-RCPA}$  which allowed the attacker to ask for real wrappings, real or random challenge wrapping, and punctured keys of their chosen with the restriction that they can never repeat tags against wrapping queries or ask for keys which have not been punctured on all of the tags that will be used for challenge wrappings. They propose a natural construction  $\text{PKW}[\text{CF}, \text{NE}]$  which applies a (reconstruable) constrained PRF to the tag to derive a subkey which is then used with an AEAD scheme to perform the actual encryption. They then prove that  $\text{PKW}[\text{CF}, \text{NE}]$  is  $\text{FIND\$-RCPA}$  secure assume  $\text{CF}$  is  $\text{IND-AC-CPRF}$  secure and  $\text{NE}$  achieves the standard  $\text{IND\$-CPA}$  security notion for nonce-based encryption [27].<sup>3</sup>

However, Backendal, et al. also note that their formalism does not allow the use of “batch puncturing” [20] (removing the ability to decrypt a collection of ciphertexts with a single puncture operation) because  $\text{FIND\$-RCPA}$  provides no assurances if the same tag is used for multiple ciphertexts. Backendal et al. propose but do not formally analyze a stronger notion where tag reuse is allowed [2, Footnote 2, ePrint]. They point out that misuse-resistant security of  $\text{NE}$  [28] would be a necessary condition for  $\text{PKW}[\text{CF}, \text{NE}]$  to be secure in this model. Moreover, they observe that the natural proof strategy would run into a commitment problem. When an adversary asks for a real encryption of  $(t, h, m)$ , we want to provide a real encryption if a key capable of decrypting nonce  $t$  will later be revealed or switch the encryption to random if  $t$  will later be used for a challenge encryption (preventing such a key being revealed). Backendal et al. proposed that the stronger notion may be achieved by using ideal model analysis (citing the  $\text{SIM-AC}$  paper [23]), but leave doing so for future work. We confirm this belief. Interestingly, in our analysis we require  $\text{SIM}^*\text{-AC}$ -based idealization of *both*  $\text{CF}$  and  $\text{NE}$ . We give the formal details in Appendix C.

## 5 Adaptively Secure Constructions

In this section, we analyze the security of several constrained function families with respect to our new  $\text{SIM}^*\text{-AC-CPRF}$  definition. In Section 5.1, we analyze the construction that is used in  $\text{FS-RCPRF}$  from Bost, et al. [11]. Then in Section 5.2, we analyze the construction obtained from composing two constrained PRF together. This allows us to analyze cascade construction in Section 5.3. Finally, we analyze  $\text{AMAC}$  in Section 5.4, which can be viewed as an instantiation of augmented cascade that we considered in Section 5.3.

### 5.1 Hashed Constrained PRF(s)

We start with the construction we view as being implicit in the searchable encryption scheme  $\text{FS-RCPRF}$  from Bost, et al. [11] which lifts non-adaptive constrained PRF security to be adaptive. Related constructions are also discussed in [30]. In essence, if  $\text{CF}$  is a constrained function family with key  $K$  and  $H$  is a hash function modeled as a random oracle with key  $K'$ , their construction computes outputs via  $H(K', \text{CF}(K, \cdot))$ .

The second  $K'$  does not seem to provide much additional value (e.g., the security proof of  $\text{FS-RCPRF}$  [11] does not make use of this key). Thus we will simplify and obtain the construction  $\text{Hashed}[\text{CF}]$  defined identically to  $\text{CF}$  except that  $\text{CF.Out}$  is always the range of random oracle  $\text{P}_{\text{rom}}$  and evaluation/constrained

<sup>3</sup> Technically, they formalize this in terms of a puncturable PRF. This is essentially just a difference of syntax, though the security notion they use for it is technically slightly weaker than  $\text{IND}^*\text{-AC-CPRF}$  security because it does not allow the attacker to access as many constrained keys.

|                                               |                                                                                                              |                                                           |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| $\text{S.Init}(1^\lambda)$                    | $\text{S.Ev}^{\text{PPRIM}}(1^\lambda, u, x : k)$                                                            | $\text{S.Con}^{\text{PPRIM}}(1^\lambda, u, R, f : k)$     |
| $k_{(\cdot)} \leftarrow \text{CF.K}(\lambda)$ | $y \leftarrow \text{Hashed}[\text{CF}].\text{Ev}^{\text{P} \times \text{P}_{\text{rom}}}(1^\lambda, k_u, x)$ | For $x \in f.D$ :                                         |
| Return $k_{(\cdot)}$                          | Return $y$                                                                                                   | $Y \leftarrow \text{CF.Ev}^{\text{P}}(1^\lambda, k_u, x)$ |
|                                               |                                                                                                              | $\text{PPRIM}(\text{Prog}, (1, Y), f(x))$                 |
|                                               |                                                                                                              | Return $\text{CF.Con}^{\text{P}}(1^\lambda, k_u, R)$      |

**Fig. 4.** Simulator  $\text{S}$  for the  $\text{SIM}^*$ -AC-CPRF security proof of  $\text{Hashed}[\text{CF}]$  (Theorem 2)

evaluation both apply  $\text{P}_{\text{rom}}$ 's to their output.

$$\text{Hashed}[\text{CF}].\text{Ev}^{\text{P} \times \text{P}_{\text{rom}}}(1^\lambda, k, \mathbf{x}) = \text{P}_{\text{rom}}.\text{Ls}(1^\lambda, \text{CF.Ev}^{\text{P}}(1^\lambda, k, \mathbf{x}) : \sigma_{\text{P}})$$

$$\text{Hashed}[\text{CF}].\text{CEv}^{\text{P} \times \text{P}_{\text{rom}}}(1^\lambda, K_R, R, \mathbf{x}) = \text{P}_{\text{rom}}.\text{Ls}(1^\lambda, \text{CF.CEv}^{\text{P}}(1^\lambda, K_R, R, \mathbf{x}) : \sigma_{\text{P}})$$

Security of this construction boils down to arguing that the attacker cannot query  $\text{P}_{\text{rom}}$  (for lazy sampling or programming) on the outputs of  $\text{CF}$  and that outputs of  $\text{CF}$  do not collide before the attacker trivially learned the outputs via constrain queries. This should be hard because these output of  $\text{CF}$  should look random. To prove security of this construction based on non-adaptive security of  $\text{CF}$ , we will need to guess ahead of time the input to  $\text{CF}$  whose output the attacker will query to  $\text{P}_{\text{rom}}$  and so security will require restricting attention to attacks that only query for evaluation from sets of polynomial size. In the case of FS-RCPRF [11] each attacker would only learn  $\text{CF}$  outputs for inputs from a set  $\{1, \dots, c\}$  where  $c$  corresponded to the maximum number of documents associated with a given keyword in an attacker chosen database. This value could depend on the attacker, but was necessarily polynomial. The following theorem captures the security of this construction.

**Theorem 2.** *If  $\min_x |\text{CF.Out}(\lambda, x)|$  is super-polynomial,  $\text{CF}$  is  $\text{IND}^*$ -NC-CPRF secure with  $\text{P}$ , and  $\text{P}$  is non-programmable; then  $\text{Hashed}[\text{CF}]$  is  $\text{SIM}^*$ -AC-CPRF secure with  $\text{P} \times \text{P}_{\text{rom}}$  against adversaries whose  $\text{Ev}$  queries are restricted to a set of polynomial size.*

The simulator we construct for proving this theorem is given in Fig. 4. It simulates the use of  $\text{CF}$  honestly and in  $\text{S.Con}$  attempts to reprogram the random oracle to make it consistent with the values returned by earlier evaluation queries. The simulation will fail if  $\text{P}_{\text{rom}}$  is queried on  $y = \text{CF.Ev}^{\text{P}}(1^\lambda, k_u, \mathbf{x})$  for an unconstrained  $x$ . This can be caused by a direct lazy sampling or programming query by the attacker or if a unrelated evaluation query happened to compute the same  $y$ . (The latter condition was missed in security proof for FS-RCPRF [11].) To bound this via a non-adaptive reduction, which knows the set of queries  $N$  that the adversary may make, will pick a random  $(u, \mathbf{x}) \leftarrow N(\lambda)$  then query  $\text{Ev}(u, \mathbf{x})$  at the start of the game to get  $\mathbf{y}$ , and then simulate the rest of the game (aborting early if  $(u, x)$  will be constrained). It returns 1 if  $\mathbf{y}$  is queried to  $\text{P}_{\text{rom}}$  (directly or via a different  $\text{Ev}$  query). The technical condition that  $\text{P}$  be non-programmable arises from the fact that the reduction will obtain  $\mathbf{y}$  at the start of the game (so that it is non-adaptive) even though it may not actually be required until a later query. If  $\text{P}$  were programmable, then it's possible that queries to  $\text{P}$  before the later query should have changed the distribution of  $\mathbf{y}$ . We defer the proof to Appendix D.1. This theorem does not allow  $\text{CF}$  to depend on the  $\text{P}_{\text{rom}}$  used by  $\text{Hashed}[\text{CF}]$ .

**Instantiations.** The searchable encryption schemes Diana [11] and  $\Sigma\phi\phi\zeta$  [10] can be viewed as essentially using instantiations of hashed CPRF. Diana uses GGM as the underlying constrained PRF, therefore it can be analyzed as a  $\text{Hashed}[\text{CF}]$  and security follows from the known non-adaptive constrained PRF security of GGM [9,12,21,25,18]. However, in Section 5.3 we will show that the cascade construction (a generalization of GGM) is already  $\text{SIM}^*$ -AC-CPRF secure, assuming  $\text{SIM}^*$ -AC-PRF security of the underlying building block. So GGM could have been used directly, without hashing. Moreover, relying on that result will provide stronger concrete security guarantees because it avoids the guessing argument.

$\Sigma\phi\phi\zeta$  is analyzed as an instantiation of FS-RCPRF by Bost, et al. [11] by constructing constrained PRF using iterations of a trapdoor permutation (TDP). The proof strategy claimed by Bost, et al. is flawed, as the constructed CPRF is not secure. However, the overall hashed construction is nonetheless secure as we will now show.



**Trapdoor permutation constrained “PRF”.** We consider the construction in [11]. Let  $\Pi$  be a trapdoor permutation with trapdoor  $tk$  and public information  $pub$ . For all valid input  $ST$ ,  $\Pi_{pub}(ST)$  computes the permutation using  $pub$ , and  $\Pi_{tk}^{-1}(ST)$  computes the inverse using  $tk$ . Let  $\Pi^n$  or  $\Pi^{-n}$  denote the  $n$ -fold iteration of these algorithms. (More formal definitions will be given for the coming security result.) Then the constrained function family  $\tilde{F}$  is as follows. Its key  $k$  is composed of  $tk$  and some initial secret input  $ST_0$ . The constrain sets are  $R_n = \{1^0, \dots, 1^n\} = \{1^i : i \leq n\}$ . Its algorithms are:

$$\begin{aligned}\tilde{F}.\text{Ev}(1^\lambda, k, 1^c) &= \Pi_{tk}^{-c}(ST_0) \\ \tilde{F}.\text{Con}(1^\lambda, (tk, ST_0), R_n) &= (pub, \Pi_{tk}^{-n}(ST_0)) \\ \tilde{F}.\text{CEv}(1^\lambda, (pub, ST_n), R_n, c) &= \Pi_{pub}^{n-c}(ST_n).\end{aligned}$$

Bost, et al. specifically considered an RSA-based trapdoor permutation and claim, “We can easily reduce the constrained-PRF security of  $\tilde{F}$  to the hardness of the RSA assumption, and directly deduce the security of  $\Sigma\text{of}\phi\phi\zeta$ ” [11, p.12, ePrint].

This is incorrect;  $\tilde{F}$  is not secure as a CPRF. Consider the following attack against IND\*-NC-CPRF security. The adversary  $\mathcal{A}$  first queries  $ST_2 \leftarrow \text{Ev}(u, 1^2)$ . Then it queries  $(pub, ST_1) \leftarrow \text{Con}(u, R_1)$ . It outputs 1 iff  $\Pi_{pub}(ST_2) = ST_1$ . This adversary always outputs 1 in the real world but will only output 1 in the random world with probably  $1/N$ , where  $N$  is the size of the permutation’s range. So its advantage is  $1 - 1/N$ . We emphasize that while  $\tilde{F}$  is insecure, it does not mean  $\Sigma\text{of}\phi\phi\zeta$  is insecure — just that its security is not captured by the proof flow being proposed by Bost, et al. A separate, direct proof was given by Bost [10] when originally proposing  $\Sigma\text{of}\phi\phi\zeta$ .

For our purposes, we will show that  $\text{Hashed}[\tilde{F}]$  (which we will call TDCF) is still secure despite  $\tilde{F}$  not being a secure CPRF. The above attack is prevented because the output of  $\text{Ev}$  is hashed first. All we need of  $\tilde{F}$ ’s outputs are that they are hard to guess, not that they are indistinguishable from random. This is captured by the following theorem.

**Theorem 3.** *If the trapdoor permutation is secure with  $P$  and  $P$  is consistent and non-programmable, then TDCF is SIM\*-AC-CPRF secure with  $P \times P_{\text{rom}}$ .*

The proof is given in Appendix D.2 as a modification of the proof of Theorem 2 where we use the security of the trapdoor permutation in the guessing argument to show that the adversary cannot query the random oracle early on values the simulator will later program it at. Because  $\text{CF.Inp} = 1^*$  the assumption that adversaries  $\text{Ev}$  queries are restricted to a set of polynomial size (required for Theorem 2) follows from the adversary being PPT.

## 5.2 Composed Construction

As discussed above, the searchable encryption scheme Diana [11], essentially, instantiates the CPRF of  $\text{Hashed}[\text{CF}]$  using the GGM constrained PRF. The known IND\*-NC-CPRF security of GGM plus the analysis of  $\text{Hashed}[\text{CF}]$  essentially shows security of Diana. (More formally, using this analysis and our analysis of FS2 gives a security proof for a variant of Diana.)

We want to show that GGM itself is SIM\*-AC-CPRF secure, and therefore the extra hash at the end is not needed. We start here by analyzing when composing two secure CPRFs gives a larger secure CPRF. In Section 5.3 we will show that GGM (and related constructions) can be viewed as being obtained from repeated applications of this composition.

In our composed construction, we will use function  $H$  to derive a key for the second function  $F$  as in  $F(H(k, x'), x'')$ . For our later use, it will be useful to preserve some of the domain of  $H$  (called  $H.\text{Inp}_1$ ) for direct use of  $H$  without applying  $F$ .

**Definition 1 (Composed Construction).** *Let  $F$  and  $H$  be function families with  $H.\text{Inp} = H.\text{Inp}_1 \sqcup H.\text{Inp}_2$  where  $H.\text{Out}(x) = F.K$  for all  $x \in H.\text{Inp}_2$ . Then  $\text{CMP}[F, H]$  has  $\text{CMP}[F, H].K = H.K$ ,  $\text{CMP}[F, H].\text{Inp} = H.\text{Inp}_1 \sqcup (H.\text{Inp}_2 \times F.\text{Inp})$ ,  $\text{CMP}[F, H].\text{Out}(x) = H.\text{Out}(x)$  for  $x \in H.\text{Inp}_1$ ,  $\text{CMP}[F, H].\text{Out}((x', x'')) = F.\text{Out}(x'')$  for  $(x', x'') \in H.\text{Inp}_2 \times F.\text{Inp}$  and*

$$\text{CMP}[F, H].\text{Ev}^P(k, x) = \begin{cases} H.\text{Ev}^P(k, x), & x \in H.\text{Inp}_1 \\ F.\text{Ev}^P(H.\text{Ev}^P(k, x'), x''), & x = (x', x'') \in H.\text{Inp}_2 \times F.\text{Inp}. \end{cases}$$

The use of  $\sqcup$  represents a requirement that the union be disjoint.

Suppose that both  $H$  and  $F$  are constrained function families. Then we can make  $\text{CMP}[F, H]$  into a constrained function family by constraining the key with respect to  $H$  or applying  $H$  for some input in  $H.\text{Inp}_2$  to get a key for  $F$ , then constraining that key with  $F$ . The first type allows us to constrain  $\text{CMP}[F, H]$  for any sets  $R_S = (S \cap H.\text{Inp}_1) \cup ((S \cap H.\text{Inp}_2) \times F.\text{Inp})$  where  $S \in H.\text{SS}$ . The second type allows us to constrain  $\text{CMP}[F, H]$  for any sets  $R_{x,S} = \{x\} \times S$  where  $x \in H.\text{Inp}_2$  and  $S \in F.\text{SS}$ .

More formally, the relevant algorithms of  $\text{CMP}[F, H]$  are as follows.

| $\text{CMP}[F, H].\text{Con}^P(1^\lambda, k, R)$   | $\text{CMP}[F, H].\text{CEv}^P(1^\lambda, K_R, R, \mathbf{x})$ |
|----------------------------------------------------|----------------------------------------------------------------|
| If $R = R_S$ :                                     | If $R = R_S$ and $\mathbf{x} \in H.\text{Inp}_1$ :             |
| $K_R \leftarrow H.\text{Con}^P(1^\lambda, k, S)$   | $y \leftarrow H.\text{CEv}^P(1^\lambda, K_R, S, \mathbf{x})$   |
| Else if $R = R_{x,S}$ :                            | Else if $R = R_S$ and $\mathbf{x} = (x, x')$ :                 |
| $K_x \leftarrow H.\text{Ev}^P(1^\lambda, k, x)$    | $K_x \leftarrow H.\text{CEv}^P(1^\lambda, K_R, S, x)$          |
| $K_R \leftarrow F.\text{Con}^P(1^\lambda, K_x, S)$ | $y \leftarrow F.\text{Ev}^P(1^\lambda, K_x, x')$               |
| Return $K_R$                                       | Else if $R = R_{x,S}$ and $\mathbf{x} = (x, x')$ :             |
|                                                    | $y \leftarrow F.\text{CEv}^P(1^\lambda, K_R, S, x')$           |
|                                                    | Return $y$                                                     |

**Security Claims.** Before giving our proof we need a technical consistency property that we will assume of  $F$  and  $H$ .

**Definition 2.** *Constrained function family  $F$  is strongly consistent with  $P$  if  $K_R \leftarrow F.\text{Con}^P(k, R)$  was computed, then all future computations  $y \leftarrow F.\text{Ev}^P(k, x)$  or  $y \leftarrow F.\text{CEv}^P(K_R, R, x)$  for  $x \in R$  give the same  $y$ , no matter what queries to  $\text{PPRIM}$  are made in between, and the  $P$  queries made by  $F.\text{Con}$  and  $F.\text{CEv}$  match those made by  $F.\text{Ev}$ .*

The constrained function families considered in our work naturally satisfy this if  $P$  is consistent. These conditions allow evaluations with  $k$  to be undetectably replaced with constrained evaluations by an already computed  $K_R$ . The following theorem captures the security we can prove of the composed construction.

**Theorem 4.** *For  $X \in \{\text{IND}^*\text{-NC}, \text{SIM}^*\text{-AC}\}$ , if  $F$  and  $H$  are  $X$ -CPRF secure and strongly consistent with  $P$ , then so is  $\text{CMP}[F, H]$ .*

Details of the proof are in Appendix E.1 (IND\*-NC) and Appendix E.2 (SIM\*-AC). The “natural” proof which applies security of  $H$  then security of  $F$  works for IND\*-NC security (and does not actually require strong consistency). We replace all keys  $H$  outputs during  $\text{EV}$  queries with random. For  $\text{CON}$  queries  $R_{x,S}$  where  $x$  was used in previous  $\text{EV}$  queries we leave the output of  $H$  random. For other  $\text{CON}$  queries we used  $H$  honestly. Then we apply security of  $F$  to replace its output with random for  $\text{EV}$  queries, using that the underlying keys were switched to random in the previous step.

This proof idea breaks down if we try to replace IND\*-NC with IND\*-AC. The issue arises when the attacker makes  $\text{CON}$  queries of the form  $R_{x,S}$ . For this query, we want to switch  $K_x = H.\text{Ev}^P(K, x)$  to a random value if the attacker later makes an  $\text{EV}$  query of the form  $(x, x'')$ . We want to leave  $K_x$  nonrandom if the attacker is going to later make a  $\text{CON}$  query for  $R_S$  where  $x \in S$ . With IND\*-NC security, all  $\text{EV}$  queries had to be made first, so we knew whether the first case would occur by the time the  $\text{CON}$  query was made. With IND\*-AC security, queries can occur in arbitrary order, so we do not know which case will occur.

With SIM\*-AC security, we can again resolve the issue. We can switch  $K_x$  to random, then later if a  $\text{CON}$  query is made which should cause  $K_x$  to look nonrandom we use the simulator for  $H$  to program the ideal primitive appropriately. However, SIM\*-AC security also creates a “race condition” between the simulators of  $F$  and  $H$ . Intuitively, when a constrain query has been made on set  $S \subseteq H.\text{Inp}_1$ , the simulator for  $H$  will pick the constrained key (and thus “owns” all inputs in  $S$ ). Later, when another constrain query is made on  $x \times F.\text{Inp}$  with  $x \in S \cap H.\text{Inp}_2$ , the game has to execute  $H.\text{CEv}$  and  $F.\text{Con}$  to derive the actual output, as the adversary has seen the constrain key. Conversely, when the  $x \times F.\text{Inp}$  is made first, the simulator for  $F$  will pick the key (and thus “owns”  $x$ ). If a query is later made on  $S \subseteq H.\text{Inp}_1$  with  $x \in S$ , then the  $H$  simulator needs to be informed of the key picked by the  $F$  simulator so that it can program the ideal primitive appropriately. The  $H$  simulator will then take over “ownership” of  $x$ . In the proof we need to carefully track the ownership of inputs to simulate correctly. We give the proof in Appendix E.2.

|                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\text{S.Init}(1^\lambda)$<br>$\sigma_H \leftarrow \$ \text{S}_H.\text{Init}(1^\lambda)$<br>$\sigma_F \leftarrow \$ \text{S}_F.\text{Init}(1^\lambda)$<br>$\text{CKey} \leftarrow [\cdot]$<br>$\text{Own} \leftarrow [\cdot]$<br>Return $(\sigma_H, \sigma_F, \text{CKey}, \text{Own})$ | $\text{S.Con}^{\text{PPRIM}}(1^\lambda, u, R, f' : \sigma)$<br>$f^H, f^F \leftarrow [\cdot]$<br>For $x \in f'.D \cap H.\text{Inp}_1$ :<br>$f_u^H(x) \leftarrow f'(x)$<br>For $(x, x') \in f'.D \cap (H.\text{Inp}_2 \times F.\text{Inp})$ :<br>$f_{u,x}^F(x') \leftarrow f'(x, x')$<br>If $R = R_S$ :<br>For $x \in S$ s.t. $\text{Own}_u[x] = F$ or $f_{u,x}^F.D \neq \emptyset$ :<br>$f_u^H(x) \leftarrow \text{S}_F.\text{Con}^{\text{PPRIM}}((u, x), F.\text{Inp}, f_{u,x}^F : \sigma_F)$<br>$K_R \leftarrow \text{S}_H.\text{Con}^{\text{PPRIM}}(1^\lambda, u, S, f_u^H : \sigma_H)$<br>$\text{CKey}_u.\text{store}(S \cap H.\text{Inp}_2 \mapsto (K_R, S))$<br>$\text{Own}_u.\text{store}(S \cap H.\text{Inp}_2 \mapsto H)$<br>Else If $R = R_{x,S}$ :<br>$X \leftarrow \text{Own}_u[x]$<br>If $X \in \{\perp, F\}$ :<br>$K_R \leftarrow \text{S}_F.\text{Con}^{\text{PPRIM}}(1^\lambda, (u, x), S, f_{u,x}^F : \sigma_F)$<br>$\text{Own}_u.\text{store}(\{x\} \mapsto F)$<br>Else:<br>$(K_{S'}, S') \leftarrow \text{CKey}_u[x]$<br>$K_x \leftarrow H.\text{CEv}^P(1^\lambda, K_{S'}, S', x)$<br>$K_R \leftarrow F.\text{Con}^P(1^\lambda, K_x, S)$<br>$\text{CKey}_u.\text{store}(R \mapsto (K_R, R))$<br>Return $K_R$ |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 5.** Simulator S for the SIM\*-AC-CPRF security proof of  $\text{CMP}[F, H]$  (Theorem 4). A  $T.\text{store}(S \mapsto X)$  operation lazily assigns  $T[x] = X$  for all  $x$  in  $S$ . It **uses table Own** to track which simulator “owns” different the F keys. Ownership transfers from F to H when keys output by  $\text{S}_F$  are given to  $\text{S}_H$ . It **uses table CKey** to track previously returned constrained keys for use in emulating future evaluations of CMP or H using constrained evaluations. Correctness of this uses the strong consistency of H and F.

The simulator we use in the proof is shown in Fig. 5. Tables **Own** tracker the owner of  $x$ . Strong consistency is needed so that S can use previously returned constrained keys to simulate evaluations by performing constrained evaluations instead (using the table **CKey** to track constrained keys). Inside  $\text{S.Con}$  the partial function  $f'$  is used to reproduce any partial functions required by  $\text{S}_F.\text{Con}$ . Both  $f'$  and  $\text{S}_F.\text{Con}$  are used to reproduce partial functions required by  $\text{S}_H.\text{Con}$ . The low-level details of S are best understood by reading the proof which uses eight hybrid games (mixing transitions based on the the security of H and F with transitions that are syntactic rewritings) to slowly transform from the real SIM\*-AC-CPRF world to a game matching the simulated world when S is used.

### 5.3 Cascade-based Constructions

**Cascade Construction.** Bellare, Canetti, and Krawczyk [6] studied the cascade construction of a PRF which iterates a function family for small inputs to construct one for that accepts large inputs. We formalize it as following:

**Definition 3 (Cascade Construction [6]).** Let  $F$  be a function family with  $F.\text{Out} = F.K$ . Let  $D$  be a subset of  $F.\text{Inp}^*$ . Then  $\text{CSC}[F, D]$  has  $\text{CSC}[F, D].K = F.K$ ,  $\text{CSC}[F, D].\text{Inp} = D$ ,  $\text{CSC}[F, D].\text{Out} = F.K$ , and evaluation function  $\text{CSC}[F, D].\text{Ev}^P(1^\lambda, k, (x_0, \dots, x_{n-1})) : \text{“For } i = 0, \dots, n-1 \text{ do } k \leftarrow F.\text{Ev}^P(1^\lambda, k, x_i); \text{Return } k\text{”}$ .

We sometimes want to run this evaluation function without regard to any particular choice of  $D$ , in which case we will simply write  $\text{CSC}[F].\text{Ev}$ .

Note that  $\text{CSC}[F, D]$  is trivially insecure if any element of  $D$  is a prefix of another element due to length-extension attacks. If  $x \sqsubseteq y$ , then the output of CSC on  $y$  can easily be computed from the output of CSC on

$\mathbf{x}$ . A PRF adversary can easily achieve advantage  $1 - 1/|\mathbf{F}.\text{Out}|$  by making queries for  $\mathbf{x}$  and  $\mathbf{y}$  then checking if the output of the latter query matches the output it can compute itself from the former query. Because of this we will restrict attention to “prefix-free” sets  $\mathbf{D}$  for which no element is a prefix of another. (We additionally require the technical assumption that for a given  $\mathbf{x}$  we can efficiently check whether  $\exists \mathbf{y} \in \mathbf{D}, \mathbf{x} \sqsubseteq \mathbf{y}$  holds.) The most common choice is  $\mathbf{D} = \mathbf{F}.\text{Inp}^n$ .

The same property allows CSC to be used as a prefix-constrained function family. We define

- $\text{CSC}[\mathbf{F}, \mathbf{D}].\text{SS} = \{R_{x\sqsubseteq} \mid x \in \mathbf{F}.\text{Inp}^*\},$
- $\text{CSC}[\mathbf{F}, \mathbf{D}].\text{Con}^P(1^\lambda, k, R_{x\sqsubseteq}) = \text{CSC}[\mathbf{F}].\text{Ev}^P(1^\lambda, k, x),$  and
- $\text{CSC}[\mathbf{F}, \mathbf{D}].\text{CEv}^P(1^\lambda, K, R_{x\sqsubseteq}, y) = \text{CSC}[\mathbf{F}].\text{Ev}^P(1^\lambda, K, y \setminus x).$

Here  $y \setminus x$  denotes the suffix of  $y$  starting after the end of  $x$ .

For a given  $\mathbf{F}$  with  $\mathbf{F}.\text{Inp} = \{0, 1\}$ , define  $G$  by  $G(k) = \mathbf{F}(k, 0) \circ \mathbf{F}(k, 1)$ . Note that  $G$  is a length-doubling function. By a straightforward analysis,  $\mathbf{F}$  is a secure PRF if and only if  $G$  is a secure PRG. As noted by Bellare, Canetti, and Krawczyk, the function family obtained by applying the cascade construction to  $\mathbf{F}$  is identical to the GGM construction of a PRF [19] applied to  $G$ . Thus CSC can be viewed as a strict generalization of GGM.

**Augmented Cascade Construction.** Bellare, Bernstein, and Tessaro [5] considered a modification to the cascade to allow it to be used securely for domain  $\mathbf{F}.\text{Inp}^*$ , rather than having to restrict to a prefix-free domain. All output values are first shrunk by applying a keyless function  $\text{Chop}$ . Intuitively, shrinking the output sufficiently should prevent the length-extension attacks described above because the internal key value cannot be recovered from this shrunken output.

**Definition 4 (Augmented Cascade [5]).** *Let  $\mathbf{F}$  be a function family with  $\mathbf{F}.\text{Out} = \mathbf{F}.\mathbf{K}$ . Let  $\text{Chop}$  be a keyless function which outputs values in  $\text{Chop}.\text{Out}$  given inputs in  $\mathbf{F}.\mathbf{K}$ . then  $\text{ACSC}[\text{Chop}, \mathbf{F}]$  has keyspace  $\text{ACSC}[\text{Chop}, \mathbf{F}].\mathbf{K} = \mathbf{F}.\mathbf{K}$ , domain  $\text{ACSC}[\text{Chop}, \mathbf{F}].\text{Inp} = \mathbf{F}.\text{Inp}^*$ , range  $\text{ACSC}[\text{Chop}, \mathbf{F}].\text{Out} = \text{Chop}.\text{Out}$ , and evaluation function defined by  $\text{ACSC}[\text{Chop}, \mathbf{F}].\text{Ev}^P(1^\lambda, k, \mathbf{x}) : “k \leftarrow \text{CSC}[\mathbf{F}].\text{Ev}^P(1^\lambda, k, \mathbf{x}); \text{Return } \text{Chop}(1^\lambda, k)”$ .*

$\text{ACSC}[\text{Chop}, \mathbf{F}]$  can be viewed as a prefix-constrained PRF analogously to CSC. We define

- $\text{ACSC}[\text{Chop}, \mathbf{F}].\text{SS} = \{R_{x\sqsubseteq} \mid x \in \mathbf{F}.\text{Inp}^*\},$
- $\text{ACSC}[\text{Chop}, \mathbf{F}].\text{Con}^P(1^\lambda, k, R_{x\sqsubseteq}) = \text{CSC}[\mathbf{F}].\text{Ev}^P(1^\lambda, k, x),$  and
- $\text{ACSC}[\text{Chop}, \mathbf{F}].\text{CEv}^P(1^\lambda, K, R_{x\sqsubseteq}, y) = \text{ACSC}[\text{Chop}, \mathbf{F}].\text{Ev}^P(1^\lambda, K, y \setminus x).$

Results of Bellare, Bernstein, and Tessaro [5] imply that this is a secure PRF assuming that  $\mathbf{F}$  is a secure PRF even against attackers given leakage of  $\text{Chop}$  applied to the key. (Recall this is equivalent to assuming that  $\mathbf{F}_{\text{Chop}}$  is a secure PRF where  $\mathbf{F}_{\text{Chop}}$  is defined to be identical to  $\mathbf{F}$  except that  $\mathbf{F}_{\text{Chop}}.\text{Ev}^P(1^\lambda, K, \varepsilon) = \text{Chop}(1^\lambda, K)$  where  $\varepsilon \notin \mathbf{F}.\text{Inp}$ .)

**Security Claim.** The following captures our security results for CSC and ACSC.

**Theorem 5.** *If  $\mathbf{F}$  is  $\text{SIM}^*\text{-AC-PRF}$  secure with  $\mathbf{P}$  and  $\mathbf{D}$  is prefix-free, then  $\text{CSC}[\mathbf{F}, \mathbf{D}]$  is  $\text{SIM}^*\text{-AC-CPRF}$  secure with  $\mathbf{P}$ . If  $\mathbf{F}_{\text{Chop}}$  is  $\text{SIM}^*\text{-AC-PRF}$  secure with  $\mathbf{P}$ , then  $\text{ACSC}[\text{Chop}, \mathbf{F}]$  is  $\text{SIM}^*\text{-AC-CPRF}$  secure with  $\mathbf{P}$ . The same holds with  $\text{IND}^*\text{-PRF}$  security of  $\mathbf{F}$  (resp.  $\mathbf{F}_{\text{Chop}}$ ) and  $\text{IND}^*\text{-NC-CPRF}$  security of  $\text{CSC}[\mathbf{F}, \mathbf{D}]$  (resp.  $\text{ACSC}[\text{Chop}, \mathbf{F}]$ ).*

*Proof (Sketch).* Consider  $\mathbf{D} = \mathbf{F}.\text{Inp}^n$ . Recursively define  $\text{CSC}_1 = \mathbf{F}$  (viewing  $\mathbf{F}$  as a constrained function family) and  $\text{CSC}_{i+1} = \text{CMP}[\mathbf{F}, \text{CSC}_i]$ . Setting  $\text{CSC}_i.\text{Inp}_1 = \emptyset$  and  $\text{CSC}_i.\text{Inp}_2 = \mathbf{F}.\text{Inp}^i$  gives  $\text{CSC}_{i+1}.\text{Inp} = \mathbf{F}.\text{Inp}^i$ . We claim that  $\text{CSC}_n = \text{CSC}[\mathbf{F}, \mathbf{D}]$  and thus the claim follows by applying Theorem 4  $n - 1$  times (using Proposition 1 for the  $\text{SIM}^*\text{-AC-CPRF}$  security of  $\mathbf{F}$ ).

Augmented cascade works similarly by setting  $\text{ACSC}_0 = \mathbf{F}_{\text{Chop}}$  and  $\text{ACSC}_{i+1} = \text{CMP}[\mathbf{F}_{\text{Chop}}, \text{ACSC}_i]$ . Setting  $\text{ACSC}_i.\text{Inp}_1 = \mathbf{F}.\text{Inp}^{\leq i}$  and  $\text{ACSC}_i.\text{Inp}_2 = \mathbf{F}.\text{Inp}^{i+1}$  gives

$$\text{ACSC}_{i+1}.\text{Inp} = \mathbf{F}.\text{Inp}^{\leq i} \sqcup \mathbf{F}.\text{Inp}^{i+1} \times (\{\varepsilon\} \sqcup \mathbf{F}.\text{Inp}) = \mathbf{F}.\text{Inp}^{\leq i+2}.$$

Here we mildly abuse notation to identify the set  $F.Inp^{i+1} \times \{\varepsilon\}$  with the set  $F.Inp^{i+1}$  in the natural way.

If the adversary makes queries of length at most  $n$  (meaning queries always in  $F.Inp^{\leq n}$ ), then  $ACSC_n$  will behave equivalently to  $ACSC[Chop, F]$  and thus the claim follows by applying Theorem 4  $n - 1$  times (using Proposition 1 for the  $SIM^*$ -AC-CPRF security of  $F_{Chop}$ ).  $\square$

The full proof is in Appendix F. It proves security for any prefix-free  $D$ . Moreover, a literal use of the above would only work for a constant value of  $n$ . To handle arbitrary polynomial  $n$  we use details of the actual proof of Theorem 4, in particular that its simulator and reductions adversaries are sufficiently blackbox and uniform. We use this to build a recursive simulator and recursive reduction adversaries that work without having to a priori fix  $n$ . The ideas sketched above can then be applied using these adversaries.

## 5.4 Augmented MAC

In general, the augmented MAC hash function  $AMAC$  is defined by  $Chop(H(K \circ M))$  for a hash function  $H$ . An important special case, analyzed by Bellare, Bernstein, and Tessaro [5], is when  $H$  is a MD-style hash function like SHA512 and  $|K|$  is the blocklength of the underlying compression function. Then the evaluation of  $AMAC$  on  $k$  and input  $(x_0, \dots, x_{n-1})$  computes  $k \leftarrow F(s, k)$  and then  $k \leftarrow F(k, x_i)$  for  $i = 0, \dots, n - 1$  before returning  $Chop(k)$ . Here  $s$  is the seed value of the hash function.

This is basically an instance of  $ACSC$ , except the inputs to the first call of  $F$  were flipped. If  $F$  is a function family, we will let  $F'$  denote its dual defined by  $F'.Inp = F.K$ ,  $F'.Out = F.Out$ ,  $F'.K = F.Inp$ , and  $F'.Ev(k, x) = F.Ev(x, k)$ . Then we can capture this special case of  $ACSC$  as follows (where we consider the seed as part of the input, rather than modeling it as a fixed value).

**Definition 5 (Augmented MAC [5]).** We define  $AMAC[Chop, F] = CMP[ACSC[Chop, F], F']$  with  $F'.Inp_1 = \emptyset$  and  $F'.Inp_2 = F'.Inp$ , for appropriate  $F$  and  $Chop$ .

The domain of this function is  $F.K \times F.Inp^*$ . It is a prefixed-constrained function family. The following captures its security as a corollary of our earlier results.

**Corollary 1.** If  $F_{Chop}$  and  $F'$  are both  $SIM^*$ -AC-PRF secure with primitive  $P$ , then  $AMAC[Chop, F]$  is  $SIM^*$ -AC-CPRF secure with  $P$ . The same holds with  $IND^*$ -NC security.

*Proof.* From Theorem 5, we have that  $ACSC[Chop, F]$  is secure with  $P$ . By Proposition 1, security of  $F'$  as a function family implies security of it as a constrained function family. Hence Theorem 4 implies the security of  $CMP[ACSC[Chop, F], F']$ .  $\square$

**Acknowledgements** We thank Nirvan Tyagi, whose recommendation we examine how  $SIM$ -AC relates to the settings in [24] inspired this work.

## References

1. Aviram, N., Gellert, K., Jager, T.: Session resumption protocols and efficient forward security for TLS 1.3 0-RTT. *Journal of Cryptology* **34**(3), 20 (Jul 2021). <https://doi.org/10.1007/s00145-021-09385-0> 2, 12
2. Backendal, M., Günther, F., Paterson, K.G.: Puncturable key wrapping and its applications. In: Agrawal, S., Lin, D. (eds.) *ASIACRYPT 2022, Part II*. LNCS, vol. 13792, pp. 651–681. Springer, Cham, Switzerland, Taipei, Taiwan (Dec 5–9, 2022). [https://doi.org/10.1007/978-3-031-22966-4\\_22](https://doi.org/10.1007/978-3-031-22966-4_22) 2, 3, 11, 12, 28, 29, 30
3. Baum, C., Beullens, W., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., Roy, L., Scholl, P.: One tree to rule them all: Optimizing GGM trees and OWFs for post-quantum signatures. In: Chung, K.M., Sasaki, Y. (eds.) *ASIACRYPT 2024, Part I*. LNCS, vol. 15484, pp. 463–493. Springer, Singapore, Singapore, Kolkata, India (Dec 9–13, 2024). [https://doi.org/10.1007/978-981-96-0875-1\\_15](https://doi.org/10.1007/978-981-96-0875-1_15) 2
4. Baum, C., Braun, L., Delpach de Saint Guilhem, C., Klooß, M., Orsini, E., Roy, L., Scholl, P.: Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. In: Handschuh, H., Lysyanskaya, A. (eds.) *CRYPTO 2023, Part V*. LNCS, vol. 14085, pp. 581–615. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 20–24, 2023). [https://doi.org/10.1007/978-3-031-38554-4\\_19](https://doi.org/10.1007/978-3-031-38554-4_19) 2



5. Bellare, M., Bernstein, D.J., Tessaro, S.: Hash-function based PRFs: AMAC and its multi-user security. In: Fischlin, M., Coron, J.S. (eds.) EUROCRYPT 2016, Part I. LNCS, vol. 9665, pp. 566–595. Springer Berlin Heidelberg, Germany, Vienna, Austria (May 8–12, 2016). [https://doi.org/10.1007/978-3-662-49890-3\\_22](https://doi.org/10.1007/978-3-662-49890-3_22) 2, 4, 6, 7, 17, 18, 49, 50, 52, 55
6. Bellare, M., Canetti, R., Krawczyk, H.: Pseudorandom functions revisited: The cascade construction and its concrete security. In: 37th FOCS. pp. 514–523. IEEE Computer Society Press, Burlington, Vermont (Oct 14–16, 1996). <https://doi.org/10.1109/SFCS.1996.548510> 2, 3, 16
7. Bellare, M., Davis, H., Di, Z.: Hardening signature schemes via derive-then-derandomize: Stronger security proofs for EdDSA. In: Boldyreva, A., Kolesnikov, V. (eds.) PKC 2023, Part I. LNCS, vol. 13940, pp. 223–250. Springer, Cham, Switzerland, Atlanta, GA, USA (May 7–10, 2023). [https://doi.org/10.1007/978-3-031-31368-4\\_9](https://doi.org/10.1007/978-3-031-31368-4_9) 49
8. Bellare, M., Rogaway, P.: The security of triple encryption and a framework for code-based game-playing proofs. In: Vaudenay, S. (ed.) EUROCRYPT 2006. LNCS, vol. 4004, pp. 409–426. Springer Berlin Heidelberg, Germany, St. Petersburg, Russia (May 28 – Jun 1, 2006). [https://doi.org/10.1007/11761679\\_25](https://doi.org/10.1007/11761679_25) 4, 35
9. Boneh, D., Waters, B.: Constrained pseudorandom functions and their applications. In: Sako, K., Sarkar, P. (eds.) ASIACRYPT 2013, Part II. LNCS, vol. 8270, pp. 280–300. Springer Berlin Heidelberg, Germany, Bangalore, India (Dec 1–5, 2013). [https://doi.org/10.1007/978-3-642-42045-0\\_15](https://doi.org/10.1007/978-3-642-42045-0_15) 2, 3, 7, 9, 11, 13
10. Bost, R.:  $\Sigma\phi\phi\phi$ : Forward secure searchable encryption. In: Weippl, E.R., Katzenbeisser, S., Kruegel, C., Myers, A.C., Halevi, S. (eds.) ACM CCS 2016. pp. 1143–1154. ACM Press, Vienna, Austria (Oct 24–28, 2016). <https://doi.org/10.1145/2976749.2978303> 2, 10, 13, 14
11. Bost, R., Minaud, B., Ohrimenko, O.: Forward and backward private searchable encryption from constrained cryptographic primitives. In: Thuraisingham, B.M., Evans, D., Malkin, T., Xu, D. (eds.) ACM CCS 2017. pp. 1465–1482. ACM Press, Dallas, TX, USA (Oct 31 – Nov 2, 2017). <https://doi.org/10.1145/3133956.3133980> 2, 3, 10, 11, 12, 13, 14, 21, 22
12. Boyle, E., Goldwasser, S., Ivan, I.: Functional signatures and pseudorandom functions. In: Krawczyk, H. (ed.) PKC 2014. LNCS, vol. 8383, pp. 501–519. Springer Berlin Heidelberg, Germany, Buenos Aires, Argentina (Mar 26–28, 2014). [https://doi.org/10.1007/978-3-642-54631-0\\_29](https://doi.org/10.1007/978-3-642-54631-0_29) 2, 3, 7, 9, 11, 13
13. Camenisch, J., Drijvers, M., Gagliardini, T., Lehmann, A., Neven, G.: The wonderful world of global random oracles. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018, Part I. LNCS, vol. 10820, pp. 280–312. Springer, Cham, Switzerland, Tel Aviv, Israel (Apr 29 – May 3, 2018). [https://doi.org/10.1007/978-3-319-78381-9\\_11](https://doi.org/10.1007/978-3-319-78381-9_11) 2, 5
14. Cash, D., Jaeger, J., Jarecki, S., Jutla, C.S., Krawczyk, H., Rosu, M.C., Steiner, M.: Dynamic searchable encryption in very-large databases: Data structures and implementation. In: NDSS 2014. The Internet Society, San Diego, CA, USA (Feb 23–26, 2014). <https://doi.org/10.14722/ndss.2014.23264> 10
15. Cheng, K., Jaeger, J.: Adaptive security for constrained PRFs. In: Kalai, Y.T., Kamara, S.F. (eds.) CRYPTO 2025, Part V. LNCS, vol. 16004, pp. 598–627. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 17–21, 2025). [https://doi.org/10.1007/978-3-032-01901-1\\_19](https://doi.org/10.1007/978-3-032-01901-1_19) 1
16. Curtmola, R., Garay, J.A., Kamara, S., Ostrovsky, R.: Searchable symmetric encryption: improved definitions and efficient constructions. In: Juels, A., Wright, R.N., De Capitani di Vimercati, S. (eds.) ACM CCS 2006. pp. 79–88. ACM Press, Alexandria, Virginia, USA (Oct 30 – Nov 3, 2006). <https://doi.org/10.1145/1180405.1180417> 10
17. Du, M., Wu, S., Wang, Q., Chen, D., Jiang, P., Mohaisen, A.: Graphshield: Dynamic large graphs for secure queries with forward privacy. IEEE Transactions on Knowledge and Data Engineering 34(7), 3295–3308 (2020) 2, 10, 11
18. Fuchsbauer, G., Konstantinov, M., Pietrzak, K., Rao, V.: Adaptive security of constrained PRFs. In: Sarkar, P., Iwata, T. (eds.) ASIACRYPT 2014, Part II. LNCS, vol. 8874, pp. 82–101. Springer Berlin Heidelberg, Germany, Kaoshiung, Taiwan, R.O.C. (Dec 7–11, 2014). [https://doi.org/10.1007/978-3-662-45608-8\\_5](https://doi.org/10.1007/978-3-662-45608-8_5) 3, 7, 13
19. Goldreich, O., Goldwasser, S., Micali, S.: How to construct random functions (extended abstract). In: 25th FOCS. pp. 464–479. IEEE Computer Society Press, Singer Island, Florida (Oct 24–26, 1984). <https://doi.org/10.1109/SFCS.1984.715949> 2, 3, 7, 11, 17
20. Green, M.D., Miers, I.: Forward secure asynchronous messaging from puncturable encryption. In: 2015 IEEE Symposium on Security and Privacy. pp. 305–320. IEEE Computer Society Press, San Jose, CA, USA (May 17–21, 2015). <https://doi.org/10.1109/SP.2015.26> 12
21. Hofheinz, D., Kastner, J., Klein, K.: The power of undirected rewindings for adaptive security. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part II. LNCS, vol. 14082, pp. 725–758. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 20–24, 2023). [https://doi.org/10.1007/978-3-031-38545-2\\_24](https://doi.org/10.1007/978-3-031-38545-2_24) 3, 7, 13
22. Jaeger, J.: Let attackers program ideal models: Modularity and composability for adaptive compromise. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023, Part III. LNCS, vol. 14006, pp. 101–131. Springer, Cham, Switzerland, Lyon, France (Apr 23–27, 2023). [https://doi.org/10.1007/978-3-031-30620-4\\_4](https://doi.org/10.1007/978-3-031-30620-4_4) 2, 5, 6, 21, 22, 50



23. Jaeger, J., Tyagi, N.: Handling adaptive compromise for practical encryption schemes. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020, Part I. LNCS, vol. 12170, pp. 3–32. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 17–21, 2020). [https://doi.org/10.1007/978-3-030-56784-2\\_1](https://doi.org/10.1007/978-3-030-56784-2_1) 2, 5, 6, 9, 10, 12, 21, 22, 24
24. Kamath, C., Klein, K., Pietrzak, K., Walter, M.: The cost of adaptivity in security games on graphs. In: Nissim, K., Waters, B. (eds.) TCC 2021, Part II. LNCS, vol. 13043, pp. 550–581. Springer, Cham, Switzerland, Raleigh, NC, USA (Nov 8–11, 2021). [https://doi.org/10.1007/978-3-030-90453-1\\_19](https://doi.org/10.1007/978-3-030-90453-1_19) 3, 18
25. Kiayias, A., Papadopoulos, S., Triandopoulos, N., Zacharias, T.: Delegatable pseudorandom functions and applications. In: Sadeghi, A.R., Gligor, V.D., Yung, M. (eds.) ACM CCS 2013. pp. 669–684. ACM Press, Berlin, Germany (Nov 4–8, 2013). <https://doi.org/10.1145/2508859.2516668> 2, 3, 7, 9, 10, 11, 13
26. Nielsen, J.B.: Separating random oracle proofs from complexity theoretic proofs: The non-committing encryption case. In: Yung, M. (ed.) CRYPTO 2002. LNCS, vol. 2442, pp. 111–126. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 18–22, 2002). [https://doi.org/10.1007/3-540-45708-9\\_8](https://doi.org/10.1007/3-540-45708-9_8) 2, 9
27. Rogaway, P.: Authenticated-encryption with associated-data. In: Atluri, V. (ed.) ACM CCS 2002. pp. 98–107. ACM Press, Washington, DC, USA (Nov 18–22, 2002). <https://doi.org/10.1145/586110.586125> 12
28. Rogaway, P., Shrimpton, T.: A provable-security treatment of the key-wrap problem. In: Vaudenay, S. (ed.) EUROCRYPT 2006. LNCS, vol. 4004, pp. 373–390. Springer Berlin Heidelberg, Germany, St. Petersburg, Russia (May 28 – Jun 1, 2006). [https://doi.org/10.1007/11761679\\_23](https://doi.org/10.1007/11761679_23) 12, 28
29. Sahai, A., Waters, B.: How to use indistinguishability obfuscation: deniable encryption, and more. In: Shmoys, D.B. (ed.) 46th ACM STOC. pp. 475–484. ACM Press, New York, NY, USA (May 31 – Jun 3, 2014). <https://doi.org/10.1145/2591796.2591825> 2
30. Song, X., Zheng, Y., Bai, J., Dong, C., Liu, Z., Chang, E.C.: DISCO: Dynamic searchable encryption with constant state. In: Zhou, J., Quek, T.Q.S., Gao, D., Cárdenas, A.A. (eds.) ASIACCS 24. ACM Press, Singapore (Jul 1–5, 2024). <https://doi.org/10.1145/3634737.3637674> 2, 10, 11, 12
31. Sun, S., Yuan, X., Liu, J.K., Steinfeld, R., Sakzad, A., Vo, V., Nepal, S.: Practical backward-secure searchable encryption from symmetric puncturable encryption. In: Lie, D., Mannan, M., Backes, M., Wang, X. (eds.) ACM CCS 2018. pp. 763–780. ACM Press, Toronto, ON, Canada (Oct 15–19, 2018). <https://doi.org/10.1145/3243734.3243782> 2, 10
32. Tyagi, N., Mughees, M.H., Ristenpart, T., Miers, I.: BurnBox: Self-revocable encryption in a world of compelled access. In: Enck, W., Felt, A.P. (eds.) USENIX Security 2018. pp. 445–461. USENIX Association, Baltimore, MD, USA (Aug 15–17, 2018), <https://www.usenix.org/conference/usenixsecurity18/presentation/tyagi> 5
33. Wang, J., Sun, S.F., Li, T., Qi, S., Chen, X.: Practical volume-hiding encrypted multi-maps with optimal overhead and beyond. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) ACM CCS 2022. pp. 2825–2839. ACM Press, Los Angeles, CA, USA (Nov 7–11, 2022). <https://doi.org/10.1145/3548606.3559345> 2, 10, 11
34. Yoneyama, K., Kimura, S.: Verifiable and forward secure dynamic searchable symmetric encryption with storage efficiency. In: Qing, S., Mitchell, C., Chen, L., Liu, D. (eds.) ICICS 17. LNCS, vol. 10631, pp. 489–501. Springer, Cham, Switzerland, Beijing, China (Dec 6–8, 2017). [https://doi.org/10.1007/978-3-319-89500-0\\_42](https://doi.org/10.1007/978-3-319-89500-0_42) 2, 10, 11

## A Proof of Theorem 1 (SIM\*-AC-CPRF implies IND\*-AC-CPRF)

*Proof.* To show SIM\*-AC-CPRF security implies IND\*-AC-CPRF security, we build  $\mathcal{A}_{\text{cprf}}^*$  using  $\mathcal{A}_{\text{cprf}}$  below. We refer to the game  $\text{G}^{\text{ind}^*-\text{ac-cprf}}$  that  $\mathcal{A}_{\text{cprf}}^*$  is emulating for  $\mathcal{A}_{\text{cprf}}$  as the “emulated game”, and the game  $\text{G}^{\text{sim}^*-\text{ac-cprf}}$  (defined in bottom of Fig. 2) that  $\mathcal{A}_{\text{cprf}}^*$  itself is playing as the “hidden game”.

| Adversary $\mathcal{A}_{\text{cprf}}^{\text{PPRIM,CON,Ev}}(1^\lambda)$           | $\text{CONSIM}(u, R)$             | $\text{EVSIM}(u, \mathbf{x})$                        |
|----------------------------------------------------------------------------------|-----------------------------------|------------------------------------------------------|
| $f_{(\cdot)} \leftarrow \text{CF.Fcs}(\lambda)$                                  | Require $R \cap E_u = \emptyset$  | Require $\mathbf{x} \notin C_u$                      |
| $b \leftarrow \{0, 1\}$                                                          | $K_R \leftarrow \text{CON}(u, R)$ | If $b = 1$ : $y \leftarrow \text{EV}(u, \mathbf{x})$ |
| $b' \leftarrow \mathcal{A}_{\text{cprf}}^{\text{PPRIM,CONSIM,EVSIM}}(1^\lambda)$ | $C_u \leftarrow C_u \cup R$       | If $b = 0$ : $y \leftarrow f_u(\mathbf{x})$          |
| Return $(b = b')$                                                                | Return $K_R$                      | $E_u \cup \{\mathbf{x}\}$                            |
|                                                                                  |                                   | Return $y$                                           |

Let  $b^*$  denote the underlying chosen bit in  $\text{G}^{\text{sim}^*-\text{ac-cprf}}$ .

**When  $b^*$  is 1.** CON and EV in the “hidden game” are returning actual constrained keys and evaluation results. Therefore,  $\mathcal{A}_{\text{cprf}}^*$  is indeed emulating the game  $\text{G}^{\text{ind}^*-\text{ac-cprf}}$  for  $\mathcal{A}_{\text{cprf}}$ .

**When  $b^*$  is 0.** First notice that  $E_u$  and  $C_u$  are consistent between the “emulated game” and the “hidden game”. Therefore, when  $b = 1$ , EV will always return a random value, just like the case in  $b = 0$ . Also, since  $R \cap E_u = \emptyset$ , simulator in the hidden game will not be told any information of prior queries. Thus, the output of CON is independent of possible EV queries. For EVSIM, in the “emulated game”, we require  $\mathbf{x} \notin C_u$ , therefore, EV in the “hidden game” will never satisfy the  $\mathbf{x} \in C_u$  condition. Therefore, when  $b$  is 1, the output of EV will always be the output of random function  $f_u$ . The requirement that  $R \cap E_u = \emptyset$  in CONSIM ensures that these output values are never shared with the simulator (and thus not used anywhere else). Thus there is no difference between these outputs and the outputs when  $b$  is 0. Therefore,  $\mathcal{A}_{\text{cprf}}^*$ ’s view is independent of  $b$  and its probability of guessing it correctly is exactly 1/2. For any simulator S, we get

$$\begin{aligned}
 \text{Adv}_{\text{CF,P},\mathcal{A}_{\text{cprf}}}^{\text{ind}^*-\text{ac-cprf}}(\lambda) &= 2 \Pr[b = b' | b^* = 1] - 1 \\
 &= 2(\text{Adv}_{\text{CF,S,P},\mathcal{A}_{\text{cprf}}^{\text{sim}^*-\text{ac-cprf}}}(\lambda) + \Pr[b = b' | b^* = 0]) - 1 \\
 &= 2\text{Adv}_{\text{CF,S,P},\mathcal{A}_{\text{cprf}}^{\text{sim}^*-\text{ac-cprf}}}(\lambda).
 \end{aligned}$$

By assumption there is a simulator that makes this negligible.  $\square$

## B Searchable Symmetric Encryption Application

In this section, we show the details of application of our constrained PRF definition to searchable symmetric encryption (SSE) as summarized in Section 4.1. We first provide the syntax and security definitions we will use for randomized symmetric encryption. Then in Appendix B.1 we analyze SSE. We give detailed syntax and security definitions of a searchable symmetric scheme. Then we define the scheme FS2, which is a tweaked version of FS-RCPRF from [11]. We prove that it meets the corresponding security definition, when the underlying building blocks are secure.

**Preliminaries: Symmetric Encryption.** A symmetric encryption scheme SE specifies encryption algorithm SE.Enc, decryption algorithm SE.Dec, and key space SE.K. The syntax of its algorithms is given by  $e \leftarrow \text{SE.Enc}^P(1^\lambda, ek, m)$  and  $m \leftarrow \text{SE.Dec}^P(1^\lambda, ek, e)$  where  $ek \in \text{SE.K}(\lambda)$  is a key,  $m$  is a message to be encrypted, and  $e$  is a ciphertext. Informally, correctness requires that  $\text{SE.Dec}^P(1^\lambda, ek, \text{SE.Enc}^P(1^\lambda, ek, m)) = m$ .

SIM\*-AC-CPA security [22,23] of SE is captured by the game shown in Fig. 6. In the “real world” ( $b = 1$ ), the adversary can ask for arbitrary encryptions via ENC and ask to expose keys via EXP. In the “simulated world” ( $b = 0$ ), encryption responses are chosen by a simulator which knows the length of the message and which user is doing the encryption. On exposure, the simulator is given all input-output pairs from prior ENC

| Game $G_{SE,S,P,A}^{sim^*-ac-cpa}(\lambda)$                 | $ENC(u, m)$                                                                |
|-------------------------------------------------------------|----------------------------------------------------------------------------|
| $\sigma_P \leftarrow \$ P.Init(1^\lambda)$                  | If $K_u = \perp$ : $\ell \leftarrow  m $ else $\ell \leftarrow (u, m)$     |
| $b \leftarrow \$ \{0, 1\}$                                  | If $b = 1$ : $e \leftarrow \$ SE.Enc^P(1^\lambda, K_u, m)$                 |
| If $b = 1$ : $K_{(\cdot)} \leftarrow \$ SE.K(\lambda)$      | If $b = 0$ : $e \leftarrow \$ S.Enc^{PPRIM}(1^\lambda, u, \ell : \sigma)$  |
| If $b = 0$ : $\sigma \leftarrow \$ S.Init(1^\lambda)$       | $M_u.add(e, m)$ ; Return $e$                                               |
| $b' \leftarrow \$ \mathcal{A}^{ENC, EXP, PPRIM}(1^\lambda)$ | $EXP(u)$                                                                   |
| Return $(b = b')$                                           | If $b = 1$ : $K_u \leftarrow K_u$ // No-op                                 |
| $PPRIM(Op, x, y)$                                           | If $b = 0$ : $K_u \leftarrow \$ S.Exp^{PPRIM}(1^\lambda, u, M_u : \sigma)$ |
| Return $P.Op(1^\lambda, x, y : \sigma_P)$                   | Return $K_u$                                                               |

Fig. 6. Game defining the SIM\*-AC CPA security of SE

queries to the given user, then outputs a key. Its goal is to reprogram  $P$  to make the previous input-output pairs look consistent with  $SE$  applied to this key.

The advantage of  $\mathcal{A}$  is  $\text{Adv}_{SE,S,P,A}^{sim^*-ac-cpa}(\lambda) = 2\Pr[G_{SE,S,P,A}^{sim^*-ac-cpa}(\lambda)] - 1$ . If there exists a PPT  $S$  such that  $\text{Adv}_{SE,S,P,A}^{sim^*-ac-cpa}$  is negligible for all PPT  $\mathcal{A}$ , then we say that  $SE$  is SIM\*-AC-CPA secure with  $P$ . Typical symmetric encryption schemes are known to be SIM\*-AC-CPA secure [22,23].

### B.1 Searchable Symmetric Encryption

**Syntax.** We consider a database  $DB$  represented as a set  $\{(ind_i, W_i)\}_{i=1}^{|DB|}$ . Here each  $ind_i$  is the index of the  $i$ -th document and  $W_i \subseteq \{0, 1\}^*$  is the set of keywords corresponding to that document. A dynamic searchable encryption scheme  $SSE$  specifies algorithms  $SSE.Init$ ,  $SSE.CSearch$ ,  $SSE.SSearch$ ,  $SSE.CUpdate$ , and  $SSE.SUpdate$ . We will only consider updates that add to the database (no deletion).

- $SSE.Init(1^\lambda)$  outputs a triple  $(EDB, K_{SSE}, \mathbf{W})$  where  $K_{SSE}$  is a master secret key (stored by the client),  $EDB$  is an initial encrypted database (stored by the server), and  $\mathbf{W}$  is a client state.<sup>4</sup>
- $(SSE.CSearch(1^\lambda, K_{SSE}, w : \mathbf{W}), SSE.SSearch(1^\lambda, \tau : EDB))$  form a one round search protocol between the client and server. The client's algorithm  $SSE.CSearch$  takes input master secret key  $K_{SSE}$ , keyword  $w$ , and state  $\mathbf{W}$ . It outputs a search token  $\tau$ . The server's algorithm  $SSE.SSearch$  takes input search token  $\tau$  and encrypted database  $EDB$ . It outputs a set of document indices  $I = \{ind_i\}$  corresponding to keyword  $w$ .
- $(SSE.CUpdate(1^\lambda, K_{SSE}, ind, w : \mathbf{W}), SSE.SUpdate(1^\lambda, \tau : EDB))$  forms an update protocol between the client and server. The client's algorithm  $SSE.CUpdate$  inputs  $K_{SSE}$ , index  $ind$ , keyword  $w$ , and  $\mathbf{W}$ . It outputs update token  $\tau$ . The server's algorithm  $SSE.SUpdate$  takes in update token  $\tau$  and encrypted database  $EDB$ .

Informally, correctness of an  $SSE$  scheme  $SSE$  requires that the search protocol returns set of document indices currently corresponding to the keyword  $w$  (with high probability).

**Security.** The security of a searchable symmetric encryption scheme guarantees that the server should not learn anything about the database and queries beyond some explicit leakage. We use a simulator-based, game-based proof to capture this idea. We define the game in Fig. 7. The adversary picks the database  $DB$  to start and is able to trigger any operations from the client via the four oracles  $SETUP$ ,  $SEARCH$ , and  $UPDATE$ . It is tasked to distinguish the real world ( $b = 1$ ) to the simulated world ( $b = 0$ ).

In the real world, the oracles will invoke the actual algorithms of  $SSE$ . In the simulated world, they will return output generated by a simulator  $S$ . The simulator is only provided with some leakage for each operation. This is captured by the *leakage function*  $L$ .

The advantage of  $\mathcal{A}_{SSE}$  is defined as  $\text{Adv}_{SSE,L,S,P,A_{SSE}}^{SSE}(\lambda) = 2\Pr[G_{SSE,L,S,P,A_{SSE}}^{SSE}(\lambda)] - 1$ . If there exists a PPT  $S$  such that  $\text{Adv}_{SSE,L,S,P,A_{SSE}}^{SSE}$  is negligible for all PPT  $\mathcal{A}_{SSE}$ , then we say that  $SSE$  is  $L$  secure with  $P$ . (Security is often defined to instead allow  $S$  to depend on  $\mathcal{A}_{SSE}$ , but constructions anyway achieve this stronger quantification.)

<sup>4</sup> Here our notation follows the analysis of FS-RCPRF by Bost, et al. [11] which assume an initially empty database.

|                                                                                                              |                                                                                                     |
|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Game $\text{G}_{\text{SSE}, \text{L}, \text{S}, \text{P}, \mathcal{A}_{\text{SSE}}}^{\text{SSE}}(\lambda)$   | $\text{SEARCH}(w)$                                                                                  |
| $\sigma_P \leftarrow \text{P.Init}(1^\lambda)$                                                               | If $b = 1$ : $\tau \leftarrow \text{SSE.CSearch}^P(1^\lambda, K_{\text{SSE}}, w : \mathbf{W})$      |
| $b \leftarrow \{0, 1\}$                                                                                      | If $b = 0$ : $\ell \leftarrow \text{L.CSearch}(1^\lambda, w : \sigma_L)$                            |
| If $b = 1$ : $(\text{EDB}, K_{\text{SSE}}, \mathbf{W}) \leftarrow \text{SSE.Init}(1^\lambda)$                | $\tau \leftarrow \text{S.CSearch}^{\text{PPRIM}}(1^\lambda, \ell : \sigma)$                         |
| If $b = 0$ : $\sigma_L \leftarrow \text{L.Init}(1^\lambda)$                                                  | Return $\tau$                                                                                       |
| $(\text{EDB}, \sigma) \leftarrow \text{S.Init}^{\text{PPRIM}}(1^\lambda)$                                    | $\text{UPDATE}(ind, w)$                                                                             |
| $b' \leftarrow \mathcal{A}_{\text{SSE}}^{\text{SEARCH}, \text{UPDATE}, \text{PPRIM}}(1^\lambda, \text{EDB})$ | If $b = 1$ : $\tau \leftarrow \text{SSE.CUpdate}^P(1^\lambda, K_{\text{SSE}}, ind, w : \mathbf{W})$ |
| Return $(b = b')$                                                                                            | If $b = 0$ : $\ell \leftarrow \text{L.CUpdate}(1^\lambda, ind, w : \sigma_L)$                       |
| $\text{PPRIM}(\text{Op}, x, y)$                                                                              | $\tau \leftarrow \text{S.CUpdate}^{\text{PPRIM}}(1^\lambda, \ell : \sigma)$                         |
| Return $\text{P.Op}(1^\lambda, x, y : \sigma_P)$                                                             | Return $\tau$                                                                                       |

**Fig. 7.** Game defining security of searchable symmetric encryption scheme SSE with respect to leakage L

|                                                                        |                                                                     |
|------------------------------------------------------------------------|---------------------------------------------------------------------|
| $\text{FS2.Init}(1^\lambda)$                                           | $\text{FS2.CSearch}^P(1^\lambda, K_{\text{SSE}}, w : \mathbf{W})$   |
| $K_{\text{SSE}} \leftarrow \text{F.K}(\lambda)$                        | $c \leftarrow \mathbf{W}[w]$                                        |
| $\mathbf{W}[\cdot] \leftarrow 0$ ; $\text{EDB} \leftarrow []$          | If $c = 0$ : Return $(\perp, 0)$                                    |
| Return $(\text{EDB}, K_{\text{SSE}}, \mathbf{W})$                      | $K_w \leftarrow \text{F.Ev}^P(1^\lambda, K_{\text{SSE}}, w)$        |
| $\text{FS2.CUpdate}^P(1^\lambda, K_{\text{SSE}}, w, ind : \mathbf{W})$ | $k_{[c]} \leftarrow \text{CF.Con}^P(1^\lambda, K_w, [c])$           |
| $K_w \leftarrow \text{F.Ev}^P(1^\lambda, K_{\text{SSE}}, w)$           | $\tau \leftarrow (k_{[c]}, c)$                                      |
| $c \leftarrow \mathbf{W}[w]$ ; $\mathbf{W}[w] \leftarrow c + 1$        | Return $\tau$                                                       |
| $(UT, ek) \leftarrow \text{CF.Ev}^P(1^\lambda, K_w, 1^{c+1})$          | $\text{FS2.SSearch}^P(1^\lambda, \tau = (k_{[c]}, c) : \text{EDB})$ |
| $e \leftarrow \text{SE.Enc}^P(1^\lambda, ek, ind)$                     | $D \leftarrow \emptyset$                                            |
| $\tau \leftarrow (UT, e)$                                              | If $c = 0$ : Return $D$                                             |
| Return $\tau$                                                          | For $i = c, \dots, 1$ :                                             |
| $\text{FS2.SUpdate}^P(1^\lambda, \tau = (UT, e) : \text{EDB})$         | $(UT, ek) \leftarrow \text{CF.CEv}^P(1^\lambda, k_{[c]}, [c], 1^i)$ |
| $\text{EDB}[UT] \leftarrow e$                                          | $e \leftarrow \text{EDB}[UT]$                                       |
| Return                                                                 | $ind_i \leftarrow \text{SE.Dec}^P(1^\lambda, ek, e)$                |
|                                                                        | $D.\text{add}(ind_i)$                                               |
|                                                                        | Return $D$                                                          |

**Fig. 8.** Algorithms of forward-secure searchable encryption scheme FS2

**Construction.** Let SE be a symmetric encryption scheme. Let CF be a constrained function family with  $\text{CF.Inp} = 1^*$ ,  $\text{CF.Out} = \text{CF.Out}_1 \times \text{SE.K}$ , and CF.SS containing  $[c] = \{1, 11, \dots, 1^c\}$  for all  $c \in \mathbb{N}$ .<sup>5</sup> Let F be a function family with  $\text{F.Out} = \text{CF.K}$ . Then FS2 is defined as shown in Fig. 8. The idea behind this construction was described in Section 4.1.

**Leakage.** On update queries, FS2 does not leak anything beyond the length of  $ind$  because the output of CF on a new input will look random, so  $UT$  looks random and  $e$  is a secure encryption. On searches, the adversary can match the constrained key it sees with earlier updates and searches. It can learn the document indices by performing the actual search procedure (and when the keyword was added to that particular document by comparing with earlier indices). Let  $t$  be a timestamp that starts at 0 and increases with each query. For simplicity, we write  $\text{Q.add}(w)$ , which means setting  $\text{Q}[t] \leftarrow w$  where  $t$  is the current timestamp.  $\text{U.add}(ind, w)$  should be interpreted similarly. This is formalized by the leakage function  $\text{L}_{\text{fs}}$  shown below.

<sup>5</sup> It suffices to restrict  $\text{CF.Inp}$  to inputs of length at most  $2^\lambda$ . Such a CF can be constructed from a prefix-constrained function family  $\text{CF}'$  with domain  $\text{CF'.Inp}(\lambda) = \{0, 1\}^\lambda$ , by interpreting elements of  $\text{CF.Inp}$  and  $\text{CF'.Inp}$  as integers (written in unary and binary respectively).

|                                              |                                              |                                                    |
|----------------------------------------------|----------------------------------------------|----------------------------------------------------|
| $L_{fs}.Init(1^\lambda)$                     | $L_{fs}.CUpdate(1^\lambda, ind, w : (Q, U))$ | $L_{fs}.CSearch(1^\lambda, w : (Q, U))$            |
| $\sigma_L \leftarrow (\emptyset, \emptyset)$ | $U.add(ind, w)$                              | $Q.add(w)$                                         |
| Return $\sigma_L$                            | Return $ ind $                               | $sp \leftarrow \{t : Q[t] = w\}$                   |
|                                              |                                              | $UpHist \leftarrow \{(t, ind) : U[t] = (ind, w)\}$ |
|                                              |                                              | Return $(sp, UpHist)$                              |

Because updates leak nothing unless the keyword is later searched, FS2 can be considered forward secure. We do not actually require the search pattern  $sp$  in our simulation. When the update history  $UpHist$  for  $w$  is non-empty, it implicitly allows us to be consistent between different searches for  $w$ . When  $UpHist$  is empty we just return  $\tau = (\perp, 0)$ .

**Security Result.** The following theorem captures that security of FS2 follows from the security of its underlying components.

**Theorem 6.** *Suppose  $F$  is  $IND^*$ -PRF secure with  $P$ ,  $CF$  is  $SIM^*$ -AC-PRF secure with  $P$ , and  $SE$  is  $SIM^*$ -AC-CPA secure with  $P$ . Then FS2 is  $L_{FS}$  secure with  $P$ .*

A similar proof [23] analyzing the SSE scheme  $\Pi_{bas}$  needed to assume a “key-private” security notion for the underlying symmetric encryption, which we do not have to do here. The difference is that for that scheme every ciphertext for a given keyword was encrypted with the same key while in FS2 each ciphertext comes from a different key.

*Proof.* The proof proceeds via a series of hybrids  $G_0$  to  $G_3$ , switching from the real world to the simulated world. In these hybrids we first switch the output of  $F$  to be random, then switch the output of  $CF$  to be simulated, then switch the output of  $SE$  to be simulated. We define a simulator  $S$  from the simulators  $S_{cprf}$  for  $CF$  and  $S_{cpa}$  for  $SE$  and build adversaries  $\mathcal{A}_{prf}$ ,  $\mathcal{A}_{cprf^*}$ , and  $\mathcal{A}_{cpa}$  for which we establish the following claims:

1.  $\Pr[G_0(\lambda)] = \Pr[G_{SSE, L_{FS}, S, P, \mathcal{A}_{sse}}^{sse}(\lambda) | b = 1]$
2.  $\Pr[G_0(\lambda)] - \Pr[G_1(\lambda)] = \text{Adv}_{F, P, \mathcal{A}_{prf}}^{ind^*-prf}(\lambda)$
3.  $\Pr[G_1(\lambda)] - \Pr[G_2(\lambda)] = \text{Adv}_{CF, S_{cprf}, P, \mathcal{A}_{cprf^*}}^{sim^*-ac-cprf}(\lambda)$
4.  $\Pr[G_2(\lambda)] - \Pr[G_3(\lambda)] = \text{Adv}_{SE, S_{cpa}, P, \mathcal{A}_{cpa}}^{sim^*-ac-cpa}(\lambda)$
5.  $\Pr[G_3(\lambda)] = \Pr[\neg G_{SSE, L_{FS}, S, P, \mathcal{A}_{sse}}^{sse}(\lambda) | b = 0]$ .

Therefore,

$$\begin{aligned} \text{Adv}_{SSE, L_{FS}, S, P, \mathcal{A}_{sse}}^{sse}(\lambda) &= \Pr[G_0(\lambda)] - \Pr[G_3(\lambda)] = \sum_{i=0}^2 \Pr[G_i(\lambda)] - \Pr[G_{i+1}(\lambda)] \\ &= \text{Adv}_{F, P, \mathcal{A}_{prf}}^{ind^*-prf}(\lambda) + \text{Adv}_{CF, S_{cprf}, P, \mathcal{A}_{cprf^*}}^{sim^*-ac-cprf}(\lambda) + \text{Adv}_{SE, S_{cpa}, P, \mathcal{A}_{cpa}}^{sim^*-ac-cpa}(\lambda). \end{aligned}$$

The algorithms we construct are PPT (assuming the given algorithms are), so this implies the result by letting  $S_{cprf}$  and  $S_{cpa}$  be the simulators assumed to exist by the security of  $CF$  and  $SE$ .

**Simulator.** The simulator  $S$  we will use for FS2 is given in Fig. 9. It responds to update queries by choosing  $UT$  at random and having  $S_{cpa}$  choose the ciphertext  $e$ . It tracks these in the list  $H$ . Then for search queries it wants to use  $S_{cprf}$  to choose the constrained key. For this it needs to give  $S_{cprf}$  a partial function telling it what the “outputs” of  $CF$  from earlier updates. It builds this in  $T$  using a for loop where it remembers the  $UT$ ’s from earlier updates and asks  $S_{cpa}$  to decide what the  $ek$ ’s should have been.

For asking the simulators to expose keys, we need to tell them the “name” of the corresponding user. We name the  $S_{cpa}$  users by the “time” at which their ciphertext was output by an update and  $S_{cprf}$  users by the first “time” the corresponding keyword was included in an update.



|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\text{S.Init}(1^\lambda)$<br>$\overline{\text{EDB}} \leftarrow [\cdot]$<br>$\sigma_{\text{cprf}} \leftarrow \$ \text{S}_{\text{cprf}}.\text{Init}(1^\lambda)$<br>$\sigma_{\text{cpa}} \leftarrow \$ \text{S}_{\text{cpa}}.\text{Init}(1^\lambda)$<br>$\text{H} \leftarrow \emptyset$<br>$\text{V} \leftarrow 0$<br>Return $(\text{EDB}, (\sigma_{\text{cprf}}, \sigma_{\text{cpa}}, \text{H}))$<br>$\text{S.CUpdate}^{\text{PPRIM}}(1^\lambda, \ell : (\sigma_{\text{cprf}}, \sigma_{\text{cpa}}, \text{H}, \text{V}))$<br>$\text{UT} \leftarrow \$ \text{CF}.\text{Out}_1(\lambda)$<br>$t \leftarrow  \text{H}  + 1$<br>$e \leftarrow \$ \text{S}_{\text{cpa}}.\text{Enc}^{\text{PPRIM}}(1^\lambda, t, \ell : \sigma_{\text{cpa}})$<br>$\text{H.add}(\text{UT}, e)$<br>Return $(\text{UT}, e)$ | $\text{S.CSearch}^{\text{PPRIM}}(1^\lambda, (\text{sp}, \text{UpHist}) : (\sigma_{\text{cprf}}, \sigma_{\text{cpa}}, \text{H}, \text{V}))$<br>$\{(t_1, \text{ind}_1), (t_2, \text{ind}_2), \dots, (t_c, \text{ind}_c)\} \leftarrow \text{UpHist}$<br>$\text{// } t_i < t_{i+1}$<br>If $c = 0$ : Return $(\perp, 0)$<br>$c_{\text{prev}} \leftarrow \text{V}[t_1]; \text{V}[t_1] \leftarrow c$<br>$T \leftarrow [\cdot]$<br>For $i = c_{\text{prev}} + 1, \dots, c$ :<br>$(\text{UT}, e) \leftarrow \text{H}[t_i]$<br>$M \leftarrow \emptyset; M.\text{add}(e, \text{ind}_i)$<br>$ek \leftarrow \$ \text{S}_{\text{cpa}}.\text{Exp}^{\text{PPRIM}}(1^\lambda, t_i, M : \sigma_{\text{cpa}})$<br>$T[i] \leftarrow (\text{UT}, ek)$<br>$k_{[c]} \leftarrow \$ \text{S}_{\text{cprf}}.\text{Con}^{\text{PPRIM}}(1^\lambda, t_1, [c], T : \sigma_{\text{cprf}})$<br>Return $(k_{[c]}, c)$ |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 9.** Simulator S for SSE scheme FS2, constructed from simulator  $\text{S}_{\text{cprf}}$  for CF and simulator  $\text{S}_{\text{cpa}}$  for SE

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Games $\text{G}_0(\lambda), \text{G}_1(\lambda)$<br>$\sigma_{\text{P}} \leftarrow \$ \text{P}.\text{Init}(1^\lambda)$<br>$K_{\text{SSE}} \leftarrow \$ \text{F}.\text{K}(\lambda)$<br>$K_{(\cdot)} \leftarrow \$ \text{CF}.\text{K}(\lambda)$<br>$\text{W}[\cdot] \leftarrow 0$<br>$b' \leftarrow \$ \mathcal{A}_{\text{SSE}}^{\text{SEARCH, UPDATE, PPRIM}}(1^\lambda, [\cdot])$<br>Return $(b' = 1)$<br>$\text{PPRIM}(\text{Op}, x, y)$<br>Return $\text{P.Op}(x, y : \sigma_{\text{P}})$ | $\text{SEARCH}(w)$<br>$c \leftarrow \text{W}[w]$<br>If $c = 0$ : Return $(\perp, 0)$<br>$K_w \leftarrow \text{F.Ev}^{\text{P}}(1^\lambda, K_{\text{SSE}}, w)$<br>$k_{[c]} \leftarrow \text{CF.Con}^{\text{P}}(K_w, [c])$<br>Return $(k_{[c]}, c)$<br>$\text{UPDATE}(\text{ind}, w)$<br>$K_w \leftarrow \text{F.Ev}^{\text{P}}(1^\lambda, K_{\text{SSE}}, w)$<br>$c \leftarrow \text{W}[w]; \text{W}[w] \leftarrow c + 1$<br>$(\text{UT}, ek) \leftarrow \text{CF.Ev}^{\text{P}}(1^\lambda, K_w, 1^{c+1})$<br>$e \leftarrow \$ \text{SE}.\text{Enc}^{\text{P}}(1^\lambda, ek, \text{ind})$<br>Return $(\text{UT}, e)$ |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 10.** Hybrid games  $\text{G}_0$  and  $\text{G}_1$  used in analysis of FS2

|                                                                                                                                                                                                                       |                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Adversary $\mathcal{A}_{\text{prf}}^{\text{Ev, PPRIM}}(1^\lambda)$<br>$\text{W}[\cdot] \leftarrow 0$<br>$b' \leftarrow \$ \mathcal{A}_{\text{SSE}}^{\text{SEARCH, UPDATE, PPRIM}}(1^\lambda, [\cdot])$<br>Return $b'$ | $\text{SEARCH}(w)$<br>$c \leftarrow \text{W}[w]$<br>If $c = 0$ : Return $(\perp, 0)$<br>$K_w \leftarrow \text{Ev}(w)$<br>$k_{[c]} \leftarrow \text{CF.Con}^{\text{P}}(K_w, [c])$<br>Return $(k_{[c]}, c)$ | $\text{UPDATE}(\text{ind}, w)$<br>$K_w \leftarrow \text{Ev}(w)$<br>$c \leftarrow \text{W}[w]; \text{W}[w] \leftarrow c + 1$<br>$(\text{UT}, ek) \leftarrow \text{CF.Ev}^{\text{P}}(1^\lambda, K_w, 1^{c+1})$<br>$e \leftarrow \$ \text{SE}.\text{Enc}^{\text{P}}(1^\lambda, ek, \text{ind})$<br>Return $(\text{UT}, e)$ |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 11.** IND\*-PRF adversary  $\mathcal{A}_{\text{prf}}$  used to switch the output of F from real to random

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Games <math>G_{1.5}(\lambda), G_2(\lambda)</math></b><br>$\sigma_P \leftarrow \$P.\text{Init}(1^\lambda)$<br>$K_{(\cdot)} \leftarrow \$CF.K(\lambda)$<br>$\sigma_{\text{cprf}} \leftarrow S_{\text{cprf}}.\text{Init}(1^\lambda)$<br>$\mathbf{V}[\cdot] \leftarrow 0$<br>$b' \leftarrow \$\mathcal{A}_{\text{SSE}}^{\text{SEARCH, UPDATE, PPRIM}}(1^\lambda, [\cdot])$<br>Return ( $b' = 1$ )<br>$\text{UPDATE}(ind, w)$<br>$c \leftarrow  \{(t, ind) : \mathbf{U}[t] = (ind, w)\} $<br>$\mathbf{U.add}(ind, w)$<br>$(UT, ek) \leftarrow CF.\text{Ev}^P(1^\lambda, K_w, 1^{c+1})$<br>$UT \leftarrow CF.\text{Out}_1(\lambda)$<br>$ek \leftarrow \$SE.K(\lambda)$<br>$e \leftarrow \$SE.\text{Enc}^P(1^\lambda, ek, ind)$<br>$\mathbf{H.add}(UT, ek)$<br>Return ( $UT, e$ ) | <b>SEARCH(<math>w</math>)</b><br>$\text{UpHist} \leftarrow \{(t, ind) : \mathbf{U}[t] = (ind, w)\}$<br>$\{(t_1, ind_1), (t_2, ind_2), \dots, (t_c, ind_c)\} \leftarrow \text{UpHist}$<br>$\parallel t_i < t_{i+1}$<br>If $c = 0$ : Return ( $\perp, 0$ )<br>$k_{[c]} \leftarrow CF.\text{Con}^P(K_w, [c])$<br>$c_{\text{prev}} \leftarrow \mathbf{V}[t_1]; \mathbf{V}[t_1] \leftarrow c$<br>$T \leftarrow [\cdot]$<br>For $i = c_{\text{prev}} + 1, \dots, c$ :<br>$(UT, ek) \leftarrow \mathbf{H}[t_i]$<br>$T[i] \leftarrow (UT, ek)$<br>$k_{[c]} \leftarrow \$S_{\text{cprf}}.\text{Con}^{\text{PPRIM}}(1^\lambda, t_1, [c], T : \sigma_{\text{cprf}})$<br>Return ( $k_{[c]}, c$ ) |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 12.** Hybrids game  $G_{1.5}$  and  $G_2$  used in analysis of FS2

|                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Adversary <math>\mathcal{A}_{\text{cprf}^*}^{\text{CON, Ev, PPRIM}}(1^\lambda)</math></b><br>$\mathbf{V}[\cdot] \leftarrow 0$<br>$b' \leftarrow \$\mathcal{A}_{\text{SSE}}^{\text{UPDATE, SEARCH, PPRIM}}(1^\lambda, [\cdot])$<br>Return $b'$ | <b>SEARCH(<math>w</math>)</b><br>$c \leftarrow  \{(t, ind) : \mathbf{U}[t] = (ind, w)\} $<br>$t_1 \leftarrow \min\{t : \mathbf{U}[t] = (\cdot, w)\}$<br>If $c = 0$ : Return ( $\perp, 0$ )<br>$k_{R_c} \leftarrow \text{CON}(t_1, [c])$<br>Return ( $k_{[c]}, c$ ) | <b>UPDATE(<math>w, ind</math>)</b><br>$c \leftarrow  \{(t, ind) : \mathbf{U}[t] = (ind, w)\} $<br>$\mathbf{U.add}(ind, w)$<br>$t_1 \leftarrow \min\{t : \mathbf{U}[t] = (\cdot, w)\}$<br>$(UT, ek) \leftarrow \text{Ev}(t_1, c + 1)$<br>$e \leftarrow SE.\text{Enc}^P(1^\lambda, ek, ind)$<br>Return ( $UT, ek$ ) |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 13.** SIM\*-AC-CPRF adversary  $\mathcal{A}_{\text{cprf}^*}$  used to switch the output of CF from real to simulated

**Claims 1 and 2.** We start with  $G_0$  defined in Fig. 11 which contains highlighted but not boxed code. It was obtained by plugging in the code of  $SSE.\text{Init}$ ,  $SSE.\text{CSearch}$ , and  $SSE.\text{CUpdate}$  into the  $b = 1$  world of  $G_{\text{SSE}, L_{fs}, S, P, \mathcal{A}_{\text{SSE}}}^{\text{SSE}}(\lambda)$ . Hence our first claim, that  $\Pr[G_0(\lambda)] = \Pr[G_{\text{SSE}, L_{fs}, S, P, \mathcal{A}_{\text{SSE}}}^{\text{SSE}}(\lambda) | b = 1]$ , follows.

Game  $G_1$  in Fig. 11 contains boxed but not highlighted code. We replace the usage of  $F$  by picking  $K_{(\cdot)}$  randomly. Naturally, we will bound the difference between  $G_0$  and  $G_1$  by the IND\*-PRF security of  $F$ . The relevant adversary  $\mathcal{A}_{\text{prf}}$  is given in Fig. 10. It simulates the view of  $\mathcal{A}_{\text{SSE}}$  in these two games by using its  $\text{Ev}$  oracle to compute  $K_w$ . Correctness of this simulation gives our second claim, that  $\Pr[G_0(\lambda)] - \Pr[G_1(\lambda)] = \text{Adv}_{F, P, \mathcal{A}_{\text{prf}}}^{\text{ind}^*-\text{prf}}(\lambda)$ .

Our games never modify PPRIM, so henceforth we will omit its code.

**Claim 3.** Now consider the games defined in Fig. 12. The first game  $G_{1.5}$  contains highlighted but not boxed code. It is identical to  $G_1$  except that we have updated the booking to omit the table  $\mathbf{W}$ . These changes are colored in red. We use a list  $\mathbf{U}$  to store all of the UPDATE queries made (matching the list stored by  $\mathbf{L}$ ) and when  $c = \mathbf{W}[w]$  is needed, we recover it by checking how many entries of  $\mathbf{U}$  contain  $w$ . Hence  $\Pr[G_{1.5}(\lambda)] = \Pr[G_1(\lambda)]$ .

Now game  $G_2$  contains boxed but not highlighted code. It switches output of CF to being simulated. In UPDATE we sample  $(UT, ek)$  at random and store this in a list  $\mathbf{H}$  (matching the list stored by  $\mathbf{S}$ , except it contains  $ek$  rather than  $e$ ). In SEARCH we have  $S_{\text{cprf}}$  choose the constrained key. To do so, we need to give it a partial function containing all the random values output by UPDATE for this keyword since the last time a SEARCH query was made for this keyword. We use list  $\mathbf{H}$  to fill the partial function  $T$ , with table  $\mathbf{V}$  tracking the value of  $c$  from the previous SEARCH query for this keyword to know what values of the list to

| Games $\mathbf{G}_{2.5}(\lambda), \mathbf{G}_3(\lambda)$                                                         | $\text{SEARCH}(w)$                                                                                                   |
|------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| $\sigma_P \leftarrow \text{P.Init}(1^\lambda)$                                                                   | $\text{UpHist} \leftarrow \{(t, \text{ind}) : \mathbf{U}[t] = (\text{ind}, w)\}$                                     |
| $\sigma_{\text{cprf}} \leftarrow \text{S}_{\text{cprf}}.\text{Init}(1^\lambda)$                                  | $\{(t_1, \text{ind}_1), (t_2, \text{ind}_2), \dots, (t_c, \text{ind}_c)\} \leftarrow \text{UpHist}$                  |
| $\sigma_{\text{cpa}} \leftarrow \text{S}_{\text{cpa}}.\text{Init}(1^\lambda)$                                    | $\parallel t_i < t_{i+1}$                                                                                            |
| $\mathbf{V}[\cdot] \leftarrow 0$                                                                                 | If $c = 0$ : Return $(\perp, 0)$                                                                                     |
| $b' \leftarrow \mathcal{A}_{\text{sse}}^{\text{SEARCH}, \text{UPDATE}, \text{PPRIM}}(1^\lambda, [\cdot])$        | $c_{\text{prev}} \leftarrow \mathbf{V}[t_1]; \mathbf{V}[t_1] \leftarrow c$                                           |
| Return $(b' = 1)$                                                                                                | $T \leftarrow [\cdot]$                                                                                               |
| $\text{UPDATE}(\text{ind}, w)$                                                                                   | For $i = c_{\text{prev}} + 1, \dots, c$ :                                                                            |
| $\mathbf{U}.\text{add}(\text{ind}, w)$                                                                           | $(UT, ek) \leftarrow \mathbf{H}[t_i]$                                                                                |
| $UT \leftarrow \text{CF.Out}_1(\lambda)$                                                                         | $(UT, e) \leftarrow \mathbf{H}[t_i]$                                                                                 |
| $ek \leftarrow \text{SE.K}(\lambda)$                                                                             | $M \leftarrow \emptyset; M.\text{add}(e, \text{ind}_i)$                                                              |
| $e \leftarrow \text{SE.Enc}^P(1^\lambda, ek, \text{ind})$                                                        | $ek \leftarrow \text{S}_{\text{cpa}}.\text{Exp}^{\text{PPRIM}}(1^\lambda, t_i, M : \sigma_{\text{cpa}})$             |
| $\mathbf{H}.\text{add}(UT, ek)$                                                                                  | $T[i] \leftarrow (UT, ek)$                                                                                           |
| $t \leftarrow  \mathbf{H}  + 1$                                                                                  | $k_{[c]} \leftarrow \text{S}_{\text{cprf}}.\text{Con}^{\text{PPRIM}}(1^\lambda, t_1, [c], T : \sigma_{\text{cprf}})$ |
| $e \leftarrow \text{S}_{\text{cpa}}.\text{Enc}^{\text{PPRIM}}(1^\lambda, t,  \text{ind}  : \sigma_{\text{cpa}})$ | Return $(k_{[c]}, c)$                                                                                                |
| $\mathbf{H}.\text{add}(UT, e)$                                                                                   |                                                                                                                      |
| Return $(UT, e)$                                                                                                 |                                                                                                                      |

**Fig. 14.** Hybrid games  $\mathbf{G}_{2.5}$  and  $\mathbf{G}_3$  used in analysis of FS2

| Adversary $\mathcal{A}_{\text{cpa}}^{\text{ENC}, \text{EXP}, \text{PPRIM}}(1^\lambda)$                    | $\text{SEARCH}(w)$                                                                                                   |
|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| $\sigma \leftarrow \text{S}_{\text{cprf}}.\text{Init}()$                                                  | $\text{UpHist} \leftarrow \{(t, \text{ind}) : \mathbf{U}[t] = (\text{ind}, w)\}$                                     |
| $\mathbf{V}[\cdot] \leftarrow 0$                                                                          | $\{(t_1, \text{ind}_1), (t_2, \text{ind}_2), \dots, (t_c, \text{ind}_c)\} \leftarrow \text{UpHist}$                  |
| $b' \leftarrow \mathcal{A}_{\text{sse}}^{\text{UPDATE}, \text{SEARCH}, \text{PPRIM}}(1^\lambda, [\cdot])$ | $\parallel t_i < t_{i+1}$                                                                                            |
| Return $b'$                                                                                               | If $c = 0$ : Return $(\perp, 0)$                                                                                     |
| $\text{UPDATE}(\text{ind}, w)$                                                                            | $c_{\text{prev}} \leftarrow \mathbf{V}[t_1]; \mathbf{V}[t_1] \leftarrow c$                                           |
| $\mathbf{U}.\text{add}(\text{ind}, w)$                                                                    | $T \leftarrow [\cdot]$                                                                                               |
| $UT \leftarrow \text{CF.Out}_1(\lambda)$                                                                  | For $i = c_{\text{prev}} + 1, \dots, c$ :                                                                            |
| $t \leftarrow  \mathbf{H}  + 1$                                                                           | $(UT, \cdot) \leftarrow \mathbf{H}[t_i]$                                                                             |
| $e \leftarrow \text{ENC}(t, \text{ind})$                                                                  | $ek \leftarrow \text{EXP}(t_i)$                                                                                      |
| $\mathbf{H}.\text{add}(UT, e)$                                                                            | $T[i] \leftarrow (UT, ek)$                                                                                           |
| Return $(UT, e)$                                                                                          | $k_{[c]} \leftarrow \text{S}_{\text{cprf}}.\text{Con}^{\text{PPRIM}}(1^\lambda, t_1, [c], T : \sigma_{\text{cprf}})$ |
|                                                                                                           | Return $(k_{[c]}, c)$                                                                                                |

**Fig. 15.** SIM\*-AC-CPA adversary  $\mathcal{A}_{\text{cpa}}$  is used to switch the output of SE from real to simulated

use. Following  $\mathbf{S}$ , we name the user for a keyword (from  $\text{S}_{\text{cprf}}$ 's perspective) after the first time that keyword was updated.

We can reduce the difference between  $\mathbf{G}_1$  and  $\mathbf{G}_2$  to SIM\*-AC-CPRF security of CF using the adversary  $\mathcal{A}_{\text{cprf}^*}$  in Fig. 13. It simulates the view of  $\mathcal{A}_{\text{sse}}$  in these two games by using its oracles to choose  $k_{[c]}$ ,  $UT$ , and  $ek$ . Correctness of this simulation gives our third claim, that  $\Pr[\mathbf{G}_1(\lambda)] - \Pr[\mathbf{G}_2(\lambda)] = \text{Adv}_{\text{CF}, \text{S}_{\text{cprf}}, \text{P}, \mathcal{A}_{\text{cprf}^*}}^{\text{sim}^*-\text{ac-cprf}}(\lambda)$ .

**Claim 4.** Now consider the games defined in Fig. 14. Game  $\mathbf{G}_{2.5}$  contains highlighted but not boxed code. It was copied directly from  $\mathbf{G}_2$ . Hence  $\Pr[\mathbf{G}_{2.5}(\lambda)] = \Pr[\mathbf{G}_2(\lambda)]$ .

Now game  $\mathbf{G}_3$  contains boxed but not highlighted code. It switches from using real encryptions and keys of SE to using simulated ones. In UPDATE we follow  $\mathbf{S}$  and use the time of the current query as the name of the user for  $\text{S}_{\text{cpa}}$ . We switch to storing  $e$ , rather than  $ek$  in  $\mathbf{H}$ . In SEARCH we need  $ek$  to give to  $\text{S}_{\text{cprf}}$ , so we recover it using  $\text{S}_{\text{cpa}}$ . We tell  $\text{S}_{\text{cpa}}$  the appropriate document index  $\text{ind}$  from  $\mathbf{U}$ .

We can reduce the difference between  $G_2$  and  $G_3$  to the  $\text{SIM}^*\text{-AC-CPA}$  security of  $\text{SE}$  using the adversary  $\mathcal{A}_{\text{cpa}}$  in Fig. 15. It simulated the view of  $\mathcal{A}_{\text{sse}}$  in these two games by using its own oracles to choose  $e$  and  $ek$ . Correctness of this simulation gives our fourth claim, that  $\Pr[G_2(\lambda)] - \Pr[G_3(\lambda)] = \text{Adv}_{\text{SE}, \text{S}_{\text{cpa}}, \text{P}, \mathcal{A}_{\text{cpa}}}^{\text{sim}^*\text{-ac-cpa}}(\lambda)$ .

**Claim 5.** Game  $G_3$  responds to oracle queries identically to the  $b = 0$  world of  $G^{\text{sse}}$ . Plugging the code of  $\text{L}_{\text{FS}}$  and  $\text{S}$  into the latter can verify this claim. Consequently, their only difference is that the former outputs the boolean ( $b' = 1$ ) while the latter outputs the boolean ( $b' = 0$ ). This gives our fifth claim, that  $\Pr[G_3(\lambda)] = \Pr[\neg G_{\text{SSE}, \text{L}_{\text{FS}}, \text{S}, \text{P}, \mathcal{A}_{\text{sse}}}^{\text{sse}}(\lambda) | b = 0]$ .  $\square$

## C Puncturable Key Wrapping Application

In this section, we show the details of our application of our constrained PRF definition to puncturable key wrapping as summarized in Section 4.2. We first provide the syntax and security definitions that we will use for nonce-based symmetric encryption (in particular providing a misuse-resistant  $\text{SIM}^*\text{-AC}$  notion). Then in Appendix C.1 we give syntax and security notions of PKW as formalized by Backendal, Günther, and Paterson [2]. We consider the batch puncturing setting discussed, but not analyzed, by Backendal, et al. We confirm that their construction from a constrained PRF and encryption scheme is secure in this setting assuming  $\text{SIM}^*\text{-AC}$  security of both components.

**Preliminaries: Nonce-based Encryption.** A nonce-based encryption scheme  $\text{NE}$  specifies encryption algorithm  $\text{NE.Enc}$  and decryption algorithm  $\text{NE.Dec}$  together with key space  $\text{NE.K}$ , nonce space  $\text{NE.N}$ , header space  $\text{NE.H}$ , message space  $\text{NE.M}$ , and ciphertext length function  $\text{NE.cl}$ . The syntax of its algorithms are  $c \leftarrow \text{NE.Enc}^P(1^\lambda, sk, n, h, m)$  and  $m \leftarrow \text{NE.Dec}^P(1^\lambda, sk, n, h, c)$  where  $sk \in \text{NE.K}(\lambda)$  is the key,  $n \in \text{NE.N}(\lambda)$  is the nonce,  $h \in \text{NE.H}(\lambda)$  is the header,  $m \in \text{NE.M}(\lambda)$  is the message, and  $c \in \{0, 1\}^{\text{NE.cl}(\lambda, |m|)}$  is the ciphertext. Informally, correctness requires that  $m = \text{NE.Dec}^P(1^\lambda, sk, n, h, \text{NE.Enc}^P(1^\lambda, sk, n, h, m))$  always holds.

We define a  $\text{SIM}^*\text{-AC}$ -style security notion for  $\text{NE}$  which captures misuse resistance [28] and that ciphertexts should be indistinguishable from random. It is captured by the game shown in Fig. 16. In the “real world” ( $b = 1$ ), the adversary can ask for arbitrary encryptions via  $\text{ENC}$  and ask to expose keys via  $\text{EXP}$ . In the “simulated world” ( $b = 0$ ), encryption responses are sampled at random and kept consistent across repeat queries. On exposure, the simulator is given all input-output pairs for prior  $\text{ENC}$  queries to the given user, then outputs a key. Its goal is to reprogram  $\text{P}$  to make the previous input-output pairs look consistent with  $\text{NE}$  applied to this key. Misuse resistance is captured because no restrictions are placed on nonces repeating.

The advantage of  $\mathcal{A}$  is  $\text{Adv}_{\text{NE}, \text{S}, \text{P}, \mathcal{A}}^{\text{sim}^*\text{-ac-mr\$}}(\lambda) = 2\Pr[G_{\text{NE}, \text{S}, \text{P}, \mathcal{A}}^{\text{sim}^*\text{-ac-mr\$}}(\lambda)] - 1$ . If there exists a PPT  $\text{S}$  such that  $\text{Adv}_{\text{NE}, \text{S}, \text{P}, \mathcal{A}}^{\text{sim}^*\text{-ac-mr\$}}$  is negligible for all PPT  $\mathcal{A}$ , then we say  $\text{NE}$  is  $\text{SIM}^*\text{-AC-MR\$}$  secure with  $\text{P}$ . We expect that “typical” misuse-resistant encryption schemes achieve  $\text{SIM}^*\text{-AC-MR\$}$  security assuming appropriate security of their building blocks.

### C.1 Puncturable Key Wrapping

**Syntax.** A puncturable key wrapping scheme [2] specifies key generation algorithm  $\text{PKW.Kg}$ , wrapping algorithm  $\text{PKW.Wrap}$ , unwrapping algorithm  $\text{PKW.Unwrap}$ , and puncturing algorithm  $\text{PKW.Punc}$  together with tag space  $\text{PKW.T}$ , header space  $\text{PKW.H}$ , message space  $\text{PKW.M}$ , and ciphertext length function  $\text{PKW.cl}$ . The syntax of its algorithms is given by  $K \leftarrow \text{PKW.Kg}(1^\lambda)$ ,  $c \leftarrow \text{PKW.Wrap}^P(1^\lambda, K, t, h, m)$ ,  $m \leftarrow \text{PKW.Unwrap}^P(1^\lambda, K, t, h, c)$ , and  $K' \leftarrow \text{PKW.Punc}^P(1^\lambda, K, t)$  where  $K$  is a key,  $K'$  is a key punctured at  $t$ ,  $t \in \text{PKW.T}(\lambda)$  is a tag,  $h \in \text{PKW.H}(\lambda)$  is a header,  $m \in \text{PKW.M}(\lambda)$  is a message, and  $c \in \{0, 1\}^{\text{PKW.cl}(\lambda, |m|)}$  is a ciphertext. The expectation is that  $K'$  “works equivalently” to  $K$  except on the tag  $t$  — in particular, wrapping or unwrapping with  $K'$  and  $t$  should return  $\perp$ . Informally, correctness of PKW requires that  $m = \text{PKW.Unwrap}^P(1^\lambda, K', t, h, \text{PKW.Wrap}^P(1^\lambda, K', t, h, m))$  if  $K', K''$  were derived from some initial key  $K \leftarrow \text{PKW.Kg}(1^\lambda)$  by puncturing on tags other than  $t$ .

In essence, a puncturable key wrapping scheme is a nonce-based encryption scheme with added functionality for puncturing the key. The differing terminology (wrapping, unwrapping, tags vs. encrypting, decrypting, nonces) hints at the primitive being intended for use with messages that are themselves keys.

|                                                                                                         |                                                                                                             |
|---------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Game $\mathcal{G}_{\text{NE}, \text{S}, \text{P}, \mathcal{A}}^{\text{sim}^* \text{-ac-mr\$}}(\lambda)$ | $\text{ENC}(u, n, h, m)$                                                                                    |
| $\sigma_P \leftarrow \text{P.Init}(1^\lambda)$                                                          | If $b = 1$ : $T_u[n, h, m] \leftarrow \text{NE.Enc}^P(1^\lambda, K_u, n, h, m)$                             |
| $b \leftarrow \{0, 1\}$                                                                                 | If $b = 0$ :                                                                                                |
| If $b = 1$ : $K_{(\cdot)} \leftarrow \text{SE.K}(\lambda)$                                              | If $K_u \neq \perp$ : $T_u[n, h, m] \leftarrow \text{S.Enc}^{\text{PPRIM}}(1^\lambda, u, n, h, m : \sigma)$ |
| If $b = 0$ : $\sigma \leftarrow \text{S.Init}(1^\lambda)$                                               | Else if $T_u[n, h, m] \neq \perp$ : $T_u[n, h, m] \leftarrow T_u[n, h, m]$ // No-op                         |
| $b' \leftarrow \mathcal{A}^{\text{ENC}, \text{EXP}, \text{PPRIM}}(1^\lambda)$                           | Else $T_u[n, h, m] \leftarrow \{0, 1\}^{\text{NE.cl}(\lambda,  m )}$                                        |
| Return $(b = b')$                                                                                       | Return $T_u[n, h, m]$                                                                                       |
| $\text{PPRIM}(\text{Op}, x, y)$                                                                         | $\text{EXP}(u)$                                                                                             |
| $y \leftarrow \text{P.Op}(1^\lambda, x, y : \sigma_P)$                                                  | If $b = 1$ : $K_u \leftarrow K_u$ // No-op                                                                  |
| Return $y$                                                                                              | If $b = 0$ : $K_u \leftarrow \text{S.Exp}^{\text{PPRIM}}(1^\lambda, u, T_u : \sigma)$                       |
|                                                                                                         | Return $K_u$                                                                                                |

Fig. 16. Game defining SIM\*-AC-MR\$ security of NE

|                                                                                                                                                                                             |                                                           |                                                        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|--------------------------------------------------------|
| Game $\mathcal{G}_{\text{PKW}, \text{P}, \mathcal{A}}^{\text{find\$}^* \text{-rcpa}}(\lambda)$ , $\mathcal{G}_{\text{PKW}, \text{P}, \mathcal{A}}^{\text{find\$}^* \text{-rcpa}+}(\lambda)$ | $\text{WRAP}(u, t, h, m)$                                 | $\text{CORRUPT}(u)$                                    |
| $\sigma_P \leftarrow \text{P.Init}(1^\lambda)$                                                                                                                                              | Require $t \notin W_u$                                    | Require $V_u \subseteq P_u$                            |
| $K_{(\cdot)} \leftarrow \text{PKW.Kg}(1^\lambda)$                                                                                                                                           | Require $(t, h, m) \notin W_u$                            | Require $\text{corr}_u = \text{false}$                 |
| $b \leftarrow \{0, 1\}$                                                                                                                                                                     | Require $t \notin P_u$ and $\text{corr}_u = \text{false}$ | $\text{corr}_u \leftarrow \text{true}$                 |
| $b' \leftarrow \mathcal{A}^{\text{RO\$-WRAP}, \text{WRAP}, \text{PUNC}, \text{CORRUPT}, \text{PPRIM}}(1^\lambda)$                                                                           | $c \leftarrow \text{PKW.Wrap}^P(1^\lambda, K_u, t, h, m)$ | Return $K_u$                                           |
| Return $(b = b')$                                                                                                                                                                           | $W_u \leftarrow W_u \cup \{t\}$                           | $\text{PPRIM}(\text{Op}, x, y)$                        |
| $\text{RO\$-WRAP}(u, t, h, m)$                                                                                                                                                              | $W_u \leftarrow W_u \cup \{(t, h, m)\}$                   | $y \leftarrow \text{P.Op}(1^\lambda, x, y : \sigma_P)$ |
| Require $t \notin W_u$                                                                                                                                                                      | Return $c$                                                | Return $y$                                             |
| Require $(t, h, m) \notin W_u$                                                                                                                                                              | $\text{PUNC}(u, t)$                                       |                                                        |
| Require $t \notin P_u$ and $\text{corr}_u = \text{false}$                                                                                                                                   | Require $t \notin P_u$ and $\text{corr}_u = \text{false}$ |                                                        |
| If $b = 1$ : $c \leftarrow \text{PKW.Wrap}^P(1^\lambda, K_u, t, h, m)$                                                                                                                      | $K_u \leftarrow \text{PKW.Punc}^P(1^\lambda, K_u, t)$     |                                                        |
| If $b = 0$ : $c \leftarrow \{0, 1\}^{\text{PKW.cl}(\lambda,  m )}$                                                                                                                          | $P_u \leftarrow P_u \cup \{t\}$                           |                                                        |
| $V_u \leftarrow V_u \cup \{t\}$                                                                                                                                                             | Return // No value is returned                            |                                                        |
| $W_u \leftarrow W_u \cup \{t\}$                                                                                                                                                             |                                                           |                                                        |
| $W_u \leftarrow W_u \cup \{(t, h, m)\}$                                                                                                                                                     |                                                           |                                                        |
| Return $y$                                                                                                                                                                                  |                                                           |                                                        |

Fig. 17. Games defining FIND\$\*-RCPA and FIND\$\*-RCPA+ security of PKW

**Security.** We capture security of PKW via the games in Fig. 17. FIND\$\*-RCPA security is captured by  $\mathcal{G}_{\text{PKW}, \text{P}, \mathcal{A}}^{\text{find\$}^* \text{-rcpa}}$  which contains the highlighted code. FIND\$\*-RCPA+ security is captured by  $\mathcal{G}_{\text{PKW}, \text{P}, \mathcal{A}}^{\text{find\$}^* \text{-rcpa}+}$  which contains the boxed code. Backendal, et al. [2] analyzed schemes with respect to the former, while they proposed, but did not concretely analyze schemes with respect to the latter. Both security notions capture forward security (F) and indistinguishability from random bits (IND\$), while allowing real wrappings (R) in a chosen-plaintext setting (CPA). The latter adds misuse-resistance (+), by allowing tags to be repeated.

In the game, PUNC allows puncturing the key on tags of the adversary's choice and CORRUPT returns the current key. In the real world ( $b = 1$ ), both RO\$-WRAP and WRAP will return real wrappings while in the random world ( $b = 0$ ) RO\$-WRAP will return random ciphertexts of the appropriate length.

Sets  $W$ ,  $V$ , and  $P$  and boolean  $\text{corr}$  store per-user information to prevent trivial wins. In  $\mathcal{G}_{\text{PKW}, \text{P}, \mathcal{A}}^{\text{find\$}^* \text{-rcpa}}$  the sets store, respectively, the tags that have been used for any wrapping, the tags that have been used for challenge real-or-random wrappings, and the tags that have been punctured at. The boolean tracks whether the key has been corrupted. The game disallows repeating tags for different wrappings, making queries for previously punctured tags, corrupting keys that have not been punctured at all tags used for challenges, and making queries for already corrupted users. Game  $\mathcal{G}_{\text{PKW}, \text{P}, \mathcal{A}}^{\text{find\$}^* \text{-rcpa}}$  adds misuse resistance by having the set  $W$  instead stores entire  $(t, h, m)$  tuples and disallowing their reuse.

|                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\text{PKW}[\text{CF}, \text{NE}].\text{Kg}(1^\lambda)$<br>$k \leftarrow \text{CF.K}(\lambda)$<br>$K \leftarrow \text{CF.Con}^P(1^\lambda, k, \text{CF.Inp}(\lambda))$<br>Return $(K, \text{CF.Inp}(\lambda))$                                        | $\text{PKW}[\text{CF}, \text{NE}].\text{Punc}^P(1^\lambda, (K_R, R), t)$<br>If $t \notin R$ : Return $(K_R, R)$<br>$K_{R \setminus \{t\}} \leftarrow \text{CF.CCon}^P(1^\lambda, K_R, R, R \setminus \{t\})$<br>Return $(K_{R \setminus \{t\}}, R \setminus \{t\})$ |
| $\text{PKW}[\text{CF}, \text{NE}].\text{Wrap}^P(1^\lambda, (K_R, R), t, h, m)$<br>If $t \notin R$ : Return $\perp$<br>$sk \leftarrow \text{CF.CEv}^P(1^\lambda, K_R, R, t)$<br>$c \leftarrow \text{NE.Enc}^P(1^\lambda, sk, n_0, h, m)$<br>Return $c$ | $\text{PKW}[\text{CF}, \text{NE}].\text{Unwrap}^P(1^\lambda, (K_R, R), t, h, c)$<br>If $t \notin R$ : Return $\perp$<br>$sk \leftarrow \text{CF.CEv}^P(1^\lambda, K_R, R, t)$<br>$m \leftarrow \text{NE.Dec}^P(1^\lambda, sk, n_0, h, c)$<br>Return $m$             |

Fig. 18. Algorithms of puncturable key wrapping scheme  $\text{PKW}[\text{CF}, \text{NE}]$ 

The  $\text{FIND\$-RCPA+}$  advantage of  $\mathcal{A}$  is  $\text{Adv}_{\text{PKW}, \mathcal{P}, \mathcal{A}}^{\text{find\$-rcpa+}}(\lambda) = 2 \Pr[\text{G}_{\text{PKW}, \mathcal{P}, \mathcal{A}}^{\text{find\$-rcpa+}}(\lambda)] - 1$ . If  $\text{Adv}_{\text{PKW}, \mathcal{P}, \mathcal{A}}^{\text{find\$-rcpa+}}$  is negligible for all PPT  $\mathcal{A}$ , then we say PKW is  $\text{FIND\$-RCPA+}$  secure with  $\mathcal{P}$ .  $\text{FIND\$-RCPA}$  security and its advantage function  $\text{Adv}_{\text{PKW}, \mathcal{P}, \mathcal{A}}^{\text{find\$-rcpa}}$  are defined analogously.

We made minor modifications from prior definitions for notational convenience in our proof. Backendal, et al. allowed all queries for corrupted users except for  $\text{RO\$-WRAP}$  queries. Preventing all queries for corrupted users is equivalent because the adversary can anyway use the exposed key to perfectly emulate the other oracles. Backendal, et al. allowed queries for previously punctured points unhindered (returning  $\perp$  in  $\text{RO\$-WRAP}$  when  $b = 0$  if it would have returned  $\perp$  when  $b = 1$ ). Preventing all queries for previously punctured points is equivalent for schemes that always return  $\perp$  when wrapping/unwrapping at a previously punctured point and return the given key when puncturing a previously punctured point. We explicitly add an ideal primitive and allow adversary programming thereof to match our other definitions.

**Construction.** We will be analyzing the construction of Backendal, et al. [2], except we formalize it in terms of a reconstrainable CPRF rather than a puncturable PRF. Let  $\text{CF}$  be a reconstrainable constrained function family with  $\text{CF.SS}$  containing  $\text{CF.Inp} \setminus S$  for all  $S \subset \text{CF.Inp}$ .<sup>6</sup> Let  $\text{NE}$  be a nonce-based encryption scheme with  $\text{NE.K} = \text{CF.Out}$ . Let  $n_0$  denote a fixed nonce in  $\text{NE.N}$ . Then  $\text{PKW}[\text{CF}, \text{NE}]$  is defined as shown in Fig. 18. Its header space, message space, and ciphertext length function match that of  $\text{NE}$ . Its tag space is  $\text{CF.Inp}$ .

Its key consists of a constrained key  $K$  for  $\text{CF}$  and a set  $R$  representing where the key has been constrained so far. Key generation constrains the initial  $\text{CF}$  key  $k$  on set  $\text{CF.Inp}(\lambda)$  and returns  $(K, \text{CF.Inp}(\lambda))$ . We expect that  $K = k$ . For wrapping or unwrapping we use the output of  $\text{CF}$  on  $t$  as a key for encrypting or decrypting with  $\text{NE}$ . We puncture keys by constraining them on sets with the given tag removed.

**Constrain Invariance.** When analyzing PKW from the security of  $\text{CF}$ , there is a minor technical gap because the formers security definition repeatedly constrains the same key, using it for evaluations of  $\text{CF}$  and returning it for corruptions. The security definitions for  $\text{CF}$  always use the original key for evaluations and only allow “one-shot” constraining. We bridge this gap by assuming a “constrain invariance” property which captures that these views of  $\text{CF}$  are indistinguishable. Constrain invariance is captured by the game shown in Fig. 19. When  $b = 1$ ,  $\text{CON}$  will always reconstrain keys, and  $\text{CEV}$  will evaluate using the current constrained key. When  $b = 0$ , a one-time constraining only happens in  $\text{CORRUPT}$ , and  $\text{CEV}$  always evaluates using the root key.

The advantage of  $\mathcal{A}$  is  $\text{Adv}_{\text{CF}, \mathcal{P}, \mathcal{A}}^{\text{coninv}^*}(\lambda) = 2 \Pr[\text{G}_{\text{CF}, \mathcal{P}, \mathcal{A}}^{\text{coninv}^*}(\lambda)] - 1$ . If  $\text{Adv}_{\text{CF}, \mathcal{P}, \mathcal{A}}^{\text{coninv}^*}$  is negligible for all PPT  $\mathcal{A}$ , then we say  $\text{CF}$  is constrain invariant with  $\mathcal{P}$ .

**Security Result.** The following theorem captures that security of PKW follows from the security of its underlying components.

<sup>6</sup> It is natural to build  $\text{CF}$  from a prefix-constrained function family by creating  $\text{CF.Inp} \setminus S$  as a union of prefix sets.



| Game $G_{CF,P,\mathcal{A}}^{\text{coninv}^*}(\lambda)$                                                                                             | $\text{PPRIM}(\text{Op}, x, y)$                                                                |
|----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| $\sigma_P \leftarrow \$ P.\text{Init}(1^\lambda)$                                                                                                  | Return $P.\text{Op}(1^\lambda, x, y : \sigma_P)$                                               |
| If $b = 1$ : $k_{(\cdot)} \leftarrow \$ CF.K(\lambda)$<br>$K_{(\cdot)} \leftarrow CF.\text{Con}^P(1^\lambda, k_{(\cdot)}, CF.\text{Inp}(\lambda))$ | $\text{CEV}(u, \mathbf{x})$<br>Require $\mathbf{x} \in R_u$ and $\text{corr}_u = \text{false}$ |
| If $b = 0$ : $k_{(\cdot)} \leftarrow \$ CF.K(\lambda)$                                                                                             | If $b = 1$ : $y \leftarrow CF.\text{CEV}^P(1^\lambda, K_u, R_u, \mathbf{x})$                   |
| $R_{(\cdot)} \leftarrow CF.\text{Inp}(\lambda)$                                                                                                    | If $b = 0$ : $y \leftarrow CF.\text{Ev}^P(1^\lambda, k_u, \mathbf{x})$                         |
| $b \leftarrow \$ \{0, 1\}$                                                                                                                         | Return $y$                                                                                     |
| $b' \leftarrow \$ \mathcal{A}^{\text{CON}, \text{CEV}, \text{CORRUPT}, \text{PPRIM}}(1^\lambda)$                                                   | $\text{CORRUPT}(u)$<br>Require $\text{corr}_u = \text{false}$                                  |
| Return $(b = b')$                                                                                                                                  | $\text{corr}_u \leftarrow \text{true}$                                                         |
| $\text{CON}(u, R')$<br>Require $R' \subset R_u$ and $\text{corr}_u = \text{false}$                                                                 | If $b = 1$ : Return $K_u$                                                                      |
| If $b = 1$ : $K_u \leftarrow CF.\text{CCon}^P(1^\lambda, K_u, R_u, R')$                                                                            | If $b = 0$ : Return $CF.\text{Con}^P(1^\lambda, k_u, R_u)$                                     |
| If $b = 0$ : // No-op                                                                                                                              |                                                                                                |
| $R_u \leftarrow R'$                                                                                                                                |                                                                                                |
| Return // No value is returned                                                                                                                     |                                                                                                |

Fig. 19. Game defining constrain invariance of CF

**Theorem 7.** Suppose CF is  $\text{SIM}^*\text{-AC-CPRF}$  secure and constraint invariant with P and NE is  $\text{SIM}^*\text{-AC-MR\$}$  secure with P. Then  $\text{PKW}[\text{CF}, \text{NE}]$  is  $\text{FIND\$}^*\text{-RCPA+}$  secure with P.

*Proof.* The proof proceeds via a series of hybrids  $G_0$  to  $G_3$ , switching from the original security game to one where the view of the adversary is independent of the bit it is guessing. In these hybrids we first switch from repeatedly constraining the CF key to using the original key for evaluations and doing a “one-shot” constraining when a user is corrupted using constrain invariance, then switch CF from real to simulated when used in *both* wrapping oracles, then switch NE from real to simulated when used in *both* wrapping oracles. From given adversary  $\mathcal{A}$ , we build adversaries  $\mathcal{A}_{\text{coninv}^*}$ ,  $\mathcal{A}_{\text{CF}}$ , and  $\mathcal{A}_{\text{NE}}$  for which we establish the following claims:

1.  $\Pr[G_0(\lambda)] = \Pr[G_{\text{PKW}[\text{CF}, \text{NE}], P, \mathcal{A}}^{\text{find\$-rcpa+}}(\lambda)]$
2.  $\Pr[G_0(\lambda)] - \Pr[G_1(\lambda)] = \text{Adv}_{CF, P, \mathcal{A}_{\text{coninv}^*}}^{\text{coninv}^*}(\lambda)$
3.  $\Pr[G_1(\lambda)] - \Pr[G_2(\lambda)] = \text{Adv}_{CF, P, S_{\text{CF}}, \mathcal{A}_{\text{CF}}}^{\text{sim}^*\text{-ac-cprf}}(\lambda)$
4.  $\Pr[G_2(\lambda)] - \Pr[G_3(\lambda)] = \text{Adv}_{NE, P, S_{\text{NE}}, \mathcal{A}_{\text{NE}}}^{\text{sim}^*\text{-ac-mr\$}}(\lambda)$
5.  $\Pr[G_3(\lambda)] = 1/2$ .

Therefore,

$$\begin{aligned}
\text{Adv}_{\text{PKW}[\text{CF}, \text{NE}], P, \mathcal{A}}^{\text{find\$-rcpa+}}(\lambda) &= 2 \Pr[G_{\text{PKW}[\text{CF}, \text{NE}], P, \mathcal{A}}^{\text{find\$-rcpa+}}(\lambda)] - 1 \\
&= 2 \cdot \left( \sum_{i=0}^2 (\Pr[G_i(\lambda)] - \Pr[G_{i+1}(\lambda)]) + \Pr[G_3(\lambda)] \right) - 1 \\
&= 2 \cdot \left( \text{Adv}_{CF, P, \mathcal{A}_{\text{coninv}^*}}^{\text{coninv}^*}(\lambda) + \text{Adv}_{CF, P, S_{\text{CF}}, \mathcal{A}_{\text{CF}}}^{\text{sim}^*\text{-ac-cprf}}(\lambda) + \text{Adv}_{NE, P, S_{\text{NE}}, \mathcal{A}_{\text{NE}}}^{\text{sim}^*\text{-ac-mr\$}}(\lambda) + 1/2 \right) - 1 \\
&= 2 \cdot \left( \text{Adv}_{CF, P, \mathcal{A}_{\text{coninv}^*}}^{\text{coninv}^*}(\lambda) + \text{Adv}_{CF, P, S_{\text{CF}}, \mathcal{A}_{\text{CF}}}^{\text{sim}^*\text{-ac-cprf}}(\lambda) + \text{Adv}_{NE, P, S_{\text{NE}}, \mathcal{A}_{\text{NE}}}^{\text{sim}^*\text{-ac-mr\$}}(\lambda) \right).
\end{aligned}$$

The adversaries we construct are PPT (assuming given algorithms are), so this implies the result by letting  $S_{\text{CF}}$  and  $S_{\text{NE}}$  be the simulators assumed to exist by the security of CF and NE.

**Claims 1 and 2.** All of the hybrids are defined in Fig. 20 where comments indicate which lines of code are included in which games. We start with  $G_0$  which was obtained by plugging the code of  $\text{PKW}[\text{CF}, \text{NE}]$ ’s algorithms into  $G_{\text{PKW}, P, \mathcal{A}}^{\text{find\$-rcpa+}}(\lambda)$ . User  $u$ ’s key is  $(K_u, R_u)$  for which it will always hold that  $R_u \sqcup P_u = \text{CF}.\text{Inp}(\lambda)$ ,

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><u>Games <math>G_i(\lambda), i \in \{0, \dots, 3\}</math></u></p> <p><math>\sigma_P \leftarrow \\$ P.\text{Init}(1^\lambda)</math></p> <p><math>k_{(\cdot)} \leftarrow \\$ CF.K(\lambda) \parallel i &lt; 2</math></p> <p><math>\sigma_{CF} \leftarrow \\$ S_{CF}.\text{Init}(1^\lambda) \parallel 2 \leq i</math></p> <p><math>sk_{(\cdot, \cdot)} \leftarrow \\$ NE.K(\lambda) \parallel 2 \leq i &lt; 3</math></p> <p><math>\sigma_{NE} \leftarrow \\$ S_{NE}.\text{Init}(1^\lambda) \parallel 3 \leq i</math></p> <p><math>K_{(\cdot)} \leftarrow CF.\text{Con}^P(1^\lambda, k_{(\cdot)}, CF.\text{Inp}(\lambda)) \parallel i &lt; 1</math></p> <p><math>R_{(\cdot)} \leftarrow CF.\text{Inp}(\lambda)</math></p> <p><math>b \leftarrow \\$ \{0, 1\}</math></p> <p><math>b' \leftarrow \\$ \mathcal{A}^{\text{RO\\$-WRAP, WRAP, PUNC, CORRUPT, PPRIM}}(1^\lambda)</math></p> <p>Return <math>(b = b')</math></p> <p><u>PPRIM(<math>Op, x, y</math>)</u></p> <p><math>y \leftarrow \\$ P.Op(1^\lambda, x, y : \sigma_P)</math></p> <p>Return <math>y</math></p> <p><u>WRAP(<math>u, t, h, m</math>)</u></p> <p>Require <math>(t, h, m) \notin W_u</math></p> <p>Require <math>t \in R_u</math> and <math>\text{corr}_u = \text{false}</math></p> <p><math>sk_{u,t} \leftarrow CF.CEV^P(1^\lambda, K_u, R_u, t) \parallel i &lt; 1</math></p> <p><math>sk_{u,t} \leftarrow CF.Ev^P(1^\lambda, k_u, t) \parallel 1 \leq i &lt; 2</math></p> <p><math>E_u \leftarrow E_u \cup \{t\}</math></p> <p><math>c \leftarrow NE.\text{Enc}^P(1^\lambda, sk_{u,t}, n_0, h, m) \parallel i &lt; 3</math></p> <p><math>c \leftarrow T_{u,t}[n_0, h, m] \leftarrow \\$ \{0, 1\}^{\text{NE.cl}(\lambda,  m )} \parallel 3 \leq i</math></p> <p><math>W_u \leftarrow W_u \cup \{(t, h, m)\}</math></p> <p>Return <math>c</math></p> | <p><u>PUNC(<math>u, t</math>)</u></p> <p>Require <math>t \in R_u</math> and <math>\text{corr}_u = \text{false}</math></p> <p><math>K_u \leftarrow CF.CCon^P(1^\lambda, K_u, R_u, R_u \setminus \{t\}) \parallel i &lt; 1</math></p> <p><math>R_u \leftarrow R_u \setminus \{t\}</math></p> <p>Return</p> <p><u>CORRUPT(<math>u</math>)</u></p> <p>Require <math>V_u \cap R_u = \emptyset</math></p> <p>Require <math>\text{corr}_u = \text{false}</math></p> <p><math>\text{corr}_u \leftarrow \text{true}</math></p> <p><math>K_u \leftarrow CF.\text{Con}^P(1^\lambda, k_u, R_u) \parallel 1 \leq i &lt; 2</math></p> <p>For <math>t \in R_u \cap E_u</math>: <math>\parallel 2 \leq i</math></p> <p><math>f'(t) \leftarrow sk_{u,t} \parallel 2 \leq i &lt; 3</math></p> <p><math>f'(t) \leftarrow \\$ S_{NE}.\text{Exp}^{\text{PPRIM}}(1^\lambda, (u, t), T_{u,t} : \sigma_{NE}) \parallel 3 \leq i</math></p> <p><math>K_u \leftarrow S_{CF}.\text{Con}^{\text{PPRIM}}(1^\lambda, u, R_u, f' : \sigma_{CF}) \parallel 2 \leq i</math></p> <p>Return <math>(K_u, R_u)</math></p> <p><u>RO\\$-WRAP(<math>u, t, h, m</math>)</u></p> <p>Require <math>(t, h, m) \notin W_u</math></p> <p>Require <math>t \in R_u</math> and <math>\text{corr}_u = \text{false}</math></p> <p>If <math>b = 1</math>: <math>sk_{u,t} \leftarrow CF.CEV^P(1^\lambda, K_u, R_u, t) \parallel i &lt; 1</math></p> <p><math>sk_{u,t} \leftarrow CF.Ev^P(1^\lambda, k_u, t) \parallel 1 \leq i &lt; 2</math></p> <p><math>E_u \leftarrow E_u \cup \{t\}</math></p> <p><math>c \leftarrow NE.\text{Enc}^P(1^\lambda, sk_{u,t}, n_0, h, m) \parallel i &lt; 3</math></p> <p><math>c \leftarrow T_{u,t}[n_0, h, m] \leftarrow \\$ \{0, 1\}^{\text{NE.cl}(\lambda,  m )} \parallel 3 \leq i</math></p> <p>If <math>b = 0</math>: <math>c \leftarrow \\$ \{0, 1\}^{\text{NE.cl}(\lambda,  m )}</math></p> <p><math>V_u \leftarrow V_u \cup \{t\}</math></p> <p><math>W_u \leftarrow W_u \cup \{(t, h, m)\}</math></p> <p>Return <math>y</math></p> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 20.** Hybrids  $G_0$  through  $G_3$  for analyzing PKW[CF, NE]. Comments indicate which hybrids a line is included in. Note that for a transition into hybrid  $G_j$ , comments that end with  $j$  are added and comments that begin with  $j$  are removed.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><u>Adversary <math>\mathcal{A}_{\text{coninv}}^{\text{CON, CORRUPT, CEV, PPRIM}}(1^\lambda)</math></u></p> <p><math>R_{(\cdot)} \leftarrow CF.\text{Inp}(\lambda)</math></p> <p><math>b \leftarrow \\$ \{0, 1\}</math></p> <p><math>b' \leftarrow \\$ \mathcal{A}^{\text{RO\\$-WRAP, WRAP, PUNC, CORRUPT}', \text{PPRIM}}(1^\lambda)</math></p> <p>If <math>(b = b')</math>: Return 1</p> <p>Return 0</p> <p><u>RO\\$-WRAP(<math>u, t, h, m</math>)</u></p> <p>Require <math>(t, h, m) \notin W_u</math></p> <p>Require <math>t \in R_u</math> and <math>\text{corr}_u = \text{false}</math></p> <p>If <math>b = 1</math>: <math>sk \leftarrow CEV(u, t)</math></p> <p><math>c \leftarrow NE.\text{Enc}^P(1^\lambda, sk, n_0, h, m)</math></p> <p>If <math>b = 0</math>: <math>c \leftarrow \\$ \{0, 1\}^{\text{NE.cl}(\lambda,  m )}</math></p> <p><math>V_u \leftarrow V_u \cup \{t\}</math></p> <p><math>W_u \leftarrow W_u \cup \{(t, h, m)\}</math></p> <p>Return <math>y</math></p> | <p><u>WRAP(<math>u, t, h, m</math>)</u></p> <p>Require <math>(t, h, m) \notin W_u</math></p> <p>Require <math>t \in R_u</math> and <math>\text{corr}_u = \text{false}</math></p> <p><math>sk \leftarrow CEV(u, t)</math></p> <p><math>c \leftarrow NE.\text{Enc}^P(1^\lambda, sk, n_0, h, m)</math></p> <p><math>W_u \leftarrow W_u \cup \{(t, h, m)\}</math></p> <p>Return <math>c</math></p> <p><u>PUNC(<math>u, t</math>)</u></p> <p>Require <math>t \in R_u</math> and <math>\text{corr}_u = \text{false}</math></p> <p><math>CON(u, R_u \setminus \{t\})</math></p> <p><math>R_u \leftarrow R_u \setminus \{t\}</math></p> <p>Return</p> | <p><u>CORRUPT'(<math>u</math>)</u></p> <p>Require <math>V_u \cap R_u = \emptyset</math></p> <p>Require <math>\text{corr}_u = \text{false}</math></p> <p><math>\text{corr}_u \leftarrow \text{true}</math></p> <p><math>K_u \leftarrow CORRUPT(u)</math></p> <p>Return <math>(K_u, R_u)</math></p> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 21.** Constrain invariance adversary  $\mathcal{A}_{\text{coninv}}^*$  used to switch from  $G_0$  to  $G_1$  in analysis of PKW[CF, NE]

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Adversary <math>\mathcal{A}_{\text{CF}}^{\text{CON, EV, PPRIM}}(1^\lambda)</math></p> <p><math>R_{(\cdot)} \leftarrow \text{CF.Inp}(\lambda)</math></p> <p><math>b \leftarrow \\$ \{0, 1\}</math></p> <p><math>b' \leftarrow \\$ \mathcal{A}^{\text{RO\\$-WRAP, WRAP, PUNC, CORRUPT, PPRIM}}(1^\lambda)</math></p> <p>If <math>(b = b')</math>: Return 1</p> <p>Return 0</p> <p><u>PUNC(<math>u, t</math>)</u></p> <p>Require <math>t \in R_u</math> and <math>\text{corr}_u = \text{false}</math></p> <p><math>R_u \leftarrow R_u \setminus \{t\}</math></p> <p>Return</p> <p><u>WRAP(<math>u, t, h, m</math>)</u></p> <p>Require <math>(t, h, m) \notin W_u</math></p> <p>Require <math>t \in R_u</math> and <math>\text{corr}_u = \text{false}</math></p> <p><math>sk_{u,t} \leftarrow \text{EV}(u, t)</math></p> <p><math>c \leftarrow \text{NE.Enc}^P(1^\lambda, sk_{u,t}, n_0, h, m)</math></p> <p><math>W_u \leftarrow W_u \cup \{(t, h, m)\}</math></p> <p>Return <math>c</math></p> | <p><u>CORRUPT(<math>u</math>)</u></p> <p>Require <math>V_u \cap R_u = \emptyset</math></p> <p>Require <math>\text{corr}_u = \text{false}</math></p> <p><math>\text{corr}_u \leftarrow \text{true}</math></p> <p><math>K_u \leftarrow \text{CON}(u, R_u)</math></p> <p>Return <math>(K_u, R_u)</math></p> <p><u>RO\\$-WRAP(<math>u, t, h, m</math>)</u></p> <p>Require <math>(t, h, m) \notin W_u</math></p> <p>Require <math>t \in R_u</math> and <math>\text{corr}_u = \text{false}</math></p> <p>If <math>b = 1</math>: <math>sk_{u,t} \leftarrow \text{EV}(u, t)</math></p> <p><math>c \leftarrow \text{NE.Enc}^P(1^\lambda, sk_{u,t}, n_0, h, m)</math></p> <p>If <math>b = 0</math>: <math>c \leftarrow \\$ \{0, 1\}^{\text{NE.cl}(\lambda,  m )}</math></p> <p><math>V_u \leftarrow V_u \cup \{t\}</math></p> <p><math>W_u \leftarrow W_u \cup \{(t, h, m)\}</math></p> <p>Return <math>y</math></p> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Fig. 22. SIM\*-AC-CPRF adversary  $\mathcal{A}_{\text{CF}}$  used to switch from  $G_1$  to  $G_2$ 

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Adversary <math>\mathcal{A}_{\text{NE}}^{\text{ENC, EXP, PPRIM}}(1^\lambda)</math></p> <p><math>\sigma_{\text{CF}} \leftarrow \\$ \text{S}_{\text{CF}}.\text{Init}(1^\lambda)</math></p> <p><math>R_{(\cdot)} \leftarrow \text{CF.Inp}(\lambda)</math></p> <p><math>b \leftarrow \\$ \{0, 1\}</math></p> <p><math>b' \leftarrow \\$ \mathcal{A}^{\text{RO\\$-WRAP, WRAP, PUNC, CORRUPT, PPRIM}}(1^\lambda)</math></p> <p>If <math>(b = b')</math>: Return 1</p> <p>Return 0</p> <p><u>PUNC(<math>u, t</math>)</u></p> <p>Require <math>t \in R_u</math> and <math>\text{corr}_u = \text{false}</math></p> <p><math>R_u \leftarrow R_u \setminus \{t\}</math></p> <p>Return</p> <p><u>WRAP(<math>u, t, h, m</math>)</u></p> <p>Require <math>(t, h, m) \notin W_u</math></p> <p>Require <math>t \in R_u</math> and <math>\text{corr}_u = \text{false}</math></p> <p><math>E_u \leftarrow E_u \cup \{t\}</math></p> <p><math>c \leftarrow \text{ENC}((u, t), n_0, h, m)</math></p> <p><math>W_u \leftarrow W_u \cup \{(t, h, m)\}</math></p> <p>Return <math>c</math></p> | <p><u>CORRUPT(<math>u</math>)</u></p> <p>Require <math>V_u \cap R_u = \emptyset</math></p> <p>Require <math>\text{corr}_u = \text{false}</math></p> <p><math>\text{corr}_u \leftarrow \text{true}</math></p> <p>For <math>t \in R_u \cap E_u</math>: <math>f'(t) \leftarrow \text{EXP}((u, t))</math></p> <p><math>K_u \leftarrow \text{S}_{\text{CF}}.\text{Con}^{\text{PPRIM}}(1^\lambda, u, R_u, f' : \sigma_{\text{CF}})</math></p> <p>Return <math>(K_u, R_u)</math></p> <p><u>RO\\$-WRAP(<math>u, t, h, m</math>)</u></p> <p>Require <math>(t, h, m) \notin W_u</math></p> <p>Require <math>t \in R_u</math> and <math>\text{corr}_u = \text{false}</math></p> <p>If <math>b = 1</math>: <math>E_u \leftarrow E_u \cup \{t\}</math></p> <p><math>c \leftarrow \text{ENC}((u, t), n_0, h, m)</math></p> <p>If <math>b = 0</math>: <math>c \leftarrow \\$ \{0, 1\}^{\text{NE.cl}(\lambda,  m )}</math></p> <p><math>V_u \leftarrow V_u \cup \{t\}</math></p> <p><math>W_u \leftarrow W_u \cup \{(t, h, m)\}</math></p> <p>Return <math>y</math></p> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Fig. 23. SIM\*-AC-MR\\$ adversary  $\mathcal{A}_{\text{NE}}$  used to switch from  $G_2$  to  $G_3$  in analysis of  $\text{PKW}[\text{CF}, \text{NE}]$

so we replace all uses of  $P$  with  $R$ . We added set  $E_u$  to track the inputs that CF is evaluated on. Hence our first claim, that  $\Pr[G_0(\lambda)] = \Pr[G_{\text{PKW}[\text{CF}, \text{NE}], P, \mathcal{A}}^{\text{find\$-rcpa+}}(\lambda)]$ , follows.

Hybrid  $G_1$  switches the usage of CF.CCon and CF.CEv to CF.Con and CF.Ev. Naturally, we bound the difference between  $G_0$  and  $G_1$  by constrain invariance of CF. The relevant adversary  $\mathcal{A}_{\text{coninv}^*}$  is given in Fig. 21. It simulates these games by querying its oracles for all uses of CF. The requirements of  $\mathcal{A}$ 's oracles ensure that  $\mathcal{A}_{\text{coninv}^*}$  satisfies the requirements of its own oracles. Correctness of this simulation gives our second claim, that  $\Pr[G_0(\lambda)] - \Pr[G_1(\lambda)] = \text{Adv}_{\text{CF}, P, \mathcal{A}_{\text{coninv}^*}}^{\text{coninv}^*}(\lambda)$ .

**Claim 3.** Hybrid  $G_2$  switches the output of CF to random (and its keys to simulated). Naturally, we bound the difference between  $G_1$  and  $G_2$  by SIM\*-AC-CPRF security of CF. The relevant adversary  $\mathcal{A}_{\text{CF}}$  is given in Fig. 22. It simulates these games by querying its oracles for all uses of CF. The requirements of  $\mathcal{A}$ 's oracles ensure that  $\mathcal{A}_{\text{CF}}$  only makes queries for  $u$  with  $C_u = \emptyset$ , which allowed us to simplify code in  $G_2$ . The game samples the  $sk$ 's as the beginning of the game and uses them to define the restricted function  $f'$  in CORRUPT when needed. Correctness of this simulation gives our third claim, that  $\Pr[G_1(\lambda)] - \Pr[G_2(\lambda)] = \text{Adv}_{\text{CF}, P, S_{\text{CF}}, \mathcal{A}_{\text{CF}}}^{\text{sim}^*-\text{ac-cprf}}(\lambda)$ .

**Claim 4.** Hybrid  $G_3$  switches the output of NE to random (and its keys to simulated). Naturally, we bound the difference between  $G_2$  and  $G_3$  by SIM\*-AC-MR\$ security of CF. The relevant adversary  $\mathcal{A}_{\text{NE}}$  is given in Fig. 23. It simulates these games by querying its oracles for all uses of NE, where key  $sk_{u,t}$  corresponds to a user  $(u, t)$ . The requirements of  $\mathcal{A}$ 's oracles ensure that  $\mathcal{A}_{\text{NE}}$  only makes queries for which the ciphertext will be chosen at random (the user is not corrupted and the query repeated), which allowed us to simplify code regarding  $T$  in  $G_3$ . Correctness of this simulation gives our fourth claim, that  $\Pr[G_2(\lambda)] - \Pr[G_3(\lambda)] = \text{Adv}_{\text{NE}, P, S_{\text{NE}}, \mathcal{A}_{\text{NE}}}^{\text{sim}^*-\text{ac-mr\$}}(\lambda)$ .

**Claim 5.** Hybrid  $G_3$  only uses  $b$  in RO\$-WRAP. The ciphertext  $c$  is uniformly random and independent of  $b$ , but when  $b = 1$  the tag  $t$  gets added to  $E_u$  and table  $T_{u,t}$  is modified. These variables could only be used in the for loop of CORRUPT. So because that oracle ensures  $V_u \cap R_u = \emptyset$ , we will have  $t \notin R_u$  for such tags. Hence the differing behavior based on  $b$  cannot affect the view of  $\mathcal{A}$ . This gives our fifth claim, that  $\Pr[G_3(\lambda)] = 1/2$ .  $\square$

## D Proofs for Hashed Constrained PRF(s)

### D.1 Proof of Theorem 2 (Hashed[CF] is SIM\*-AC-CPRF secure)

*Proof.* Our proof will proceed in a series of hybrids. We construct a series of hybrids  $G_0$  to  $G_3$  switching from the real world of SIM\*-AC-CPRF to the simulated world. The first transition ( $G_0$  to  $G_1$ ) is bounded by a bad event that  $\text{CF.Ev}(k, x) = y$  inside of a  $\text{Ev}(u, x)$  query and while the  $x$  has not been constrained for  $u$ , the adversary manages to query the random oracle on  $y$  or a different Ev query that also computes  $y$  from  $\text{CF.Ev}(k', x')$ . The difference between these games is bounded by the advantage of an IND\*-NC-CPRF adversary against CF because these events will be unlikely if  $y$  is random. IND\*-NC-CPRF security suffices because the adversary will guess  $(u, x)$  and query for it at the start of the game. This required that P be non-programmable so that making this query at the start of the game did not change the distribution of Ev. Game  $G_2$  is a syntactic rewriting of  $G_1$  to prepare for the transition to  $G_3$ . This latter transition is bounded by a bad event that two different executions of  $\text{CF.Ev}(k, x)$  for an unexposed user gives different results. If we assumed that P was consistent (an eminently reasonable assumption), then the probability of this would be zero. Without that assumption, we instead make a reduction to the IND\*-PRF security of CF that repeatedly queries for all  $(u, x)$  pairs and checks whether the outputs are consistent. (Here we are using that the random world of IND\*-PRF security always returns consistent random values.) Now in  $G_3$  we have reached a game that can be captured by the simulated world of SIM\*-AC-CPRF with a simulator that attempts to reprogram the random oracle appropriately on constrain queries and otherwise simply runs the honest algorithms of CF.

We will call our simulator S. Let  $\mathcal{A}_{\text{cprf}^*}$  be the given adversary against Hashed[CF]. Assume it makes at most  $p(\lambda)$  queries to PPRIM and  $q(\lambda)$  queries to Ev. Assume that  $N(\lambda)$  is a set such that whenever  $\mathcal{A}_{\text{cprf}^*}$

queries  $\text{Ev}(u, x)$  it holds that  $(u, x) \in N$ . From  $\mathcal{A}_{\text{cprf}^*}$  (and  $N$ ) we will construct  $\mathcal{A}_{\text{cprf}}$  and  $\mathcal{A}_{\text{prf}}$ . Then the following claims will hold:

1.  $\Pr[\mathbf{G}_0(\lambda)] = \Pr[\mathbf{G}_{\text{Hashed}[\text{CF}], S, P \times P_{\text{rom}}, \mathcal{A}_{\text{cprf}^*}}^{\text{sim}^*-\text{ac-cprf}}(\lambda) | b = 1]$
2.  $\Pr[\mathbf{G}_0(\lambda)] - \Pr[\mathbf{G}_1(\lambda)] \leq |N(\lambda)| \cdot (\text{Adv}_{\text{CF}, P, \mathcal{A}_{\text{cprf}}}^{\text{ind}^*-\text{nc-cprf}}(\lambda) + \frac{q(\lambda) + p(\lambda)}{\min_x |\text{CF.Out}(\lambda, x)|})$
3.  $\Pr[\mathbf{G}_1(\lambda)] = \Pr[\mathbf{G}_2(\lambda)]$
4.  $\Pr[\mathbf{G}_2(\lambda)] - \Pr[\mathbf{G}_3(\lambda)] \leq \text{Adv}_{\text{CF}, P, \mathcal{A}_{\text{prf}}}^{\text{ind}^*-\text{prf}}(\lambda)$
5.  $\Pr[\mathbf{G}_3(\lambda)] = \Pr[\neg \mathbf{G}_{\text{Hashed}[\text{CF}], S, P \times P_{\text{rom}}, \mathcal{A}_{\text{cprf}^*}}^{\text{sim}^*-\text{ac-cprf}}(\lambda) | b = 0]$ .

Putting these claims together gives

$$\begin{aligned} \text{Adv}_{\text{Hashed}[\text{CF}], u, S, P \times P_{\text{rom}}, \mathcal{A}_{\text{cprf}^*}}^{\text{sim}^*-\text{ac-cprf}}(\lambda) &= \Pr[\mathbf{G}_0(\lambda)] - \Pr[\mathbf{G}_3(\lambda)] \\ &= \Pr[\mathbf{G}_0(\lambda)] - \Pr[\mathbf{G}_1(\lambda)] + \Pr[\mathbf{G}_1(\lambda)] - \Pr[\mathbf{G}_2(\lambda)] + \Pr[\mathbf{G}_2(\lambda)] - \Pr[\mathbf{G}_3(\lambda)] \\ &\leq |N(\lambda)| \cdot \text{Adv}_{\text{CF}, P, \mathcal{A}_{\text{cprf}}}^{\text{ind}^*-\text{nc-cprf}}(\lambda) + \frac{|N(\lambda)|(q(\lambda) + p(\lambda))}{\min_x |\text{CF.Out}(\lambda, x)|} + \text{Adv}_{\text{CF}, P, \mathcal{A}_{\text{prf}}}^{\text{ind}^*-\text{prf}}(\lambda). \end{aligned}$$

Then result holds from  $|N(\lambda)|$ ,  $q(\lambda)$ , and  $p(\lambda)$  being polynomial;  $\min_x |\text{CF.Out}(\lambda, x)|$  being super-polynomial; and the assumed security of  $\text{CF}$ . (Note that  $\text{IND}^*\text{-NC-CPRF}$  security directly implies  $\text{IND}^*\text{-PRF}$  security.)

If  $P$  is consistent, then  $\text{Adv}_{\text{CF}, P, \mathcal{A}_{\text{prf}}}^{\text{ind}^*-\text{prf}}(\lambda) = 0$ .

**Claim 1.** Game  $\mathbf{G}_0$  is defined in Fig. 24. It contains highlighted code. Throughout we use  $x$  to refer to inputs to  $\text{CF}$ ,  $y$  to refer to outputs of  $\text{CF}$  or inputs to the random oracle, and  $z$  to refer to outputs of the random oracle (which are often implicitly outputs of  $\text{Hashed}[\text{CF}]$ ).

This game makes explicit the random sampling of the random oracle using table  $\sigma_{\text{rom}}$  and moreover splits the storing of values in the random oracle to also use tables  $T_{u,x}$  (in preparation for the coming game transitions). We store  $T_{u,x}[y] \leftarrow z$  at the end of  $\text{Ev}(u, x)$  when  $y$  is the output computed for  $\text{CF}$  and  $z$  the output of the random oracle. For unconstrained inputs ( $x \notin C_u$ ), we attempt to use  $T_{u,x}$  for our lazy sampling of the random oracle. To prevent inconsistencies with the random oracle as presented by  $\text{PPRIM}$  we check if  $\sigma_{\text{rom}}[y]$  has a value and use that instead, if so. To prevent inconsistencies with  $\text{Ev}$  queries for any other inputs —  $(v, w) \neq (u, x)$  — which computed the same  $y$  we check if  $T_{v,w}[y]$  has a value and use that instead, if so. For constrained inputs ( $x \in C_u$ ) we call  $\text{PPRIM}$  to use  $\sigma_{\text{rom}}$ .

Inside  $\text{PPRIM}$  we similarly attempt to lazily sample the random oracle using  $\sigma_{\text{rom}}$ . To prevent inconsistencies with  $\text{Ev}$  queries for unconstrained inputs we check if any  $T_{u,x}[y]$  has a value for  $x \notin C_u$  and use that instead, if so. When an earlier  $\text{Ev}$  input  $(u, x)$  is first constrained, we tell program  $\sigma_{\text{rom}}$  to be consistent with  $T_{u,x}$ .

That this correctly simulates the real  $\text{SIM}^*\text{-AC-CPRF}$  world (i.e., the first claim holds) follows from observing that the following properties will be invariant.

- Any combination of the  $T_{u,x}$  tables and/or  $\sigma_{\text{rom}}$  which contain non- $\perp$  values all contain the same value.
- Once a table entry for  $y$  is set, it will never be changed.
- The first time a table entry for  $y$  is set, it is set appropriately (randomly in  $\text{Ev}$  or a lazy sampling query to  $\text{PPRIM}$ , non-randomly in a programming query to  $\text{PPRIM}$ ).
- If  $x \in C_u$  and  $T_{u,x}[y] \neq \perp$ , then  $\sigma_{\text{rom}}[y] \neq \perp$ .

**Claim 2.** Game  $\mathbf{G}_1$  is defined in Fig. 24. It does not contain highlighted code. The removed code contains all of the table rewriting to preserve consistency of the random oracle across different types of queries. It is syntactically clear that  $\mathbf{G}_0$  and  $\mathbf{G}_1$  are identical until a  $\text{bad}_{u,x}$  flag is set. The fundamental lemma of game playing [8] gives  $\Pr[\mathbf{G}_0(\lambda)] - \Pr[\mathbf{G}_1(\lambda)] \leq \Pr[\bigvee_{(U,X) \in N(\lambda)} \mathbf{G}_1(\lambda) \text{ sets } \text{bad}_{U,X}]$ .

Consider  $\text{bad}_{U,X}$  for fixed  $U, X$ . Setting this flag in  $\text{PPRIM}$  requires the adversary to query the random oracle on a  $Y = \text{CF.Ev}(k_U, X)$  value while  $X$  has not been constrained for  $U$ . The first place the flag could be set in  $\text{Ev}$  is the same, except that the random oracle query happened first. The second place the flag

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><u>Games <math>G_0(\lambda), G_1(\lambda)</math></u></p> <p><math>\sigma_P \leftarrow \text{P.Init}(1^\lambda); \sigma_{\text{rom}} \leftarrow [\cdot]</math></p> <p><math>k_{(\cdot)} \leftarrow \text{CF.K}(\lambda)</math></p> <p><math>b' \leftarrow \mathcal{A}_{\text{cprf}^*}^{\text{CON}, \text{EV}, \text{PPRIM}}(1^\lambda)</math></p> <p>Return <math>(b' = 1)</math></p> <p><u>PPRIM(<math>\text{Op}, (d, y), z</math>)</u></p> <p>If <math>d = 0</math>: Return <math>\text{P.Op}(1^\lambda, y, z : \sigma_P)</math></p> <p>If <math>\text{Op} = \text{Ls}</math>: <math>z \leftarrow \mathcal{R}_\lambda</math></p> <p>If <math>\sigma_{\text{rom}}[y] = \perp</math>:</p> <p style="padding-left: 20px;"><math>\sigma_{\text{rom}}[y] \leftarrow z</math></p> <p style="padding-left: 20px;">If <math>\exists u, x</math> s.t. <math>T_{u,x}[y] \neq \perp</math> and <math>x \notin C_u</math>:</p> <p style="padding-left: 40px;"><math>\text{bad}_{u,x} \leftarrow \text{true}; \sigma_{\text{rom}}[y] \leftarrow T_{u,x}[y]</math></p> <p>If <math>\text{Op} = \text{Ls}</math>: Return <math>\sigma_{\text{rom}}[y]</math></p> <p>Return <math>\perp</math></p> | <p><u>CON(<math>u, R</math>)</u></p> <p>For <math>x \in R \cap E_u \setminus C_u</math>:</p> <p style="padding-left: 20px;">For <math>y</math> s.t. <math>T_{u,x}[y] \neq \perp</math>:</p> <p style="padding-left: 40px;">If <math>\sigma_{\text{rom}}[y] = \perp</math>: <math>\sigma_{\text{rom}}[y] \leftarrow T_{u,x}[y]</math></p> <p><math>K_R \leftarrow \text{CF.Con}^P(1^\lambda, k_u, R)</math></p> <p><math>C_u \leftarrow C_u \cup R</math></p> <p>Return <math>K_R</math></p> <p><u>EV(<math>u, x</math>)</u></p> <p><math>y \leftarrow \text{CF.EV}^P(1^\lambda, k_u, x)</math></p> <p>If <math>x \notin C_u</math>:</p> <p style="padding-left: 20px;"><math>z \leftarrow \mathcal{R}_\lambda</math></p> <p style="padding-left: 20px;">If <math>T_{u,x}[y] \neq \perp</math>: <math>z \leftarrow T_{u,x}[y]</math></p> <p style="padding-left: 20px;">If <math>\sigma_{\text{rom}}[y] \neq \perp</math>: <math>\text{bad}_{u,x} \leftarrow \text{true}; z \leftarrow \sigma_{\text{rom}}[y]</math></p> <p style="padding-left: 20px;">If <math>\exists (v, w) \neq (u, x)</math> s.t. <math>T_{v,w}[y] \neq \perp</math>:</p> <p style="padding-left: 40px;"><math>\text{bad}_{u,x} \leftarrow \text{true}; z \leftarrow T_{v,w}[y]</math></p> <p>Else: <math>z \leftarrow \text{PPRIM}(\text{Ls}, (1, y), \perp)</math></p> <p><math>E_u \leftarrow E_u \cup \{x\}; T_{u,x}[y] \leftarrow z</math></p> <p>Return <math>z</math></p> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Fig. 24. Hybrids  $G_0$  and  $G_1$  for analyzing Hashed[CF]

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><u>Adversary <math>\mathcal{A}_{(U,X)}^{\text{EV}, \text{CON}, \text{PPRIM}}(1^\lambda)</math></u></p> <p><math>\sigma_{\text{rom}} \leftarrow [\cdot]</math></p> <p>For <math>t = 1, \dots, q(\lambda)</math>:</p> <p style="padding-left: 20px;"><math>Y_t \leftarrow \text{EV}(U, X)</math></p> <p><math>t \leftarrow 0</math></p> <p><math>\mathcal{A}_{\text{cprf}^*}^{\text{CONSIM}, \text{EVSIM}, \text{PPRIMSIM}}(1^\lambda)</math></p> <p>Return 0</p> <p><u>PPRIMSIM(<math>\text{Op}, (d, y), z</math>)</u></p> <p>If <math>d = 0</math>: Return <math>\text{PPRIM}(\text{Op}, y, z)</math></p> <p>If <math>\text{Op} = \text{Ls}</math>: <math>z \leftarrow \mathcal{R}_\lambda</math></p> <p>If <math>\sigma_{\text{rom}}[y] = \perp</math>:</p> <p style="padding-left: 20px;"><math>\sigma_{\text{rom}}[y] \leftarrow z</math></p> <p style="padding-left: 20px;">If <math>\exists u, x</math> s.t. <math>T_{u,x}[y] \neq \perp</math> and <math>x \notin C_u</math>:</p> <p style="padding-left: 40px;">If <math>(u, x) = (U, X)</math>: Abort 1</p> <p>If <math>\text{Op} = \text{Ls}</math>: Return <math>\sigma_{\text{rom}}[y]</math></p> <p>Return <math>\perp</math></p> | <p><u>CONSIM(<math>u, R</math>)</u></p> <p>If <math>u = U</math> and <math>X \in R</math>: Abort 0</p> <p>For <math>x \in R \cap E_u \setminus C_u</math>:</p> <p style="padding-left: 20px;">For <math>y</math> s.t. <math>T_{u,x}[y] \neq \perp</math>:</p> <p style="padding-left: 40px;">If <math>\sigma_{\text{rom}}[y] = \perp</math>: <math>\sigma_{\text{rom}}[y] \leftarrow T_{u,x}[y]</math></p> <p><math>K_R \leftarrow \text{CON}(u, R)</math></p> <p><math>C_u \leftarrow C_u \cup R</math></p> <p>Return <math>K_R</math></p> <p><u>EVSIM(<math>u, x</math>)</u></p> <p>If <math>(u, x) = (U, X)</math>: <math>y \leftarrow Y_t; t \leftarrow t + 1</math></p> <p>Else <math>y \leftarrow \text{CON}(u, \{x\})</math></p> <p>If <math>x \notin C_u</math>:</p> <p style="padding-left: 20px;"><math>z \leftarrow \mathcal{R}_\lambda</math></p> <p style="padding-left: 20px;">If <math>T_{u,x}[y] \neq \perp</math>: <math>z \leftarrow T_{u,x}[y]</math></p> <p style="padding-left: 20px;">If <math>\sigma_{\text{rom}}[y] \neq \perp</math> or <math>\exists (v, w) \neq (u, x)</math> s.t. <math>T_{v,w}[y] \neq \perp</math>:</p> <p style="padding-left: 40px;">If <math>(u, x) = (U, X)</math>: Abort 1</p> <p>Else: <math>z \leftarrow \text{PPRIMSIM}(\text{Ls}, (1, y), \perp)</math></p> <p><math>E_u \leftarrow E_u \cup \{x\}; T_{u,x}[y] \leftarrow z</math></p> <p>Return <math>z</math></p> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Fig. 25. Non-adaptive IND\*-NC-CPRF adversary  $\mathcal{A}_{(U,X)}$  for bounding the probability  $\text{bad}_{U,X}$  is set in  $G_1$ . Abort statements indicate that  $\mathcal{A}_{U,X}$  will halt immediately with the specified output.



could be set in Ev requires a Ev query for a different input to have calculated the same  $Y$ . In all these cases,  $Y$  should look uniformly random so the probability of these events should be small.

We formalize this idea with the IND\*-NC-CPRF adversary  $\mathcal{A}_{(U,X)}$  shown in Fig. 25. It attempts to simulate  $G_1$  with the goal of outputting 1 whenever it would set  $\text{bad}_{U,X}$  and outputting 0 otherwise. It simulates the random oracle for itself and uses its oracle to for CF. For constrain queries and evaluation queries other than for  $(U, X)$  it uses its CON oracle. (For the latter we are using our convention that  $\text{CF.Con}(k, \{x\}) = \text{CF.Ev}(k, x)$ .) For simulating evaluation queries on  $(U, X)$  it precomputes a list of CF.Ev outputs at the beginning of execution using Ev and then uses the values from this list to respond to later queries. Note that we can only precompute these values because  $P$  is non-programmable. If we assumed  $P$  was consistent, then CF.Ev would be too and we would only have needed to make a single Ev query. If  $\text{bad}_{U,X}$  would ever be set,  $\mathcal{A}_{(U,X)}$  aborts early and returns 1. It cannot simulate constrain queries that include  $(U, X)$  so, as  $\text{bad}_{U,X}$  can no longer be set after such a query, it aborts early and returns 0.

When  $\mathcal{A}_{(U,X)}$ 's oracles are honestly running CF, it correctly simulates  $G_1$  and so  $\Pr[G_1(\lambda) \text{ sets } \text{bad}_{U,X}] = \Pr[G_{\text{CF},P,\mathcal{A}_{(U,X)}}^{\text{ind}^*\text{-ac-cprf}}(\lambda) | b = 1]$ . Consider when its oracles are simulated. The  $Y_t$ 's are uniformly random and all equal to each other. Each query to the random oracle query (whether directly or through an evaluation query with  $x \in C_u$ ) and each evaluation query for a different input with  $x \notin C_u$  can be viewed as giving a single guess to the value of  $Y_1$ . There are at most  $q(\lambda) + p(\lambda)$  such queries, each of which have an at most  $1/|\text{CF.Out}(\lambda, X)|$  probability of being correct so a union bound gives  $\Pr[\neg G_{\text{CF},P,\mathcal{A}_{(U,X)}}^{\text{ind}^*\text{-ac-cprf}}(\lambda) | b = 0] \leq (q(\lambda) + p(\lambda))/|\text{CF.Out}(\lambda, X)|$ . Hence  $\Pr[G_1(\lambda) \text{ sets } \text{bad}_{U,X}] \leq \text{Adv}_{\text{CF},P,\mathcal{A}_{U,X}}^{\text{ind}^*\text{-nc-cprf}}(\lambda) + (q(\lambda) + p(\lambda))/|\text{CF.Out}(\lambda, X)|$ .

A union bound over all of the possible  $(U, X)$  values in  $N(\lambda)$  gives that  $\Pr[G_0(\lambda)] - \Pr[G_1(\lambda)] \leq \left( \sum_{U,X} \text{Adv}_{\text{CF},P,\mathcal{A}_{U,X}}^{\text{ind}^*\text{-nc-cprf}}(\lambda) + (q(\lambda) + p(\lambda))/|\text{CF.Out}(\lambda, X)| \right)$ . Now letting  $\mathcal{A}_{\text{cprf}}$  be the adversary that samples  $(U, X) \leftarrow N(\lambda)$  then runs  $\mathcal{A}_{U,X}$  gives us the second claim  $\Pr[G_0(\lambda)] - \Pr[G_1(\lambda)] \leq |N(\lambda)| \cdot (\text{Adv}_{\text{CF},P,\mathcal{A}_{\text{cprf}}}^{\text{ind}^*\text{-nc-cprf}}(\lambda) + \frac{q(\lambda) + p(\lambda)}{\min_x |\text{CF.Out}(\lambda, x)|})$ .

**Claim 3.** Game  $G_1$  is in some senses “most of the way” to the conclusion of the proof. However, in this game  $\text{Ev}(u, x)$  may return different results across repeated queries for inputs that have not been constrained if different  $y$ 's are computed inside of them. However, the simulated world of SIM\*-AC-CPRF ensures repeated queries are answered consistently for such inputs. Our next pair of games will switch from using tables  $T_{u,x}[y]$  to using  $T_u[x]$ , removing the dependence on  $y$  that causes inconsistent behavior.

Consider  $G_2$  defined in Fig. 26. It contains highlighted but not boxed code. In Ev, we use  $T_u[x]$  in place of  $T_{u,x}[y]$  if they are equal. If they are unequal, we set  $\text{bad}$  and use  $T_{u,x}$  as before. In CON, we break down the reprogramming of the random oracle. First we compute  $Y = \text{CF.Ev}(k_u, x)$ . (Non-programmability of  $P$  ensures this does not change the adversary's view.) If the set of  $y$ 's computed in prior  $\text{Ev}(u, x)$  queries (computed by checking if  $T_{u,x}[y]$  is empty) is not a singleton set containing only  $Y$ , we set  $\text{bad}$  and program  $\sigma_{\text{rom}}$  as before. Otherwise, if  $T_u[x]$  and  $T_{u,x}[Y]$  are unequal, we set  $\text{bad}$  and program  $\sigma_{\text{rom}}$  using  $T_{u,x}[Y]$  as before. If both of those pass, we program  $\sigma_{\text{rom}}$  using  $T_u[x]$ .

This logic ensures that  $T_u[x]$  and  $Y$  are used precisely when they give the same result as would have been obtained before (and otherwise we fall back to not using them). Consequently,  $G_2$  is equivalent to  $G_1$  giving the third claim that  $\Pr[G_1(\lambda)] = \Pr[G_2(\lambda)]$ .

**Claim 4.** Consider  $G_3$  also in Fig. 26 which contains boxed but not highlighted code. Whenever  $G_2$  would fall back on acting like  $G_1$  because using  $T_u[x]$  and  $Y$  would behave incorrectly,  $G_3$  just uses them anyway. It is syntactically clear that  $G_2$  and  $G_3$  are identical until the  $\text{bad}$  flag is set. Hence,  $\Pr[G_2(\lambda)] - \Pr[G_3(\lambda)] \leq \Pr[G_3(\lambda) \text{ sets } \text{bad}]$ .

It is not hard to verify that if for all  $(u, x)$  executing  $\text{CF.Ev}(k_u, x)$  always returns the same value, then  $\text{bad}$  will never be set. So we can bound the probability of  $\text{bad}$  by the probability we ever see inconsistent outputs of CF.Ev. If  $P$  is consistent, the probability of this is 0. Otherwise, we can reduce this to the IND\*-PRF security of CF. This is captured by the adversary  $\mathcal{A}_{\text{prf}}$  given in Fig. 27. It simply makes  $q(\lambda) + 1$  queries for each possible Ev query and outputs 1 if it sees any inconsistency. Because  $P$  is non-programmable, querying Ev at the beginning of the game does not change the distribution of its outputs and

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Games <math>\mathbf{G}_2(\lambda), \mathbf{G}_3(\lambda)</math></b><br>$\sigma_P \leftarrow \mathcal{P}.\text{Init}(1^\lambda); \sigma_{\text{rom}} \leftarrow [\cdot]$<br>$k_{(\cdot)} \leftarrow \mathcal{CF}.\text{K}(\lambda)$<br>$b' \leftarrow \mathcal{A}_{\text{cprf}^*}^{\text{CON}, \text{Ev}, \text{PPRIM}}(1^\lambda)$<br>Return $(b' = 1)$<br>$\text{PPRIM}(\text{Op}, (d, y), z)$<br>If $d = 0$ : Return $\mathcal{P}.\text{Op}(1^\lambda, y, z : \sigma_P)$<br>If $\text{Op} = \text{Ls}$ : $z \leftarrow \mathcal{R}_\lambda$<br>If $\sigma_{\text{rom}}[y] = \perp$ : $\sigma_{\text{rom}}[y] \leftarrow z$<br>If $\text{Op} = \text{Ls}$ : Return $\sigma_{\text{rom}}[y]$<br>Return $\perp$ | <b>CON(<math>u, R</math>)</b><br>For $x \in R \cap E_u \setminus C_u$ :<br>$Y \leftarrow \mathcal{CF}.\text{Ev}^P(1^\lambda, k_u, x)$<br>If $\{Y\} \neq \{y : T_{u,x}[y] \neq \perp\}$ :<br>$\text{bad} \leftarrow \text{true}$<br>For $y$ s.t. $T_{u,x}[y] \neq \perp$ :<br>If $\sigma_{\text{rom}}[y] = \perp$ : $\sigma_{\text{rom}}[y] \leftarrow T_{u,x}[y]$<br>If $\sigma_{\text{rom}}[Y] = \perp$ : $\sigma_{\text{rom}}[Y] \leftarrow T_u[x]$<br>Else if $T_{u,x}[Y] \neq T_u[x]$ :<br>$\text{bad} \leftarrow \text{true}$<br>If $\sigma_{\text{rom}}[Y] = \perp$ : $\sigma_{\text{rom}}[Y] \leftarrow T_{u,x}[Y]$<br>If $\sigma_{\text{rom}}[Y] = \perp$ : $\sigma_{\text{rom}}[Y] \leftarrow T_u[x]$<br>Else if $\sigma_{\text{rom}}[Y] = \perp$ : $\sigma_{\text{rom}}[Y] \leftarrow T_u[x]$<br>$K_R \leftarrow \mathcal{CF}.\text{Con}^P(1^\lambda, k_u, R)$<br>$C_u \leftarrow C_u \cup R$<br>Return $K_R$ | <b>Ev(<math>u, x</math>)</b><br>$y \leftarrow \mathcal{CF}.\text{Ev}^P(1^\lambda, k_u, x)$<br>If $x \notin C_u$ :<br>$z \leftarrow \mathcal{R}_\lambda$<br>If $T_{u,x}[y] \neq T_u[x]$ :<br>$\text{bad} \leftarrow \text{true}$<br>If $T_{u,x}[y] \neq \perp$ : $z \leftarrow T_{u,x}[y]$<br>If $T_u[x] \neq \perp$ : $z \leftarrow T_u[x]$<br>Else if $T_u[x] \neq \perp$ : $z \leftarrow T_u[x]$<br>Else: $z \leftarrow \text{PPRIM}(\text{Ls}, (1, y), \perp)$<br>$E_u \leftarrow E_u \cup \{x\}$<br>$T_{u,x}[y] \leftarrow z; T_u[x] \leftarrow z$<br>Return $z$ |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 26.** Hybrids  $\mathbf{G}_2$  and  $\mathbf{G}_3$  for analyzing Hashed[CF]

|                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Adversary <math>\mathcal{A}_{\text{prf}}^{\text{Ev}, \text{PPRIM}}(1^\lambda)</math></b><br>For $(u, x) \in N(\lambda)$ :<br>For $i = 1, \dots, q(\lambda) + 1$ :<br>$y_{u,x,i} \leftarrow \text{Ev}(u, x)$<br>If $\exists u, x, i, j$ s.t. $y_{u,x,i} \neq y_{u,x,j}$ :<br>Return 1<br>Return 0 | <b>S.Init()</b><br>$k_{(\cdot)} \leftarrow \mathcal{CF}.\text{K}(\lambda)$<br>Return $k_{(\cdot)}$<br><b>S.Ev<sup>PPRIM</sup>(<math>u, x : k</math>)</b><br>$y \leftarrow \text{Hashed[CF]}. \text{Ev}^{P \times P_{\text{rom}}}(1^\lambda, k_u, x)$<br>Return $y$ | <b>S.Con<sup>PPRIM</sup>(<math>u, R, f : k</math>)</b><br>For $x \in f.D$ :<br>$Y \leftarrow \mathcal{CF}.\text{Ev}^P(1^\lambda, k_u, x)$<br>$\text{PPRIM}(\text{Prog}, (1, Y), T_u[x])$<br>Return $\mathcal{CF}.\text{Con}^P(1^\lambda, k, R)$ |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 27.** IND\*-PRF adversary  $\mathcal{A}_{\text{prf}}$  for bounding the probability of bad in  $\mathbf{G}_3$ **Fig. 28.** Simulator S for Hashed[CF]

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Games <math>\mathbf{G}_4(\lambda)</math></b><br>$\sigma_P \leftarrow \mathcal{P}.\text{Init}(1^\lambda); \sigma_{\text{rom}} \leftarrow [\cdot]$<br>$k_{(\cdot)} \leftarrow \mathcal{CF}.\text{K}(\lambda)$<br>$b' \leftarrow \mathcal{A}_{\text{cprf}^*}^{\text{CON}, \text{Ev}, \text{PPRIM}}(1^\lambda)$<br>Return $(b' = 1)$<br>$\text{PPRIM}(\text{Op}, (d, y), z)$<br>If $d = 0$ : Return $\mathcal{P}.\text{Op}(1^\lambda, y, z : \sigma_P)$<br>Else: Return $\mathcal{P}_{\text{rom}}.\text{Op}(1^\lambda, y, z : \sigma_{\text{rom}})$ | <b>CON(<math>u, R</math>)</b><br>For $x \in R \cap E_u \setminus C_u$ :<br>$Y \leftarrow \mathcal{CF}.\text{Ev}^P(1^\lambda, k_u, x)$<br>$\text{PPRIM}(\text{Prog}, (1, Y), T_u[x])$<br>$K_R \leftarrow \mathcal{CF}.\text{Con}^P(1^\lambda, k_u, R)$<br>$C_u \leftarrow C_u \cup R$<br>Return $K_R$ | <b>Ev(<math>u, x</math>)</b><br>$y \leftarrow \mathcal{CF}.\text{Ev}^P(1^\lambda, k_u, x)$<br>If $x \notin C_u$ :<br>$z \leftarrow \mathcal{R}_\lambda$<br>If $T_u[x] \neq \perp$ : $z \leftarrow T_u[x]$<br>Else: $z \leftarrow \text{PPRIM}(\text{Ls}, (1, y), \perp)$<br>$E_u \leftarrow E_u \cup \{x\}$<br>$T_u[x] \leftarrow z$<br>Return $z$ |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 29.** Hybrid  $\mathbf{G}_4$  for analyzing Hashed[CF]

so  $\Pr[G_3(\lambda) \text{ sets bad}] \leq \Pr[G_{CF,P,A_{prf}}^{\text{ind}^*-prf}(\lambda) | b = 1]$ . The outputs of  $\text{Ev}$  are forced to be consistent in the random world, so  $\Pr[\neg G_{CF,P,A_{prf}}^{\text{ind}^*-prf}(\lambda) | b = 0] = 0$ . This gives,  $\Pr[G_3(\lambda) \text{ sets bad}] \leq \text{Adv}_{CF,P,A_{prf}}^{\text{ind}^*-prf}(\lambda)$  and hence the claim.

**Claim 5.** For the final claim, first consider game  $G_4$  in Fig. 29. It was obtained by copying  $G_3$ , simplifying the if statements that are identical in all branches, simplifying PPRIM, and replacing explicit programming of  $\sigma_{\text{rom}}$  in CON with the equivalent query to PPRIM. We have  $\Pr[G_3(\lambda)] = \Pr[G_4(\lambda)]$ .

Now we can naturally see that this game is identical to the simulated  $\text{SIM}^*\text{-AC-CPRF}$  world if we use the simulator given in Fig. 28. The function  $f_u$  is lazily sampled as table  $T_u$ . There are extra  $\text{CF.Ev}$  queries in  $\text{Ev}$  for unconstrained inputs, but non-programmability of  $P$  ensures this does not change the view. The simulator is quite natural, it runs the honest evaluation for constrained inputs. For generating constrained keys, it first tries to program the random oracle to be consistent with prior evaluations and then runs the honest constrain algorithm. The fifth claim holds,  $\Pr[G_3(\lambda)] = \Pr[\neg G_{\text{Hashed}[CF],S,P \times P_{\text{rom}},A_{\text{cprf}}^*}^{\text{sim}^*\text{-ac-cprf}}(\lambda) | b = 0]$ .  $\square$

**Proof variant using  $\text{IND}^*\text{-AC-CPRF}$ .** It seems likely that if we are willing to assume  $\text{IND}^*\text{-AC-CPRF}$  security rather than  $\text{IND}^*\text{-NC-CPRF}$ , then the theorem would hold without requiring  $N(\lambda)$  be a priori fixed. Adversary  $A_{\text{cprf}}$  could instead guess  $i \in \{1, \dots, q(\lambda)\}$  corresponding to the first time  $\text{Ev}$  will be queried for a  $(U, X)$  that causes the bad event, then query  $\text{Ev}$  for calculating  $\text{CF.Ev}(U, X)$  then and all future times it is needed. It can check whether  $\text{bad}_{U,X}$  has been set at the end of the game.

Similarly for  $A_{\text{prf}}$ , it could guess the first time  $\text{Ev}$  will be queried for a  $(U, X)$  that will cause **bad** by giving inconsistent responses. It can then immediately make the  $q(\lambda) + 1$  queries for  $(U, X)$  and check for collisions.

In both of these reductions, the adaptivity should let it simulate the view of  $A_{\text{cprf}}^*$  before knowing what will be queried to  $\text{Ev}$ .

## D.2 Proof of Theorem 3 (TDCF is $\text{SIM}^*\text{-AC-CPRF}$ secure)

**Trapdoor Permutations.** We start by providing formal syntax for a trapdoor permutation and writing the code of TDCF in terms of this syntax. We recall the standard security definitions for a trapdoor permutation.

A trapdoor permutation  $\Pi$  specifies key generation algorithm  $\Pi.\text{Kg}$ , evaluation algorithm  $\Pi.\text{Ev}$ , and inverse algorithm  $\Pi.\text{Inv}$ . It also specifies sets  $\Pi.\text{Inp}$ . Their syntaxes are  $(tk, \text{pub}) \leftarrow \Pi.\text{Kg}(1^\lambda)$ ,  $ST' \leftarrow \Pi.\text{Ev}^P(1^\lambda, \text{pub}, ST)$ , and  $ST \leftarrow \Pi.\text{Inv}^P(1^\lambda, tk, ST')$ . Here  $ST, ST' \in \Pi.\text{Inp}(\lambda, \text{pub})$ . We often write  $\Pi_{\text{pub}}^n(\cdot)$  for the  $n$ -fold repetition of  $\Pi.\text{Ev}^P(1^\lambda, \text{pub}, \cdot)$  and  $\Pi_{tk}^{-n}(\cdot)$  for the  $n$ -fold repetition of  $\Pi.\text{Inv}^P(1^\lambda, tk, \cdot)$ . Correctness implies that  $\Pi_{\text{pub}}$  is a permutation with inverse  $\Pi_{tk}^{-1}$ .

One-wayness security of  $\Pi$  is defined as follows where PPRIM is defined in the usual way:

$$\text{Adv}_{\Pi,P,A}^{\text{ow}^*}(\lambda) = \Pr[ST = \mathcal{A}^{\text{PPRIM}}(1^\lambda, \text{pub}, \Pi_{\text{pub}}(ST)) : (tk, \text{pub}) \leftarrow \Pi.\text{Kg}(1^\lambda); ST \leftarrow \Pi.\text{Inp}(\lambda, \text{pub})].$$

If  $\text{Adv}_{\Pi,P,A}^{\text{ow}^*}$  is negligible for all PPT  $\mathcal{A}$ , we say that  $\Pi$  is  $\text{OW}^*$  secure with  $P$ .

**Analysis.** Now we proceed to the actual proof of Theorem 3.

*Proof.* Recall that  $\Pi$  can be expressed as  $\text{Hashed}[CF]$  where  $\text{CF.Kg}(1^\lambda)$  samples  $(tk, \text{pub}) \leftarrow \Pi.\text{Kg}(1^\lambda)$  and  $ST_0 \leftarrow \Pi.\text{Inp}(\lambda, \text{pub})$ , then returns  $k = (tk, \text{pub}, ST_0)$  and its algorithms are defined as

$$\begin{aligned} \text{CF.Ev}(1^\lambda, k, 1^c) &= \Pi_{tk}^{-c}(ST_0) \\ \text{CF.Con}(1^\lambda, k, R_n) &= (\text{pub}, \Pi_{tk}^{-n}(ST_0)) \\ \text{CF.CEv}(1^\lambda, (\text{pub}, ST_n), R_n, c) &= \Pi_{\text{pub}}^{n-c}(ST_n). \end{aligned}$$

This technically does not follow our formalism for constrained function families because its output space,  $\Pi.\text{Inp}(\lambda, \text{pub})$ , depends on the key and because it has a key generation algorithm instead of a keyspace. It's clear these differences are not material for the proof.

Our proofs follows from the same sequence of hybrids (replacing  $\text{CF.K}(\lambda)$  with  $\text{CF.Kg}(1^\lambda)$ ) and claims as the proof of Theorem 2 given in Appendix D.1 except we will replace the use of CPRF security for  $\text{CF}$ . In particular, the new claims are

| Adversary $\mathcal{A}_{(U,X)}^{\text{PPRIM}}(1^\lambda, \text{pub}, ST_{ X -1})$ | $\text{CON}(u, R)$                                                  | $\text{EV}(u, x)$                                                |
|-----------------------------------------------------------------------------------|---------------------------------------------------------------------|------------------------------------------------------------------|
| $\sigma_{\text{rom}} \leftarrow [\cdot]$                                          | If $u = U$ and $X \in R$ : Goto Location 1                          | If $u \neq U$ : $y \leftarrow \text{CF.EV}^P(1^\lambda, k_u, x)$ |
| $k_{(\cdot)} \leftarrow \text{CF.KG}(1^\lambda)$                                  | For $x \in R \cap E_u \setminus C_u$ :                              | Else if $ x  <  X $ : $y \leftarrow ST_{ x }$                    |
| $k_U \leftarrow \perp$                                                            | If $u \neq U$ : $Y \leftarrow \text{CF.EV}^P(1^\lambda, k_u, x)$    | Else if $ x  >  X $ : $\parallel x \notin C_u$                   |
| For $i =  X  - 1, \dots, 1$ :                                                     | Else: $Y \leftarrow ST_{ x } \parallel  x  \leq  X  - 1$            | $y \leftarrow \Pi_{\text{pub}}^{ x - X -1}(ST_{ X -1})$          |
| $ST_{i-1} \leftarrow \Pi_{\text{pub}}(ST_i)$                                      | $\text{PPRIM}(\text{Prog}, (1, Y), T_u[x])$                         | Else $y \leftarrow \perp \parallel x = X \notin C_u$             |
| $S \leftarrow \emptyset$                                                          | If $u \neq U$ : $K_R \leftarrow \text{CF.Con}^P(1^\lambda, k_u, R)$ | $S \leftarrow S \cup \{y\}$                                      |
| $b' \leftarrow \mathcal{A}_{\text{cprf}^*}^{\text{CON, Ev, PPRIMSim}}(1^\lambda)$ | Else $K_R \leftarrow (\text{pub}, ST_{ R -1})$                      | If $x \notin C_u$ :                                              |
| $\parallel$ Location 1                                                            | $\parallel  R  - 1 \leq  X  - 1$                                    | $z \leftarrow \mathcal{R}_\lambda$                               |
| For $y \in S$ :                                                                   | $C_u \leftarrow C_u \cup R$                                         | If $T_u[x] \neq \perp$ : $z \leftarrow T_u[x]$                   |
| If $\Pi_{\text{pub}}(y) = ST_{ X -1}$ : Return $y$                                | Return $K_R$                                                        | Else: $z \leftarrow \text{PPRIM}(\text{Ls}, (1, y), \perp)$      |
| Return $\perp$                                                                    |                                                                     | $E_u \leftarrow E_u \cup \{x\}$                                  |
| $\text{PPRIM}(\text{Op}, (d, y), z)$                                              |                                                                     | $T_u[x] \leftarrow z$                                            |
| If $d = 0$ : Return $\text{P.Op}(1^\lambda, y, z : \sigma_P)$                     |                                                                     | Return $z$                                                       |
| $S \leftarrow S \cup \{y\}$                                                       |                                                                     |                                                                  |
| Return $\text{P}_{\text{rom}}.\text{Op}(1^\lambda, y, z : \sigma_{\text{rom}})$   |                                                                     |                                                                  |

**Fig. 30.** OW\* adversary  $\mathcal{A}_{(U,X)}$  for bounding the probability  $\text{bad}_{U,X}$  is set in  $\mathbf{G}_1$  (equivalently,  $\mathbf{G}_4$ ). We use that if  $X = 1^\ell$ , then  $|X| = \ell$  and if  $R = R_n = \{1^i : 0 \leq i \leq n\}$ , then  $|R| - 1 = n$ .

2.  $\Pr[\mathbf{G}_0(\lambda)] - \Pr[\mathbf{G}_1(\lambda)] \leq |N(\lambda)| \cdot \text{Adv}_{\Pi, \text{P}, \mathcal{A}}^{\text{OW}^*}(\lambda)$
4.  $\Pr[\mathbf{G}_2(\lambda)] - \Pr[\mathbf{G}_3(\lambda)] = 0$

Let  $u$  denote an upper bound on how many different users  $\mathcal{A}_{\text{cprf}^*}$  will make queries for and  $l$  denote an upper bound on the longest input it will query to  $\text{EV}$ . Then  $\mathcal{A}_{\text{cprf}^*}$  will only make  $\text{EV}$  queries in  $N(\lambda) = \{1, \dots, u(\lambda)\} \times \{1^n : 0 \leq n \leq l(\lambda)\}$ . (The restriction to user in  $\{1, \dots, u(\lambda)\}$  is without loss of generality by a simple renaming argument.)

The new fourth claim follows from the logic mentioned in the proof of Theorem 2 in Appendix D.1 that the difference between  $\mathbf{G}_2$  and  $\mathbf{G}_3$  would be 0 if  $\text{P}$  was consistent. For the new second claim, we define adversaries  $\mathcal{A}_{U,X}$  such that  $\Pr[\mathbf{G}_1 \text{ sets } \text{bad}_{U,X}] \leq \text{Adv}_{\Pi, \text{P}, \mathcal{A}_{U,X}}^{\text{OW}^*}(\lambda)$  and then let  $\mathcal{A}$  pick  $(U, X) \leftarrow N(\lambda)$  and run  $\mathcal{A}_{U,X}$ .

Recall from the proof of Theorem 2 that  $\mathbf{G}_1$  will only set  $\text{bad}_{U,X}$  if (while  $X$  has not been constrained for  $U$ )  $Y = \text{CF.EV}(k_U, X)$  is queried to the random oracle or internally computed inside of a  $\text{EV}$  query for inputs other than  $(U, X)$ . If  $(tk, \text{pub}, ST_0)$  is the key of user  $U$ , then define  $ST_i = \Pi_{tk}^{-i}(ST_0)$  for positive  $i$  (for one corner case we also need  $ST_{-1} = \Pi_{\text{pub}}(ST_0)$ ). Note that because  $ST_0$  is uniformly random and  $\Pi_{\text{pub}}$  is a permutation with inverse  $\Pi_{tk}^{-1}$ , sampling any of the  $ST_i$  at random and deriving the others from it gives an identical distribution. Consequently, our adversary will treat its input as  $ST_{|X|-1}$ . It stores the relevant  $y$ 's queries to the random oracle or sampled inside  $\text{EV}$  in a set  $S$ , then at the end of the game or when  $\mathcal{A}_{\text{cprf}^*}$  makes a constrain query including  $(U, X)$  it checks if any of these are a  $\Pi_{\text{pub}}$  preimage of its input, winning if so. If  $\text{bad}_{U,X}$  would have been set, such a  $y$  must have been found.

Code of the adversary  $\mathcal{A}_{U,X}$  is given in Fig. 30. For simplicity, we wrote it to match  $\mathbf{G}_4$  from Fig. 29 which we already argued is equivalent to  $\mathbf{G}_1$ . For users other than  $U$ , it honestly executes the algorithms of  $\text{CF}$ . For  $U$  it precomputes  $ST_i$  for  $i = |X| - 1, \dots, 0$ . It uses these values as the output of  $\text{CF.EV}$  or  $\text{CF.Con}$  whenever relevant. Consistency and non-programmability of  $\text{P}$  ensures this precomputing does not change the distribution of the game. Because we abort whenever  $X$  (and hence also anything longer than  $X$ ) would be constrained for  $U$ , we never need any  $ST_i$  for  $i \geq |X|$  to respond properly to  $\mathcal{A}_{\text{cprf}^*}$ . However, we still want to capture if some  $(U, x)$  with  $|x| > |X|$  is queried to  $\text{EV}$  which would compute a  $y$  equal to  $ST_{|X|}$ . If this was the case, then  $ST_{|x|} = y = ST_{|X|}$  so we have found a cycle in  $\Pi_{\text{pub}}$ . Thus applying  $\Pi_{\text{pub}}$  to  $ST_{|X|-1}$  the appropriate number of times  $(|x| - |X| - 1)$  would recover  $ST_{|x|}$ . So we add  $\Pi_{\text{pub}}^{|x|-|X|-1}(ST_{|X|-1})$  to  $S$  for this case. Adversary  $\mathcal{A}_{(U,X)}$  correctly simulates the view of  $\mathcal{A}_{\text{cprf}^*}$  (up to when  $(U, X)$  would be constrained and hence  $\text{bad}_{U,X}$  could no longer be set) and wins its game whenever  $\text{bad}_{U,X}$  would occur. Hence  $\Pr[\mathbf{G}_1 \text{ sets } \text{bad}_{U,X}] \leq \text{Adv}_{\Pi, \text{P}, \mathcal{A}_{U,X}}^{\text{OW}^*}(\lambda)$  from which the second claim follows.  $\square$

## E Proof of Theorem 4 (Security of Composed Construction)

We prove security of  $\text{CMP}[\mathbf{F}, \mathbf{H}]$  security assuming security of the underlying constrained PRFs. In Appendix E.1, we prove  $\text{IND}^*\text{-NC-CPRF}$  security. In Appendix E.2, we prove  $\text{SIM}^*\text{-AC-CPRF}$  security.

### E.1 Proof of $\text{IND}^*\text{-NC-CPRF}$ Security of $\text{CMP}$

The proof of  $\text{IND}^*\text{-NC-CPRF}$  for  $\text{CMP}[\mathbf{F}, \mathbf{H}]$  is fairly straightforward. We switch outputs of  $\mathbf{H}$ , then  $\mathbf{F}$  to random. The only subtlety is that an attacker can query  $(x', x'')$  to  $\text{Ev}$  and  $R_{x', S}$  to  $\text{Con}$  as long as  $x'' \notin S$ . Then when the  $\mathbf{F}$  key that  $\mathbf{H}$  derives on input  $x'$  is switched to random in  $\text{Ev}$ , we must also use the appropriate random value in  $\text{Con}$ . If  $\mathbf{P}$  was consistent, then the  $\mathbf{H}$  adversary could simply reuse this key in  $\text{Con}$ . We are trying to avoid that assumption, so we instead use our convention that  $\text{IND}^*\text{-NC-CPRF}$  allow  $\text{Ev}$  queries after  $\text{Con}$  queries as long as they are repeats of earlier  $\text{Ev}$  queries.

We construct a series of hybrid games  $\mathbf{G}_0$  to  $\mathbf{G}_3$  (Fig. 31), with  $\mathbf{G}_0$  being  $\mathbf{G}^{\text{ind}^*\text{-nc-cprf}}$  and  $\mathbf{G}_3$  being a game where the adversary's view is independent of the bit  $b$ . Hybrid  $\mathbf{G}_0$  is obtained by plugging in the code of  $\text{CMP}[\mathbf{F}, \mathbf{H}]$ , while  $\mathbf{G}_1$  does basic rewriting so that when a query  $\text{Con}(u, R_{x', S})$  is made where no  $\text{Ev}(u, (x', x''))$  query has been made, we replace evaluation of  $\mathbf{H}$  on  $x'$  with constraining  $\mathbf{H}$  on  $\{x\}$ . Then  $\mathbf{G}_2$  switches the output of  $\mathbf{H}.\text{Ev}$  to random everywhere it is used (inside of  $\text{Ev}$  queries or  $\text{Con}(u, R_{x', S})$  queries when a prior  $\text{Ev}(u, (x', x''))$  query was made). Game  $\mathbf{G}_3$  switches the output of  $\mathbf{F}.\text{Ev}$  to random in  $\text{Ev}$ . We define  $\text{IND}^*\text{-NC-CPRF}$  adversaries  $\mathcal{B}$  and  $\mathcal{C}$  (Fig. 32) and establish the following claims.

1.  $\Pr[\mathbf{G}_0(\lambda)] = \Pr[\mathbf{G}_{\text{CMP}[\mathbf{F}, \mathbf{H}], \mathbf{P}, \mathcal{A}_{\text{cprf}}}^{\text{ind}^*\text{-nc-cprf}}(\lambda)]$
2.  $\Pr[\mathbf{G}_1(\lambda)] = \Pr[\mathbf{G}_0(\lambda)]$
3.  $\Pr[\mathbf{G}_2(\lambda)] - \Pr[\mathbf{G}_1(\lambda)] \leq \text{Adv}_{\mathbf{H}, \mathbf{P}, \mathcal{B}}^{\text{ind}^*\text{-nc-cprf}}(\lambda)$
4.  $\Pr[\mathbf{G}_3(\lambda)] - \Pr[\mathbf{G}_2(\lambda)] \leq \text{Adv}_{\mathbf{F}, \mathbf{P}, \mathcal{C}}^{\text{ind}^*\text{-nc-cprf}}(\lambda)$
5.  $\Pr[\mathbf{G}_3(\lambda)] \leq 1/2$

Therefore,

$$\begin{aligned}
 \text{Adv}_{\text{CMP}[\mathbf{F}, \mathbf{H}], \mathbf{P}, \mathcal{A}_{\text{cprf}}}^{\text{ind}^*\text{-nc-cprf}}(\lambda) &= 2 \Pr[\mathbf{G}_{\text{CMP}[\mathbf{F}, \mathbf{H}], \mathbf{P}, \mathcal{A}_{\text{cprf}}}^{\text{ind}^*\text{-nc-cprf}}(\lambda)] - 1 \\
 &= 2 \Pr[\mathbf{G}_1(\lambda)] - 1 \\
 &= 2 \Pr[\mathbf{G}_1(\lambda)] - 2 \Pr[\mathbf{G}_2(\lambda)] + 2 \Pr[\mathbf{G}_2(\lambda)] - 2 \Pr[\mathbf{G}_3(\lambda)] + 2 \Pr[\mathbf{G}_3(\lambda)] - 1 \\
 &\leq 2 \text{Adv}_{\mathbf{H}, \mathbf{P}, \mathcal{B}}^{\text{ind}^*\text{-nc-cprf}}(\lambda) + 2 \text{Adv}_{\mathbf{F}, \mathbf{P}, \mathcal{C}}^{\text{ind}^*\text{-nc-cprf}}(\lambda)
 \end{aligned}$$

Hybrid games  $\mathbf{G}_2$  and  $\mathbf{G}_3$  are identical when  $b = 0$ , so one can modify  $\mathcal{C}$  and the analysis thereof to remove the factor 2 in front of its advantage.

**Claim 1.** Hybrid game  $\mathbf{G}_0$  why defined by simply plugging the code of  $\text{CMP}[\mathbf{F}, \mathbf{H}]$  into  $\mathbf{G}^{\text{ind}^*\text{-nc-cprf}}$ . This gives the first claim.

**Claim 2.** Hybrid game  $\mathbf{G}_1$  splits the behavior of  $\text{Con}$  on input sets of the form  $R = R_{x', S}$ . If  $x'$  was part of a prior query to  $\text{Ev}$  for the same user, then the behavior is as before. Otherwise,  $\mathbf{H}.\text{Ev}^{\mathbf{P}}(1^\lambda, k_u, x')$  is replaced with  $\mathbf{H}.\text{Con}^{\mathbf{P}}(1^\lambda, k_u, \{x'\})$ . The second claim follows by our convention that the latter simply runs the former.

**Claim 3.** Hybrid game  $\mathbf{G}_2$  switches the output of  $\mathbf{H}.\text{Ev}$  to being picked by a random function (both in  $\text{Ev}$  and  $\text{Con}$ ). This is bounded by the adversary  $\mathcal{B}$  which simulates  $\mathbf{G}_1$  or  $\mathbf{G}_2$  for  $\mathcal{A}_{\text{cprf}}$ , replacing uses of  $\mathbf{H}$  with oracle queries. Note that the  $\text{Ev}$  queries inside of its simulation of constrain (which may occur after it has already queried  $\text{Con}$ ) can only be repeats of earlier  $\text{Ev}$  queries. Thus, it correctly simulates the view in either of the two games giving the third claim.

| Games $G_i(\lambda), i \in \{0, \dots, 3\}$                                     | $\text{CON}(u, R)$                                                              | $\text{EV}(u, \mathbf{x})$                                                |
|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| $\sigma_P \leftarrow \$ \text{P.Init}(1^\lambda)$                               | Require $R \cap E_u = \emptyset$                                                | Require $\mathbf{x} \notin C_u$                                           |
| $k_{(\cdot)} \leftarrow \$ \text{H.K}(\lambda)$                                 | If $R = R_S$ :                                                                  | Require $C_u = \emptyset$ or $\mathbf{x} \in E_u$                         |
| $f_{(\cdot)}^H \leftarrow \$ \text{H.Fcs}(\lambda) \quad // 2 \leq i$           | $K_R \leftarrow \text{H.Con}^P(1^\lambda, k_u, S)$                              | If $b = 1$ :                                                              |
| $f_{(\cdot)}^F \leftarrow \$ \text{F.Fcs}(\lambda) \quad // 3 \leq i$           | Else if $R = R_{x', S}$ :                                                       | If $\mathbf{x} \in \text{H.Inp}_1$ :                                      |
| $f_{(\cdot)} \leftarrow \$ \text{CMP}[\text{F}, \text{H}].\text{Fcs}(\lambda)$  | $K_{u, x'} \leftarrow \text{H.Ev}^P(1^\lambda, k_u, x') \quad // i < 1$         | $y \leftarrow \text{H.Ev}^P(1^\lambda, k_u, \mathbf{x}) \quad // i < 2$   |
| $b \leftarrow \$ \{0, 1\}$                                                      | $K_R \leftarrow \text{F.Con}^P(1^\lambda, K_{u, x'}, S) \quad // i < 1$         | $y \leftarrow f_u^H(\mathbf{x}) \quad // 2 \leq i$                        |
| $b' \leftarrow \$ \mathcal{A}_{\text{cprf}}^{\text{CON, EV, PPRIM}}(1^\lambda)$ | If $\exists x''$ s.t. $(x', x'') \in E_u$ : $// 1 \leq i$                       | Else if $\mathbf{x} = (x', x'') \in \text{H.Inp}_2 \times \text{F.Inp}$ : |
| Return $(b = b')$                                                               | $K_{u, x'} \leftarrow \text{H.Ev}^P(1^\lambda, k_u, x') \quad // 1 \leq i < 2$  | $K_{u, x'} \leftarrow \text{H.Ev}^P(1^\lambda, k_u, x') \quad // i < 2$   |
| $\text{PPRIM}(\text{Op}, x, y)$                                                 | $K_{u, x'} \leftarrow f_u^H(x') \quad // 2 \leq i$                              | $K_{u, x'} \leftarrow f_u^H(x') \quad // 2 \leq i < 3$                    |
| Return $\text{P.Op}(1^\lambda, x, y : \sigma_P)$                                | $K_R \leftarrow \text{F.Con}^P(1^\lambda, K_{u, x'}, S) \quad // 1 \leq i$      | $y \leftarrow \text{F.Ev}^P(1^\lambda, K_{u, x'}, x'') \quad // i < 3$    |
|                                                                                 | Else: $// 1 \leq i$                                                             | $y \leftarrow f_{u, x'}^F(x'') \quad // 3 \leq i$                         |
|                                                                                 | $K_{u, x'} \leftarrow \text{H.Con}^P(1^\lambda, k_u, \{x'\}) \quad // 1 \leq i$ | If $b = 0$ : $y \leftarrow f_u(\mathbf{x})$                               |
|                                                                                 | $K_R \leftarrow \text{F.Con}^P(1^\lambda, K_{u, x'}, S) \quad // 1 \leq i$      | $E_u \leftarrow E_u \cup \{\mathbf{x}\}$                                  |
|                                                                                 | $C_u \leftarrow C_u \cup R$                                                     | Return $y$                                                                |
|                                                                                 | Return $K_R$                                                                    |                                                                           |

**Fig. 31.** Hybrid games used for proving IND\*-NC-CPRF security of  $f_{(\cdot)} \leftarrow \$ \text{CMP}[\text{F}, \text{H}].\text{Fcs}(\lambda)$ . Comments indicate which hybrids a line is included in. Note that for a transition into hybrid  $G_j$ , comments that end with  $j$  are added and comments that begin with  $j$  are removed.

| Adversary $\mathcal{B}^{\text{CON, EV, PPRIM}}(1^\lambda)$                            | $\text{CONSIM}(u, R)$                                    | $\text{EVSIM}(u, \mathbf{x})$                                             |
|---------------------------------------------------------------------------------------|----------------------------------------------------------|---------------------------------------------------------------------------|
| $f_{(\cdot)} \leftarrow \$ \text{CMP}[\text{F}, \text{H}].\text{Fcs}(\lambda)$        | Require $R \cap E_u = \emptyset$                         | Require $\mathbf{x} \notin C_u$                                           |
| $b \leftarrow \$ \{0, 1\}$                                                            | If $R = R_S$ :                                           | Require $C_u = \emptyset$ or $\mathbf{x} \in E_u$                         |
| $b' \leftarrow \$ \mathcal{A}_{\text{cprf}}^{\text{CONSIM, EVSIM, PPRIM}}(1^\lambda)$ | $K_R \leftarrow \text{CON}(u, S)$                        | If $b = 1$ :                                                              |
| If $b = b'$ : Return 1                                                                | Else if $R = R_{x', S}$ :                                | If $\mathbf{x} \in \text{H.Inp}_1$ :                                      |
| Return 0                                                                              | If $\exists x''$ s.t. $(x', x'') \in E_u$ :              | $y \leftarrow \text{EV}(u, \mathbf{x})$                                   |
|                                                                                       | $K_{u, x'} \leftarrow \text{EV}(u, x')$                  | Else if $\mathbf{x} = (x', x'') \in \text{H.Inp}_2 \times \text{F.Inp}$ : |
|                                                                                       | $K_R \leftarrow \text{F.Con}^P(1^\lambda, K_{u, x'}, S)$ | $K_{x'} \leftarrow \text{EV}(u, x')$                                      |
|                                                                                       | Else:                                                    | $y \leftarrow \text{F.Ev}^P(1^\lambda, K_{u, x'}, x'')$                   |
|                                                                                       | $K_{u, x'} \leftarrow \text{CON}(u, \{x'\})$             | If $b = 0$ : $y \leftarrow f_u(\mathbf{x})$                               |
|                                                                                       | $K_R \leftarrow \text{F.Con}^P(1^\lambda, K_{u, x'}, S)$ | $E_u \leftarrow E_u \cup \{\mathbf{x}\}$                                  |
|                                                                                       | $C_u \leftarrow C_u \cup R$                              | Return $y$                                                                |
|                                                                                       | Return $K_R$                                             |                                                                           |

  

| Adversary $\mathcal{C}^{\text{CON, EV, PPRIM}}(\lambda)$                              | $\text{CONSIM}(u, R)$                                         | $\text{EVSIM}(u, \mathbf{x})$                                             |
|---------------------------------------------------------------------------------------|---------------------------------------------------------------|---------------------------------------------------------------------------|
| $k_{(\cdot)} \leftarrow \$ \text{H.K}(\lambda)$                                       | Require $R \cap E_u = \emptyset$                              | Require $\mathbf{x} \notin C_u$                                           |
| $f_{(\cdot)}^H \leftarrow \$ \text{H.Fcs}(\lambda)$                                   | If $R = R_S$ :                                                | Require $C_u = \emptyset$ or $\mathbf{x} \in E_u$                         |
| $f_{(\cdot)} \leftarrow \$ \text{CMP}[\text{F}, \text{H}].\text{Fcs}(\lambda)$        | $K_R \leftarrow \text{H.Con}^P(1^\lambda, k_u, S)$            | If $b = 1$ :                                                              |
| $b \leftarrow \$ \{0, 1\}$                                                            | Else if $R = R_{x', S}$ :                                     | If $\mathbf{x} \in \text{H.Inp}_1$ :                                      |
| $b' \leftarrow \$ \mathcal{A}_{\text{cprf}}^{\text{CONSIM, EVSIM, PPRIM}}(1^\lambda)$ | If $\exists x''$ s.t. $(x', x'') \in E_u$ :                   | $y \leftarrow f_u^H(\mathbf{x})$                                          |
| If $b = b'$ : Return 1                                                                | $K_R \leftarrow \text{CON}((u, x'), S)$                       | Else if $\mathbf{x} = (x', x'') \in \text{H.Inp}_2 \times \text{F.Inp}$ : |
| Return 0                                                                              | Else:                                                         | $y \leftarrow \text{EV}((u, x'), x'')$                                    |
|                                                                                       | $K_{u, x'} \leftarrow \text{H.Con}^P(1^\lambda, k_u, \{x'\})$ | If $b = 0$ : $y \leftarrow f_u(\mathbf{x})$                               |
|                                                                                       | $K_R \leftarrow \text{F.Con}^P(1^\lambda, K_{u, x'}, S)$      | $E_u \leftarrow E_u \cup \{\mathbf{x}\}$                                  |
|                                                                                       | $C_u \leftarrow C_u \cup R$                                   | Return $y$                                                                |
|                                                                                       | Return $K_R$                                                  |                                                                           |

**Fig. 32.** IND\*-NC-CPRF adversaries  $\mathcal{B}$  (against H, transitioning from  $G_1$  to  $G_2$ ) and  $\mathcal{C}$  (against F, transitioning from  $G_2$  to  $G_3$ ) used to prove security of  $\text{CMP}[\text{F}, \text{H}]$ .



**Claim 4.** Hybrid game  $G_3$  switches the output of  $F.Ev$  to being picked by a random function in  $Ev$ . This is bounded by the adversary  $\mathcal{C}$  which simulates  $G_2$  or  $G_3$  for  $\mathcal{A}_{\text{cprf}}$ , replacing uses of  $F$  with oracle queries whenever  $F$ 's keys are random from the function  $f^H$ . Importantly, the use of  $f^H$  that  $\mathcal{C}$  still handles (for  $x \in H.Inp_1$  in  $Ev$ ) which never overlap with the uses it lets its oracles hand (for  $x' \in H.Inp_1$  in  $Ev$  or  $Con$ ). Thus, it correctly simulates the view in either of the two games giving the fourth claim.

**Claim 5.** Finally, we note that the view of  $\mathcal{A}_{\text{cprf}}$  no longer depends on the bit  $b$ . Its only use is in  $Ev$  to decide whether the output comes from  $f$  or one of  $f^H$  and  $f^F$ . In either case, it is an output of a random function which will not be evaluated on the same inputs anywhere else. Hence, the final claim follows.  $\square$

## E.2 Proof of SIM\*-AC-CPRF Security of CMP

We conclude by proving the SIM\*-AC-CPRF security of CMP. We follow a similar style to the IND\*-NC-CPRF proof in Appendix E.1, replacing first the output  $H$  then  $F$  with randomness/simulation. However, while the conceptual core is the same there are some additional challenges in the low-level details of this proof. In particular, we face a “race condition” between the simulators for  $F$  and  $H$  as discussed in the main body. In addressing these two challenges, we will use two tables: **CKey** to keep track of constrained keys that have already been returned to the adversary and **Own** to keep track of which simulator “owns” (and thus will pick) the intermediate key corresponding to some  $x \in H.Inp_1$ .

We construct a series of games  $G_0$  to  $G_7$ , with  $G_0$  being the real world of  $G^{\text{sim*}-\text{ac-cprf}}$  and  $G_7$  being the simulated world of  $G^{\text{sim*}-\text{ac-cprf}}$ . We also define the corresponding simulator  $S$  in Fig. 35. Of these, we establish a series of claims:

1.  $\Pr[G_0(\lambda)] = \Pr[G_{\text{CMP}[F,H],S,P,A}^{\text{sim*}-\text{ac-cprf}} | b = 1]$
2.  $\Pr[G_1(\lambda)] = \Pr[G_0(\lambda)]$
3.  $\Pr[G_2(\lambda)] = \Pr[G_1(\lambda)]$
4.  $\Pr[G_3(\lambda)] - \Pr[G_2(\lambda)] = \text{Adv}_{H,S_H,P,B}^{\text{sim*}-\text{ac-cprf}}(\lambda)$
5.  $\Pr[G_4(\lambda)] = \Pr[G_3(\lambda)]$
6.  $\Pr[G_5(\lambda)] = \Pr[G_4(\lambda)]$
7.  $\Pr[G_6(\lambda)] - \Pr[G_5(\lambda)] = \text{Adv}_{F,S_F,P,C}^{\text{sim*}-\text{ac-cprf}}(\lambda)$
8.  $\Pr[G_7(\lambda)] = \Pr[G_6(\lambda)]$
9.  $\Pr[G_7(\lambda)] = \Pr[\neg G_{\text{CMP}[F,H],S,P,A}^{\text{sim*}-\text{ac-cprf}} | b = 0]$

Therefore,

$$\begin{aligned} \text{Adv}_{\text{CMP}[F,H],S,P,A}^{\text{sim*}-\text{ac-cprf}}(\lambda) &= \Pr[G_0(\lambda)] - \Pr[G_7(\lambda)] = \sum_{i=0}^6 (\Pr[G_i(\lambda)] - \Pr[G_{i+1}(\lambda)]) \\ &= \text{Adv}_{H,S_H,P,B}^{\text{sim*}-\text{ac-cprf}}(\lambda) + \text{Adv}_{F,S_F,P,C}^{\text{sim*}-\text{ac-cprf}}(\lambda). \end{aligned}$$

The result follows from the assumed security of  $H$  and  $F$ .

Throughout this section, we write  $T.\text{store}(S \mapsto y)$  to mean an operation equivalent to “For  $x \in S$ :  $T[x] \leftarrow y$ ”. In case  $S$  is large,  $T$  will need to somehow represent this implicitly for efficiency.

**Claims 1 and 2.** Hybrid  $G_0$  is defined on the top left of Fig. 33 is the real world ( $b = 1$ ) of SIM\*-AC-CPRF game defined in Fig. 2 with code of CMP plugged in. Hybrid  $G_1$  defined on the top middle of Fig. 33 contains highlighted code. In this hybrid, we are merely adding tracking tables **CKey** and **Own** which are marked in blue and pink respectively. When a  $(x, x')$  evaluation query is made,  $F$  is assigned ownership of  $x$  (associated with the appropriate user  $u$ ). Similarly when a key is constrained on  $R_{x,S}$ , if  $x$  is not already owned by  $F$ . When a  $R_S$  query is made,  $H$  takes ownership over all  $x \in S \cap H.Inp_2$ . This may result in  $H$  taking over ownership of  $x$ 's previously owned by  $F$  (in later games  $S_F.Con$  will be run for user  $(u, x)$  on set  $R = F.Inp$  here to derive a key for  $x$  that is then passed to  $S_H.Con$ ). Note that  $F$  will never take ownership of a value previously owned by  $H$ .

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Game <math>G_0(\lambda)</math></b></p> <p><math>\sigma_P \leftarrow \\$P.\text{Init}(1^\lambda)</math><br/> <math>k(\cdot) \leftarrow \\$H.K(\lambda)</math><br/> <math>b' \leftarrow \\$\mathcal{A}_{\text{cprf}^*}^{\text{CON}, \text{EV}, \text{PPRIM}}(1^\lambda)</math><br/> Return <math>(b' = 1)</math></p> <p><math>\text{Ev}(u, \mathbf{x})</math><br/> <b>If</b> <math>\mathbf{x} \in H.\text{Inp}_1</math><br/> <math>y \leftarrow H.\text{Ev}^P(1^\lambda, k_u, \mathbf{x})</math><br/> <b>Else if</b> <math>\mathbf{x} = (x, x')</math><br/> <math>K_x \leftarrow H.\text{Ev}^P(1^\lambda, k_u, x)</math><br/> <math>y \leftarrow F.\text{Ev}^P(1^\lambda, K_x, x')</math><br/> <math>E_u \leftarrow E_u \cup \{\mathbf{x}\}</math><br/> Return <math>y</math></p> <p><math>\text{CON}(u, R)</math><br/> <b>If</b> <math>R = R_S</math>:<br/> <math>K_R \leftarrow H.\text{Con}^P(1^\lambda, k_u, S)</math><br/> <b>Else if</b> <math>R = R_{x,S}</math>:<br/> <math>K_x \leftarrow H.\text{Ev}^P(1^\lambda, k_u, x)</math><br/> <math>K_R \leftarrow F.\text{Con}^P(1^\lambda, K_x, S)</math><br/> <math>C_u \leftarrow C_u \cup R</math><br/> Return <math>K_R</math></p>                                                                                                                                                                                                                                                                                                                                  | <p><b>Games <math>G_1(\lambda), G_2(\lambda)</math></b></p> <p><math>\sigma_P \leftarrow \\$P.\text{Init}(1^\lambda)</math><br/> <math>k(\cdot) \leftarrow \\$H.K(\lambda)</math><br/> <math>b' \leftarrow \\$\mathcal{A}_{\text{cprf}^*}^{\text{CON}, \text{EV}, \text{PPRIM}}(1^\lambda)</math><br/> Return <math>(b' = 1)</math></p> <p><math>\text{Ev}(u, \mathbf{x})</math><br/> <b>If</b> <math>\mathbf{x} \in C_u</math>: <math>(K_R, R) \leftarrow \text{CKey}_u[\mathbf{x}]</math><br/> <math>y \leftarrow \text{CMP}[F, H].\text{CEv}^P(1^\lambda, K_R, R, \mathbf{x})</math><br/> <b>Else:</b><br/> <b>If</b> <math>\mathbf{x} \in H.\text{Inp}_1</math><br/> <math>y \leftarrow H.\text{Ev}^P(1^\lambda, k_u, \mathbf{x})</math><br/> <b>Else if</b> <math>\mathbf{x} = (x, x')</math><br/> <math>K_x \leftarrow H.\text{Ev}^P(1^\lambda, k_u, x)</math><br/> <math>y \leftarrow F.\text{Ev}^P(1^\lambda, K_x, x')</math><br/> <math>\text{Own}_u.\text{store}(\{x\} \mapsto F)</math><br/> <math>E_u \leftarrow E_u \cup \{\mathbf{x}\}</math><br/> Return <math>y</math></p> <p><math>\text{CON}(u, R)</math><br/> <b>If</b> <math>R = R_S</math>:<br/> <math>K_R \leftarrow H.\text{Con}^P(1^\lambda, k_u, S)</math><br/> <math>\text{CKey}_u.\text{store}(S \cap H.\text{Inp}_2 \mapsto (K_R, S))</math><br/> <math>\text{Own}_u.\text{store}(S \cap H.\text{Inp}_2 \mapsto H)</math><br/> <b>Else if</b> <math>R = R_{x,S}</math>:<br/> <math>X \leftarrow \text{Own}_u[x]</math><br/> <b>If</b> <math>X \in \{\perp, F\}</math>:<br/> <math>K_x \leftarrow H.\text{Ev}^P(1^\lambda, k_u, x)</math><br/> <math>K_R \leftarrow F.\text{Con}^P(1^\lambda, K_x, S)</math><br/> <math>\text{Own}_u.\text{store}(\{x\} \mapsto F)</math><br/> <b>Else</b><br/> <math>K_x \leftarrow H.\text{Ev}^P(1^\lambda, k_u, x)</math><br/> <math>K_R \leftarrow F.\text{Con}^P(1^\lambda, K_x, S)</math><br/> <math>(K_{S'}, S') \leftarrow \text{CKey}_u[x]</math><br/> <math>(K_x \leftarrow H.\text{CEv}^P(1^\lambda, K_{S'}, S', x))</math><br/> <math>K_R \leftarrow F.\text{Con}^P(1^\lambda, K_x, S)</math><br/> <math>\text{CKey}_u.\text{store}(R \mapsto (K_R, R))</math><br/> <math>C_u \leftarrow C_u \cup R</math><br/> Return <math>K_R</math></p> | <p><b>Adversary <math>\mathcal{B}^{\text{CON}_H, \text{EV}_H, \text{PPRIM}}</math></b></p> <p><math>\sigma_P \leftarrow \\$P.\text{Init}(1^\lambda)</math><br/> <math>k(\cdot) \leftarrow \\$H.K(\lambda)</math><br/> <math>b' \leftarrow \\$\mathcal{A}_{\text{cprf}^*}^{\text{CON}, \text{EV}, \text{PPRIM}}(1^\lambda)</math><br/> Return <math>b'</math></p> <p><math>\text{Ev}(u, \mathbf{x})</math><br/> <b>If</b> <math>\mathbf{x} \in C_u</math>: <math>(K_R, R) \leftarrow \text{CKey}_u[\mathbf{x}]</math><br/> <math>y \leftarrow \text{CMP}[F, H].\text{CEv}^P(1^\lambda, K_R, R, \mathbf{x})</math><br/> <b>Else if</b> <math>\mathbf{x} \in H.\text{Inp}_1</math><br/> <math>y \leftarrow \text{EV}_H(u, \mathbf{x})</math><br/> <b>Else if</b> <math>\mathbf{x} = (x, x')</math><br/> <math>K_x \leftarrow \text{EV}_H(u, x)</math><br/> <math>y \leftarrow F.\text{Ev}^P(1^\lambda, K_x, x')</math><br/> <math>\text{Own}_u.\text{store}(\{x\} \mapsto F)</math><br/> <math>E_u \leftarrow E_u \cup \{\mathbf{x}\}</math><br/> Return <math>y</math></p> <p><math>\text{CON}(u, R)</math><br/> <b>If</b> <math>R = R_S</math>:<br/> <math>K_R \leftarrow \text{CON}_H(u, S)</math><br/> <math>\text{CKey}_u.\text{store}(S \cap H.\text{Inp}_2 \mapsto (K_R, S))</math><br/> <math>\text{Own}_u.\text{store}(S \cap H.\text{Inp}_2 \mapsto H)</math><br/> <b>Else if</b> <math>R = R_{x,S}</math>:<br/> <math>X \leftarrow \text{Own}_u[x]</math><br/> <b>If</b> <math>X \in \{\perp, F\}</math>:<br/> <math>K_x \leftarrow \text{EV}_H(u, x)</math><br/> <math>K_R \leftarrow F.\text{Con}^P(1^\lambda, K_x, S)</math><br/> <math>\text{Own}_u.\text{store}(\{x\} \mapsto F)</math><br/> <b>Else</b><br/> <math>(K_{S'}, S') \leftarrow \text{CKey}_u[x]</math><br/> <math>K_x \leftarrow H.\text{CEv}^P(1^\lambda, K_{S'}, S', x)</math><br/> <math>K_R \leftarrow F.\text{Con}^P(1^\lambda, K_x, S)</math><br/> <math>\text{CKey}_u.\text{store}(R \mapsto (K_R, R))</math><br/> <math>C_u \leftarrow C_u \cup R</math><br/> Return <math>K_R</math></p> |
| <p><b>Games <math>G_3(\lambda), G_4(\lambda)</math></b></p> <p><math>\sigma_P \leftarrow \\$P.\text{Init}(1^\lambda)</math><br/> <math>\sigma_H \leftarrow \\$S_H.\text{Init}(1^\lambda)</math><br/> <math>f(\cdot) \leftarrow \\$H.\text{Fcs}(\lambda)</math><br/> <math>b' \leftarrow \\$\mathcal{A}_{\text{cprf}^*}^{\text{CON}, \text{EV}, \text{PPRIM}}(1^\lambda)</math><br/> Return <math>b' = 1</math></p> <p><math>\text{Ev}(u, \mathbf{x})</math><br/> <b>If</b> <math>\mathbf{x} \in C_u</math>: <math>(K_R, R) \leftarrow \text{CKey}_u[\mathbf{x}]</math><br/> <math>y \leftarrow \text{CMP}[F, H].\text{CEv}^P(1^\lambda, K_R, R, \mathbf{x})</math><br/> <b>Else if</b> <math>\mathbf{x} \in H.\text{Inp}_1</math><br/> <b>If</b> <math>\mathbf{x} \in C_u^H</math>: <math>y \leftarrow S_H.\text{Ev}^{\text{PPRIM}}(1^\lambda, u, \mathbf{x} : \sigma_H)</math><br/> <b>Else:</b> <math>y \leftarrow f_u^H(\mathbf{x})</math><br/> <math>E_u^H \leftarrow E_u^H \cup \{\mathbf{x}\}</math><br/> <b>Else if</b> <math>\mathbf{x} = (x, x')</math><br/> <b>If</b> <math>x \in C_u^H</math>: <math>K_x \leftarrow S_H.\text{Ev}^{\text{PPRIM}}(1^\lambda, u, x : \sigma_H)</math><br/> <b>Else:</b> <math>K_x \leftarrow f_u^H(x)</math><br/> <math>E_u^H \leftarrow E_u^H \cup \{x\}</math><br/> <math>y \leftarrow F.\text{Ev}^P(1^\lambda, K_x, x')</math><br/> <math>\text{Own}_u.\text{store}(\{x\} \mapsto F)</math><br/> <math>E_u \leftarrow E_u \cup \{\mathbf{x}\}</math><br/> Return <math>y</math></p> | <p><math>\text{CON}(u, R)</math><br/> <b>If</b> <math>R = R_S</math>:<br/> <math>K_R \leftarrow S_H.\text{Con}^{\text{PPRIM}}(1^\lambda, u, S, f_u^H(S \cap E_u^H \setminus C_u^H) : \sigma_H)</math><br/> <math>C_u^H \leftarrow C_u^H \cup S</math><br/> <math>\text{CKey}_u.\text{store}(S \cap H.\text{Inp}_2 \mapsto (K_R, S))</math><br/> <math>\text{Own}_u.\text{store}(S \cap H.\text{Inp}_2 \mapsto H)</math><br/> <b>Else if</b> <math>R = R_{x,S}</math>:<br/> <math>X \leftarrow \text{Own}_u[x]</math><br/> <b>If</b> <math>X \in \{\perp, F\}</math>:<br/> <b>If</b> <math>x \in C_u^H</math>: <math>K_x \leftarrow S_H.\text{Ev}^{\text{PPRIM}}(1^\lambda, u, x : \sigma_H)</math><br/> <b>Else:</b> <math>K_x \leftarrow f_u^H(x)</math><br/> <math>E_u^H \leftarrow E_u^H \cup \{x\}</math><br/> <math>K_R \leftarrow F.\text{Con}^P(1^\lambda, K_x, S)</math><br/> <math>\text{Own}_u.\text{store}(\{x\} \mapsto F)</math><br/> <b>Else</b><br/> <math>(K_{S'}, S') \leftarrow \text{CKey}_u[x]</math><br/> <math>K_x \leftarrow H.\text{CEv}^P(1^\lambda, K_{S'}, S', x)</math><br/> <math>K_R \leftarrow F.\text{Con}^P(1^\lambda, K_x, S)</math><br/> <math>\text{CKey}_u.\text{store}(R \mapsto (K_R, R))</math><br/> <math>C_u \leftarrow C_u \cup R</math><br/> Return <math>K_R</math></p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

**Fig. 33.** Games and adversary for CMP security proof.  $G_0$  plugs the code of CMP into the real world.  $G_1$  adds code to track constrained keys and key owners.  $G_2$  reuses constrained keys when possible.  $\mathcal{B}$  reduces to security of  $H$  so that  $G_3$  can replace it with simulation.  $G_4$  removes an unused branches.

| Games $\mathbf{G}_5(\lambda), \mathbf{G}_6(\lambda)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Adversary $\mathcal{C}^{\text{CON}, \text{EV}, \text{PPRIM}}$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\sigma_P \leftarrow \text{P.Init}(1^\lambda)$<br>$\sigma_H \leftarrow \text{S}_H.\text{Init}(1^\lambda)$<br>$f_{(\cdot)}^H \leftarrow \text{H.Fcs}(\lambda)$<br>$k_{(\cdot)} \leftarrow \text{F.K}(\lambda)$<br>$\sigma_F \leftarrow \text{S}_F.\text{Init}(1^\lambda)$<br>$f_{(\cdot)}^F \leftarrow \text{F.Fcs}(\lambda)$<br>$b' \leftarrow \mathcal{A}_{\text{cpf}^*}^{\text{CON}, \text{EV}, \text{PPRIM}}(1^\lambda)$<br>Return $b' = 1$<br><br>$\text{Ev}(u, \mathbf{x})$<br>If $\mathbf{x} \in C_u$ : $(K_R, R) \leftarrow \text{CKey}_u[\mathbf{x}]$<br>$y \leftarrow \text{CMP}[\text{F}, \text{H}].\text{CEv}^P(1^\lambda, K_R, R, \mathbf{x})$<br>Else if $\mathbf{x} \in \text{H.Inp}_1$<br>$y \leftarrow f_u^H(\mathbf{x})$<br>$E_u^H \leftarrow E_u^H \cup \{\mathbf{x}\}$<br>Else if $\mathbf{x} = (x, x')$<br>$K_x \leftarrow k_{u,x}$<br>$E_u^H \leftarrow E_u^H \cup \{\mathbf{x}\}$<br>$y \leftarrow \text{F.Ev}^P(1^\lambda, K_x, x')$<br>If $x' \in C_{u,x}^F$ : $y \leftarrow \text{S}_F.\text{Ev}^{\text{PPRIM}}(1^\lambda, (u, x), x' : \sigma_F)$<br>Else: $y \leftarrow f_{u,x}^F(x')$<br>$E_{u,x}^F \leftarrow E_{u,x}^F \cup \{x'\}$<br>$\text{Own}_u.\text{store}(\{x\} \mapsto \text{F})$<br>$E_u \leftarrow E_u \cup \{\mathbf{x}\}$<br>Return $y$<br><br>$\text{CON}(u, R)$<br>If $R = R_S$ :<br>For $x \in S$ s.t. $\text{Own}_u[x] = \text{F}$ :<br>$f_u^H(x) \leftarrow k_{u,x}$<br>$X \leftarrow \text{F.Inp} \cap E_{u,x}^F \setminus C_{u,x}^F$<br>$f_u^H(x) \leftarrow \text{S}_F.\text{Con}^{\text{PPRIM}}(1^\lambda, (u, x), \text{F.Inp}, f_{u,x}^F(X) : \sigma_F)$<br>$C_{u,x}^F \leftarrow C_{u,x}^F \cup \text{F.Inp}$<br>$K_R \leftarrow \text{S}_H.\text{Con}^{\text{PPRIM}}(1^\lambda, u, S, f_u^H(S \cap E_u^H \setminus C_u^H) : \sigma_H)$<br>$C_u^H \leftarrow C_u^H \cup S$<br>$\text{CKey}_u.\text{store}(S \cap \text{H.Inp}_2 \mapsto (K_R, S))$<br>$\text{Own}_u.\text{store}(S \cap \text{H.Inp}_2 \mapsto \text{H})$<br>Else if $R = R_{x,S}$ :<br>$X \leftarrow \text{Own}_u[x]$<br>If $X \in \{\perp, \text{F}\}$ :<br>$K_x \leftarrow k_{u,x}$<br>$E_u^H \leftarrow E_u^H \cup \{x\}$<br>$K_R \leftarrow \text{F.Con}^P(1^\lambda, K_x, S)$<br>$X \leftarrow S \cap E_{u,x}^F \setminus C_{u,x}^F$<br>$K_R \leftarrow \text{S}_F.\text{Con}^{\text{PPRIM}}(1^\lambda, (u, x), S, f_{u,x}^F(X) : \sigma_F)$<br>$C_{u,x}^F \leftarrow C_{u,x}^F \cup S$<br>$\text{Own}_u.\text{store}(\{x\} \mapsto \text{F})$<br>Else<br>$(K_{S'}, S') \leftarrow \text{CKey}_u[x]$<br>$K_x \leftarrow \text{H.CEv}^P(1^\lambda, K_{S'}, S', x)$<br>$K_R \leftarrow \text{F.Con}^P(1^\lambda, K_x, S)$<br>$\text{CKey}_u.\text{store}(R \mapsto (K_R, R))$<br>$C_u \leftarrow C_u \cup R$<br>Return $K_R$ | $\sigma_P \leftarrow \text{P.Init}(1^\lambda)$<br>$\sigma_H \leftarrow \text{S}_H.\text{Init}(1^\lambda)$<br>$f_{(\cdot)}^H \leftarrow \text{H.Fcs}(\lambda)$<br>$k_{(\cdot)} \leftarrow \text{F.K}(\lambda)$<br>$b' \leftarrow \mathcal{A}_{\text{cpf}^*}^{\text{CON}, \text{EV}, \text{PPRIM}}(1^\lambda)$<br>Return $b'$<br><br>$\text{Ev}(u, \mathbf{x})$<br>If $\mathbf{x} \in C_u$ : $(K_R, R) \leftarrow \text{CKey}_u[\mathbf{x}]$<br>$y \leftarrow \text{CMP}[\text{F}, \text{H}].\text{CEv}^P(1^\lambda, K_R, R, \mathbf{x})$<br>Else if $\mathbf{x} \in \text{H.Inp}_1$<br>$y \leftarrow f_u^H(\mathbf{x})$<br>$E_u^H \leftarrow E_u^H \cup \{\mathbf{x}\}$<br>Else if $\mathbf{x} = (x, x')$<br>$E_u^H \leftarrow E_u^H \cup \{\mathbf{x}\}$<br>$y \leftarrow \text{EV}_F((u, x), x')$<br>$\text{Own}_u.\text{store}(\{x\} \mapsto \text{F})$<br>$E_u \leftarrow E_u \cup \{\mathbf{x}\}$<br>Return $y$<br><br>$\text{CON}(u, R)$<br>If $R = R_S$ :<br>For $x \in S$ s.t. $\text{Own}_u[x] = \text{F}$ :<br>$f_u^H(x) \leftarrow \text{CON}_F((u, x), \text{F.Inp})$<br>$K_R \leftarrow \text{S}_H.\text{Con}^{\text{PPRIM}}(1^\lambda, u, S, f_u^H(S \cap E_u^H \setminus C_u^H) : \sigma_H)$<br>$C_u^H \leftarrow C_u^H \cup S$<br>$\text{CKey}_u.\text{store}(S \cap \text{H.Inp}_2 \mapsto (K_R, S))$<br>$\text{Own}_u.\text{store}(S \cap \text{H.Inp}_2 \mapsto \text{H})$<br>Else if $R = R_{x,S}$ :<br>$X \leftarrow \text{Own}_u[x]$<br>If $X \in \{\perp, \text{F}\}$ :<br>$E_u^H \leftarrow E_u^H \cup \{x\}$<br>$K_R \leftarrow \text{CON}_F((u, x), S)$<br>$\text{Own}_u.\text{store}(\{x\} \mapsto \text{F})$<br>Else<br>$(K_{S'}, S') \leftarrow \text{CKey}_u[x]$<br>$K_x \leftarrow \text{H.CEv}^P(1^\lambda, K_{S'}, S', x)$<br>$K_R \leftarrow \text{F.Con}^P(1^\lambda, K_x, S)$<br>$\text{CKey}_u.\text{store}(R \mapsto (K_R, R))$<br>$C_u \leftarrow C_u \cup R$<br>Return $K_R$ |

**Fig. 34.** Games and adversary for CMP security proof.  $\mathbf{G}_5$  uses keys in place of  $f^H$  for  $x$ 's "owned" by  $\text{F}$ , but re-inserts then into  $f^H$  when ownership changes.  $\mathcal{C}$  reduces to security of  $\text{F}$  so that  $\mathbf{G}_6$  can replace it with simulation. Struck code in  $\mathbf{G}_6$  are unused branches.

**Claim 3.** Hybrid  $G_2$  is defined on the top middle of Fig. 34. It contains boxed code but not highlighted code. In this hybrid, we make use of constrained keys stored in  $\text{CKey}$  to respond to some evaluation queries. Notice that we always update  $\text{CKey}$  with the constrained keys for  $\text{CMP}$  at the end of  $\text{CON}$ . Consequently,  $\text{CKey}$  will be non- $\perp$  for any inputs in  $C_u$ . So these keys are used in  $\text{EV}$  whenever the input is in  $C$  to do constrained evaluation instead of evaluation.

We additionally update  $\text{CKey}$  inside  $\text{CON}$  in the “If  $R = R_S$ ” branch to store constrained keys for  $H$ . Note that  $H$  is also given ownership of all inputs that these keys are being stored for. These constrained keys allow for evaluating inputs to  $\text{CMP}$  in the form of  $(S \cap H.\text{Inp}_2H) \times F.\text{Inp}$ . Then later when a  $\text{CON}$  query is made in the form of  $\{x\} \times F.\text{Inp}$  where  $x \in H.\text{Inp}_2$  is owned by  $H$ , we replace  $H$  evaluation with constrained evaluation using the stored key.

The third claim follows from the strong consistency of  $H$  and  $F$ .

**Claim 4.** Hybrid  $G_3$  is defined on the bottom of Fig. 33. It contains highlighted code. In this hybrid, we want to **replace the output of  $H$  with random or simulated values**. We additionally keep sets  $E_u^H$  and  $C_u^H$  set to match the corresponding sets in the coming reduction adversary’s game against  $H$ .  $\text{SIM}^*\text{-AC-CPRF}$  adversary  $\mathcal{B}$  for  $H$  is defined on the top right of Fig. 33 to bound the difference between  $\Pr[G_2(\lambda)]$  and  $\Pr[G_3(\lambda)]$ . The highlighted code is where  $\mathcal{B}$  uses its own oracles to perform evaluations of  $H$ .

Let  $b_H$  be the hidden bit in  $\mathcal{B}$ ’s game. When  $b_H$  is 1,  $\mathcal{B}$  is emulating  $G_2$  for  $\mathcal{A}$ . When  $b_H$  is 0,  $\mathcal{B}$  is emulating  $G_3$  for  $\mathcal{A}$ . The correctness of this emulation gives the fourth claim.

**Claims 5 and 6.** Hybrid  $G_4$  is defined on the bottom of Fig. 33 and does not contain highlighted code. It removes unnecessary branches in  $G_3$ . Notice whenever we update  $C_u^H$  with set  $S$  in  $\text{CON}$ , we always update  $\text{Own}_u[x]$  to  $H$  for any  $x \in S \cap H.\text{Inp}_2$  and update  $C_u$  with  $R_S$ . Therefore, inside  $\text{EV}$ , in the “ $x \notin C_u$ ” branch, we know  $x \notin C_u^H$  will hold. Inside  $\text{CON}$ , when  $\text{Own}_u[x] \neq H$ , we know  $x \notin C_u^H$  will hold. So the branches are indeed unused and can be removed.

Hybrid  $G_5$  is defined on the left of Fig. 34 and contains highlighted code. In  $G_4$  there are several places where keys for  $F$  which are owned by  $F$  were chosen by the random function  $f^H$ . Now in  $G_5$ , we instead lazily sample such keys using  $k$ . Inside of  $\text{CON}$  when  $R = R_S$ , the simulator  $S_H$  expects to be given the partial function of  $f^H$  restricted to  $S \cap E_u^H \setminus C_u^H$ . We create this partial function by looping over all  $x$ ’s in  $S$  that are owned by  $F$  and inserting their keys into  $f^H$ . Since both  $k$  and  $f^H$  are both sampled uniformly at random and otherwise unused,  $G_5$  is identical to  $G_4$ . This gives the sixth claim.

**Claim 7.** Hybrid  $G_6$  is defined on the left of Fig. 34 and contains boxed but not highlighted code. We are switching outputs of  $F$  to be random or simulated. Similarly, we keep track of  $C^F$  and  $E^F$  to match the sets in the reduction adversary’s game. The struck code of  $G_6$  is unreachable because  $x' \in C_{u,x}^F$  implies  $x \in C_u$ .  $\text{SIM}^*\text{-AC-CPRF}$  adversary  $\mathcal{C}$  for  $F$  is defined on the right of Fig. 34 to bound the difference between  $\Pr[G_5(\lambda)]$  and  $\Pr[G_6(\lambda)]$ . The highlighted code is where  $\mathcal{B}$  replaces evaluations of  $F$  using its own oracles to emulate the game for  $\mathcal{A}$ .

Let  $b_F$  be the hidden bit in  $\mathcal{C}$ ’s game, when  $b_F$  is 1,  $\mathcal{C}$  is emulating  $G_5$  for  $\mathcal{A}$ . When  $b_F$  is 0,  $\mathcal{C}$  is emulating  $G_6$  for  $\mathcal{A}$ . Hence claim 7 follows.

**Claims 8 and 9.** Hybrid  $G_7$  is defined on the left of Fig. 35. Now, instead of sampling  $f^H$  and  $f^F$  in the beginning, we only sample a single  $f$  for  $\text{CMP}$ . Later, we temporarily reconstruct the appropriate  $f^H$  and  $f^F$  using  $f$  inside  $\text{CON}$ . Importantly,  $F$  owns a relevant  $x$ , we still program  $f^H$  using values from  $S_F$ . This change does not modify the behavior of the game and so the eighth claim holds.

Finally, we define the simulator on the right of Fig. 35. It was designed so that  $G_7$  matches the simulated world of the  $\text{SIM}^*\text{-AC-CPRF}$  game for  $\text{CMP}[F, H]$ . We emphasize that depends on the simulators for  $F$  and  $H$ , but not on the given adversary. This justifies claim 9 and hence completes the proof.  $\square$

| Game $G_7(\lambda)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | Simulator $S$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\sigma_P \leftarrow \$P.\text{Init}(1^\lambda)$<br>$\sigma_H \leftarrow \$S_H.\text{Init}(1^\lambda)$<br>$\sigma_F \leftarrow \$S_F.\text{Init}(1^\lambda)$<br>$f_{(\cdot)} \leftarrow \text{CMP}[F, H].\text{CEv}(\lambda)$<br>$b' \leftarrow \$\mathcal{A}_{\text{cprf}^*}^{\text{CON}, \text{Ev}, \text{PPRIM}}(1^\lambda)$<br>Return $(b' = 1)$<br><br>$\text{Ev}(u, \mathbf{x})$<br>If $\mathbf{x} \in C_u$ : $(K_R, R) \leftarrow \text{CKey}_u[\mathbf{x}]$<br>$y \leftarrow \text{CMP}[F, H].\text{CEv}^P(1^\lambda, K_R, R, \mathbf{x})$<br>Else: $y \leftarrow f_u(\mathbf{x})$<br>$E_u \leftarrow E_u \cup \{\mathbf{x}\}$<br>Return $y$<br><br>$\text{CON}(u, R)$<br>$f' \leftarrow f_u(R \cap E_u \setminus C_u)$<br>$f^H, f^F \leftarrow [\cdot]$<br>For $\mathbf{x} \in f'.D \cap H.\text{Inp}_1$ :<br>$f_u^H(\mathbf{x}) \leftarrow f'(\mathbf{x})$<br>For $(x, x') \in f'.D \cap (H.\text{Inp}_2 \times F.\text{Inp})$ :<br>$f_{u,x}^F(x') \leftarrow f'(x, x')$<br>If $R = R_S$ :<br>For $x \in S$ s.t. $\text{Own}_u[x] = F$ or $f_{u,x}^F.D \neq \emptyset$ :<br>$f_u^H(x) \leftarrow S_F.\text{Con}^{\text{PPRIM}}(1^\lambda, (u, x), F.\text{Inp}, f_{u,x}^F : \sigma_F)$<br>$K_R \leftarrow S_H.\text{Con}^{\text{PPRIM}}(1^\lambda, u, S, f_u^H : \sigma_H)$<br>$\text{CKey}_u.\text{store}(S \cap H.\text{Inp}_2 \mapsto (K_R, S))$<br>$\text{Own}_u.\text{store}(S \cap H.\text{Inp}_2 \mapsto H)$<br>Else if $R = R_{x,S}$ :<br>$X \leftarrow \text{Own}_u[x]$<br>If $X \in \{\perp, F\}$ :<br>$K_R \leftarrow S_F.\text{Con}^{\text{PPRIM}}(1^\lambda, (u, x), S, f_{u,x}^F : \sigma_F)$<br>$\text{Own}_u.\text{store}(\{x\} \mapsto F)$<br>Else:<br>$(K_{S'}, S') \leftarrow \text{CKey}_u[x]$<br>$K_x \leftarrow H.\text{CEv}^P(1^\lambda, K_{S'}, S', x)$<br>$K_R \leftarrow F.\text{Con}^P(1^\lambda, K_x, S)$<br>$\text{CKey}_u.\text{store}(R \mapsto (K_R, R))$<br>$C_u \leftarrow C_u \cup R$<br>Return $K_R$ | $S.\text{Init}(1^\lambda)$<br>$\sigma_H \leftarrow \$S_H.\text{Init}(1^\lambda)$<br>$\sigma_F \leftarrow \$S_F.\text{Init}(1^\lambda)$<br>$\text{CKey} \leftarrow [\cdot]$<br>$\text{Own} \leftarrow [\cdot]$<br>Return $\sigma = (\sigma_H, \sigma_F, \text{CKey}, \text{Own})$<br><br>$S.\text{Ev}^{\text{PPRIM}}(1^\lambda, u, \mathbf{x} : \sigma = (\sigma_H, \sigma_F, \text{CKey}, \text{Own}))$<br>$(K_R, R) \leftarrow \text{CKey}_u[\mathbf{x}]$<br>$y \leftarrow \text{CMP}[F, H].\text{CEv}^P(1^\lambda, K_R, R, \mathbf{x})$<br>Return $y$<br><br>$S.\text{Con}^{\text{PPRIM}}(1^\lambda, u, R, f' : \sigma = (\sigma_H, \sigma_F, \text{CKey}, \text{Own}))$<br>$f^H, f^F \leftarrow [\cdot]$<br>For $\mathbf{x} \in f'.D \cap H.\text{Inp}_1$ :<br>$f_u^H(\mathbf{x}) \leftarrow f'(\mathbf{x})$<br>For $(x, x') \in f'.D \cap (H.\text{Inp}_2 \times F.\text{Inp})$ :<br>$f_{u,x}^F(x') \leftarrow f'(x, x')$<br>If $R = R_S$ :<br>For $x \in S$ s.t. $\text{Own}_u[x] = F$ or $f_{u,x}^F.D \neq \emptyset$ :<br>$f_u^H(x) \leftarrow S_F.\text{Con}^{\text{PPRIM}}(1^\lambda, (u, x), F.\text{Inp}, f_{u,x}^F : \sigma_F)$<br>$K_R \leftarrow S_H.\text{Con}^{\text{PPRIM}}(1^\lambda, u, S, f_u^H : \sigma_H)$<br>$\text{CKey}_u.\text{store}(S \cap H.\text{Inp}_2 \mapsto (K_R, S))$<br>$\text{Own}_u.\text{store}(S \cap H.\text{Inp}_2 \mapsto H)$<br>Else if $R = R_{x,S}$ :<br>$X \leftarrow \text{Own}_u[x]$<br>If $X \in \{\perp, F\}$ :<br>$K_R \leftarrow S_F.\text{Con}^{\text{PPRIM}}(1^\lambda, (u, x), S, f_{u,x}^F : \sigma_F)$<br>$\text{Own}_u.\text{store}(\{x\} \mapsto F)$<br>Else:<br>$(K_{S'}, S') \leftarrow \text{CKey}_u[x]$<br>$K_x \leftarrow H.\text{CEv}^P(1^\lambda, K_{S'}, S', x)$<br>$K_R \leftarrow F.\text{Con}^P(1^\lambda, K_x, S)$<br>$\text{CKey}_u.\text{store}(R \mapsto (K_R, R))$<br>Return $K_R$ |

**Fig. 35.** Games and adversary for CMP security proof.  $G_7$  uses a combined  $f$  in place of  $f^H$  and  $f^F$ . When needed, it defines components of  $f^H$  and  $f^F$  from  $f$ . It stops tracking when  $\text{Own}$  is used in  $\text{Ev}$  and sets  $E^H$ ,  $C^H$ ,  $E^F$ , and  $C^F$ . It infers the needed information from  $E$  and  $C$ . Simulator  $S$  perfectly simulates  $G_7$  by reusing constrained keys and simulating  $H$  and  $F$ .

## F Proof for Theorem 5 (Security of Cascade-based Constructions)

*Proof.* The proof divides into two parts. First we argue syntactically why both CSC and ACSC can be viewed as being obtained from repeated applications of CMP using the underlying PRF. Then we give a generic argument using the proof of Theorem 4 to argue why schemes obtained by repeated application of CMP are secure.

To start, assume that the CPRF attacker under consideration will make queries of length at most  $n(\lambda)$ . That is, for any  $\mathbf{x}$  queried to  $\text{Ev}$  and any  $R_{\mathbf{x}} \sqsubseteq$  queried to  $\text{CON}$ ,  $\mathbf{x} \in F.\text{Inp}^{\leq n(\lambda)}$ . For now, treat the security parameter  $\lambda$  as fixed.

**Cascade Construction.** We start with  $\text{CSC}[F, D]$ . It suffices to restrict the domain to inputs of length at most  $n(\lambda)$  and to add inputs to ensure that every string is a prefix of or is prefixed by a string in the domain. Let  $E = D_{\leq n(\lambda)} \cup \{\mathbf{x} \in F.\text{Inp}^{n(\lambda)} \mid \forall \mathbf{y} \in D_{\leq n(\lambda)}, \mathbf{y} \not\sqsubseteq \mathbf{x}\}$  where we let  $D_{\leq n(\lambda)} = D \cap \mathbf{x} \in F.\text{Inp}^{\leq n(\lambda)}$ . (Using  $E$  in place of  $D$  adds minor notational convenience to the following.)

Now we recursively define a sequence of constrained function families  $\text{CSC}_i$  by  $\text{CSC}_1 = \text{F}$  (viewing  $\text{F}$  as a constrained function family) and  $\text{CSC}_{i+1} = \text{CMP}[\text{F}, \text{CSC}_i]$ . For this use of  $\text{CMP}$  we use  $\text{CSC}_i.\text{Inp}_1 = \text{F}.\text{Inp}^{\leq i} \cap \text{E}$  and  $\text{CSC}_i.\text{Inp}_2 = \text{F}.\text{Inp}^i \cap \{\mathbf{x} \mid \exists y \in \text{E}, \mathbf{x} \sqsubset y\}$ .

Our claim is that from the perspective of an attacker making queries of length at most  $n(\lambda)$ ,  $\text{CSC}_{n(\lambda)}$  and  $\text{CSC}[\text{F}, \text{D}]$  are equivalent.<sup>7</sup> Formally this follows from an induction, the details of which are straightforward.

**Augmented Cascade.** Next we consider  $\text{ACSC}[\text{Chop}, \text{F}]$ . Recursively define  $\text{ACSC}_0 = \text{F}_{\text{Chop}}$  and  $\text{ACSC}_{i+1} = \text{CMP}[\text{F}_{\text{Chop}}, \text{ACSC}_i]$ . For this use of  $\text{CMP}$  we set  $\text{ACSC}_i.\text{Inp}_1 = \text{F}.\text{Inp}^{\leq i}$  and  $\text{ACSC}_i.\text{Inp}_2 = \text{F}.\text{Inp}^{i+1}$ . Note that (identifying  $\text{F}.\text{Inp}^{i+1} \times \{\varepsilon\}$  with  $\text{F}.\text{Inp}^{i+1}$ ) this gives us

$$\text{ACSC}_{i+1}.\text{Inp} = \text{F}.\text{Inp}^{\leq i} \sqcup \text{F}.\text{Inp}^{i+1} \times (\{\varepsilon\} \sqcup \text{F}.\text{Inp}) = \text{F}.\text{Inp}^{\leq i+2}.$$

Our claim is that from the perspective of an attacker making queries of length at most  $n(\lambda)$ ,  $\text{ACSC}_{n(\lambda)}$  and  $\text{ACSC}[\text{Chop}, \text{F}]$  are equivalent. This holds because

$$\text{ACSC}_i.\text{Ev}^P(1^\lambda, k, \mathbf{x}) = \begin{cases} \text{ACSC}[\text{Chop}, \text{F}].\text{Ev}^P(1^\lambda, k, \mathbf{x}), & \mathbf{x} \in \text{F}.\text{Inp}^{\leq i} \\ \text{CSC}[\text{F}].\text{Ev}^P(1^\lambda, k, \mathbf{x}), & \mathbf{x} \in \text{F}.\text{Inp}^{i+1}. \end{cases}$$

Formally this follows from an induction, the details of which are straightforward.

**Security Argument.** From the security of Theorem 4 showing the  $\text{SIM}^*\text{-AC-CPRF}$  security of  $\text{CMP}[\text{F}, \text{H}]$  (given in Appendix E.2), we get three efficient maps  $M_1 : (\text{F}, \text{H}, \mathcal{A}_{\text{cprf}^*}) \mapsto \mathcal{B}$ ,  $M_2 : (\text{F}, \text{H}, \mathcal{A}_{\text{cprf}^*}, \text{S}_\text{H}) \mapsto \mathcal{C}$ , and  $M_3 : (\text{F}, \text{H}, \text{S}_\text{F}, \text{S}_\text{H}) \mapsto \text{S}$ . Here  $\mathcal{A}_{\text{cprf}^*}$  is the given  $\text{CMP}$  adversary,  $\text{S}_\text{F}$  is the given  $\text{F}$  simulator, and  $\text{S}_\text{H}$  is the given  $\text{H}$  simulator. The maps correspond to how the proof used these to construct adversary  $\mathcal{B}$  for  $\text{H}$ , adversary  $\mathcal{C}$  for  $\text{F}$ , and simulator  $\text{S}$  for  $\text{CMP}$ . The proof establishes that

$$\text{Adv}_{\text{CMP}[\text{F}, \text{H}], M_3(\text{F}, \text{H}, \text{S}_\text{F}, \text{S}_\text{H}), \text{P}, \mathcal{A}_{\text{cprf}^*}}^{\text{sim}^*\text{-ac-cprf}}(\lambda) = \text{Adv}_{\text{H}, \text{S}_\text{H}, \text{P}, M_1(\text{F}, \text{H}, \mathcal{A}_{\text{cprf}^*})}^{\text{sim}^*\text{-ac-cprf}}(\lambda) + \text{Adv}_{\text{F}, \text{S}_\text{F}, \text{P}, M_2(\text{F}, \text{H}, \mathcal{A}_{\text{cprf}^*}, \text{S}_\text{H})}^{\text{sim}^*\text{-ac-cprf}}(\lambda).$$

holds for all  $\text{F}, \text{H}, \mathcal{A}_{\text{cprf}^*}, \text{S}_\text{F}, \text{S}_\text{H}$ .

Now we apply this for proving  $\text{CSC}$  secure. Let  $\mathcal{A}_{\text{CSC}}$  be the given adversary for  $\text{CSC}$  making queries of length at most  $n(\lambda)$  and  $\text{S}_\text{F}$  be the given simulator for  $\text{F}$ . Recursively define  $\text{S}_1 = \text{S}_\text{F}$  and  $\text{S}_{i+1} = M_3(\text{F}, \text{CSC}_i, \text{S}_\text{F}, \text{S}_i)$  for  $i = 1, \dots, n(\lambda) - 1$ . Recursively define  $\mathcal{B}_{n(\lambda)} = \mathcal{A}_{\text{CSC}}$  and  $\mathcal{B}_{i-1} = M_1(\text{F}, \text{CSC}_{i-1}, \mathcal{B}_i)$  for  $i = n(\lambda), \dots, 2$ . Define  $\mathcal{C}_{i-1} = M_2(\text{F}, \text{CSC}_{i-1}, \mathcal{B}_i, \text{S}_{i-1})$  for  $i = n(\lambda), \dots, 2$ . Then for  $i = 1, \dots, n(\lambda) - 1$  we have,

$$\begin{aligned} \text{Adv}_{\text{CSC}_{i+1}, \text{S}_{i+1}, \text{P}, \mathcal{B}_{i+1}}^{\text{sim}^*\text{-ac-cprf}}(\lambda) &= \text{Adv}_{\text{CSC}_{i+1}, M_3(\text{F}, \text{CSC}_i, \text{S}_\text{F}, \text{S}_i), \text{P}, \mathcal{B}_{i+1}}^{\text{sim}^*\text{-ac-cprf}}(\lambda) \\ &= \text{Adv}_{\text{CMP}[\text{F}, \text{CSC}_i], M_3(\text{F}, \text{CSC}_i, \text{S}_\text{F}, \text{S}_i), \text{P}, \mathcal{B}_{i+1}}^{\text{sim}^*\text{-ac-cprf}}(\lambda) \\ &= \text{Adv}_{\text{CSC}_i, \text{S}_i, \text{P}, M_1(\text{F}, \text{CSC}_i, \mathcal{B}_{i+1})}^{\text{sim}^*\text{-ac-cprf}}(\lambda) + \text{Adv}_{\text{F}, \text{S}_\text{F}, \text{P}, M_2(\text{F}, \text{CSC}_i, \mathcal{B}_{i+1}, \text{S}_i)}^{\text{sim}^*\text{-ac-cprf}}(\lambda). \\ &= \text{Adv}_{\text{CSC}_i, \text{S}_i, \text{P}, \mathcal{B}_i}^{\text{sim}^*\text{-ac-cprf}}(\lambda) + \text{Adv}_{\text{F}, \text{S}_\text{F}, \text{P}, \mathcal{C}_i}^{\text{sim}^*\text{-ac-cprf}}(\lambda). \end{aligned}$$

Recall that the view of  $\mathcal{A}_{\text{CSC}}$  interacting with  $\text{CSC}[\text{F}, \text{D}]$  is identical to its view interacting with  $\text{CSC}_{n(\lambda)}$ . So repeated applications of the above gives

$$\text{Adv}_{\text{CSC}[\text{F}, \text{D}], \text{S}_{n(\lambda)}, \text{P}, \mathcal{A}_{\text{CSC}}}^{\text{sim}^*\text{-ac-cprf}}(\lambda) = \text{Adv}_{\text{F}, \text{S}_\text{F}, \text{P}, \mathcal{B}_1}^{\text{sim}^*\text{-ac-cprf}}(\lambda) + \sum_{i=1}^{n(\lambda)-1} \text{Adv}_{\text{F}, \text{S}_\text{F}, \text{P}, \mathcal{C}_i}^{\text{sim}^*\text{-ac-cprf}}(\lambda).$$

Let  $\mathcal{D}$  be an adversary that randomly selects an adversary from  $\{\mathcal{B}_1, \mathcal{C}_1, \dots, \mathcal{C}_{n(\lambda)-1}\}$  then runs that adversary we get

$$\text{Adv}_{\text{CSC}[\text{F}, \text{D}], \text{S}_{n(\lambda)}, \text{P}, \mathcal{A}_{\text{CSC}}}^{\text{sim}^*\text{-ac-cprf}}(\lambda) = n(\lambda) \cdot \text{Adv}_{\text{F}, \text{S}_\text{F}, \text{P}, \mathcal{D}}^{\text{sim}^*\text{-ac-cprf}}(\lambda).$$

<sup>7</sup> Technically, this does not cover the case when  $\varepsilon \in \text{D}$ , but then  $\text{D} = \{\varepsilon\}$  holds. Security of  $\text{CSC}[\text{F}, \{\varepsilon\}]$  is trivial.



This almost concludes the proof, except that  $S_{n(\lambda)}$  depends on  $n$  and thus  $\mathcal{A}_{\text{CSC}}$ . We need a single  $S$  defined such that for any  $\mathcal{A}_{\text{CSC}}$  making queries of length at most  $n(\lambda)$  will end up behaving identically to  $S_{n(\lambda)}$ .

The basic idea is for  $S$  to run like  $S_l$  where  $l$  is the longest query it's seen so far (whether from  $x$  given to  $S.\text{Ev}$ ,  $R_{x\subseteq}$  given to  $S.\text{Con}$ , or  $x$  queried to  $\text{Ev}$  and then later included as part of the domain of an  $f'$  given to  $S.\text{Con}$ ). It's not hard to verify that such behavior can be done consistently based on the simulator of Fig. 35 by noting that running  $S_{i+1}$  on inputs of length at most  $i$  essentially just makes  $S_i$  respond to all queries.

The details work identically for ACSC by replacing instances of CSC and  $F$  with ACSC and  $F_{\text{Chop}}$  and defining  $S_i$ ,  $\mathcal{B}_i$ , and  $\mathcal{C}_i$  for  $i = 0$  as well. We get

$$\text{Adv}_{\text{ACSC}[\text{Chop}, F], S, P, \mathcal{A}_{\text{CSC}}}^{\text{sim}^*-\text{ac-cprf}}(\lambda) = (n(\lambda) + 1) \cdot \text{Adv}_{F_{\text{Chop}}, S_{F_{\text{Chop}}}, P, \mathcal{D}}^{\text{sim}^*-\text{ac-cprf}}(\lambda).$$

where  $\mathcal{D}$  selects an adversary from  $\{\mathcal{B}_0, \mathcal{C}_0, \dots, \mathcal{C}_{n(\lambda)-1}\}$  to run.

The security for IND\*-NC security proceeds analogously, except without the need for a simulator.  $\square$

## G SIM\*-AC-PRF Security With Leakage

In this section we analyze the security of low-level compression functions as SIM\*-AC-PRFs in the face of key leakage. We start with some combinatorial preliminaries and then consider random oracles in Section G.1 and the Davies-Meyer construction in Section G.2.

**Combinatorial Preliminaries.** Output function  $\text{Chop}$  is a key-less function that maps from  $\text{Chop.Inp}(\lambda)$  to  $\text{Chop.Out}(\lambda)$ . Let  $\rho_{\text{Chop}}(\lambda)$  be a lower bound on the number of preimages that any  $y \in \text{Chop.Out}(\lambda)$  has. We will require that this be large (superpolynomial). We require that the output of  $\text{Chop}$  on random input be statistically close to uniform, meaning

$$\delta_{\text{Chop}}(\lambda) = \max_{\mathcal{A}} \Pr[1 \leftarrow \mathcal{A}(1^\lambda, \text{Chop}(1^\lambda, x))] - \Pr[1 \leftarrow \mathcal{A}(1^\lambda, y)]$$

is negligible where  $x \leftarrow \text{Chop.Inp}(\lambda)$  and  $y \leftarrow \text{Chop.Out}(\lambda)$ . Next we require that when sampling many independent  $y = \text{Chop}(1^\lambda, x)$ , it is unlikely for any of these outputs to repeat many times. To formalize this, define  $\mu(y_1, \dots, y_u) = \max_y |\{i : y_i = y\}|$  and

$$P_{\text{Chop}}^{\text{coll}}(\lambda, u, m) = \Pr[\mu(\text{Chop}(1^\lambda, x_1), \dots, \text{Chop}(1^\lambda, x_u)) \geq m(\lambda)]$$

where  $x_1, \dots, x_u \leftarrow \text{Chop.Inp}(\lambda)$ . Bellare, et al. [5, Lemma 7.1] bound this as follows.

**Lemma 1 (Multi-collision Probability).** *Let  $u \geq 1, \kappa \geq 0$  and  $m \geq \kappa \cdot \ln |\text{Chop.Out}| + 2u/|\text{Chop.Out}|$ . Then  $P_{\text{Chop}}^{\text{coll}}(u, m) \leq u \cdot \delta_{\text{Chop}} + \exp(-\kappa/3)$ .*

The analysis of AMAC as a PRF uses the above quantities [5]. We require an additional property of  $\text{Chop}$ , that it is possible to sample pre-images uniformly. This property is required so that simulators can find preimages to the previously random outputs of evaluation. For similar reasons, this property is also required when analyzing AMAC's indistinguishability from a random oracle [7]. Formally, we require the existence of a randomized algorithm  $\text{Chop}^{-1}$  such that  $\text{Chop}(1^\lambda, \text{Chop}^{-1}(1^\lambda, y)) = y$  and

$$\varepsilon_{\text{Chop}}(\lambda) = \max_{\mathcal{A}} \Pr[1 \leftarrow \mathcal{A}(1^\lambda, x)] - \Pr[1 \leftarrow \mathcal{A}(1^\lambda, \text{Chop}^{-1}(1^\lambda, y))]$$

is negligible where  $x \leftarrow \text{Chop.Inp}(\lambda)$  and  $y \leftarrow \text{Chop.Out}(\lambda)$ . Note that  $\delta_{\text{Chop}}(\lambda) \leq \varepsilon_{\text{Chop}}(\lambda)$ . Concrete bounds for these terms are known [5, 7] for when  $\text{Chop}$  truncates a large bitstring or reduces a number modulo some fixed  $M$ .

### G.1 Security of Ideal Compression Function (Random Oracle) with Leakage

We consider the case that our underlying compression function is a random oracle. Formally, we capture this by the function family which simply returns the output of its ideal primitive on  $(K, x)$ . Recall that our leakage resilient definition in Section 3.1 was equivalent to considering plain SIM\*-AC-PRF security of a related function family that used  $\text{Chop}(K)$  as the output on a new domain point.

**Theorem 8.** Let  $F$  be a function family with  $F.Ev^{P_{rom}}(1^\lambda, K, x) = P_{rom}(K, x)$  and  $\varepsilon \notin F.Inp$ . Let  $Chop(1^\lambda, \cdot) : F.K(\lambda) \rightarrow Chop.Out(\lambda)$  and  $m(\lambda) \geq 1$ . Let  $F_{Chop}$  be defined equivalently to  $F$ , except that  $F_{Chop}.Ev^P(1^\lambda, K, \varepsilon) = Chop(1^\lambda, K)$ . Then we can build an efficient  $S$  such that

$$Adv_{F_{Chop}, S, P, \mathcal{A}_{prf}}^{sim^*-ac-prf}(\lambda) \leq \frac{u(\lambda)^2}{2|F_{Chop}.K(\lambda)|} + P_{Chop}^{coll}(\lambda, u(\lambda), m(\lambda)) + \frac{(m(\lambda) - 1)p(\lambda)}{\rho_{Chop}(\lambda)} + u(\lambda) \cdot \varepsilon_{Chop}(\lambda)$$

holds for all  $\mathcal{A}_{prf}$  making  $p(\lambda)$  queries to  $PPRIM$ , and querying at most  $u(\lambda)$  distinct users.

This bound exactly matches the bound in Bellare, et al.’s analogous result in their Theorem 7.2, except we have an additional  $u \cdot \varepsilon$ . It does not arise in their proof because their definition of PRF security under leakage returns  $Chop(1^\lambda, K)$  in both the real and random world. If a random value was returned in the random world instead, it would add a  $u \cdot \delta$  term to the bound. They don’t need  $\varepsilon$  because they don’t need a simulator applying  $Chop^{-1}$  to the random value. With their proof modularization,  $Chop(1^\lambda, K)$  looking random instead arises in the final step of analyzing ACSC or AMAC and arises in their Theorem 8.2 and Theorem 8.5 as a  $q_{Ev} \cdot \delta$  term where  $q_{Ev}$  is the number of evaluation queries made. With our modularization, combining Theorem 8 with Theorem 5 would give a  $(q_{Ev} + q_{Con}) \cdot n \cdot \varepsilon$  term where  $q_{Con}$  is the number of constrained queries made and  $n$  is the length of the longest query to ACSC. This is small and so not important to optimize, though the “true” bound should be  $q_{Ev} \cdot \varepsilon$  because  $Chop(1^\lambda, \cdot)$  is never applied by constraining and is applied once per evaluation.

*Proof.* We define our simulator  $S$  as follows.

|                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\begin{array}{l} S.Init(1^\lambda) \\ K_{(\cdot)} \leftarrow [\cdot] \\ \text{Return } K_{(\cdot)} \\ S.Ev^{PPRIM}(1^\lambda, u, x : K) \\ y \leftarrow F_{Chop}.Ev^P(1^\lambda, K_u, x) \\ \text{Return } y \end{array}$ | $\begin{array}{l} S.Exp^{PPRIM}(1^\lambda, u, T_u : K) \\ \text{If } K_u = \perp: \\ \quad \text{If } T_u[\varepsilon] = \perp: T_u[\varepsilon] \leftarrow \$ Chop.Out(\lambda) \\ \quad K_u \leftarrow \$ Chop^{-1}(1^\lambda, T_u[\varepsilon]) \\ \quad \text{For } x \neq \varepsilon \text{ s.t. } T_u[x] \neq \perp: \\ \quad \quad PPRIM(Prog, K_u, x, T_u[x]) \\ \text{Return } K_u \end{array}$ |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Our proof closely follows the steps of the proof of PRF security with leakage [5, Theorem 7.2, ePrint version] and  $SIM^*$ -AC-PRF security [22, Theorem D.1] of random oracles.

We construct a series of hybrids  $G_0$  to  $G_3$  switching from the real world of  $SIM^*$ -AC-PRF to the ideal world. Game  $G_0$  modifies the real world to sample keys so that no two users have the same key and no  $m$  users have keys with the same leakage. A birthday bound and  $P_{Chop}^{coll}$  bound how detectable this change is. Additionally,  $PPRIM$  and  $Ev$  are rewritten to separately sample their outputs at random with appropriate checks to make sure they are consistent. Game  $G_1$  removes these consistency checks. This modification is only detectable if the adversary queries an unexposed user’s key to  $PPRIM$ , which we can bound with  $(m(\lambda) - 1)p(\lambda)/\rho_{Chop}(\lambda)$ . Game  $G_2$  returns to sampling all of the keys uniformly at random. Finally,  $G_3$  switches from sampling uniformly random keys to first sampling the “leakage” of the key, then sampling the key using  $Chop^{-1}$ . This makes it equivalent to the ideal world. This switch is bounded by  $u(\lambda) \cdot \varepsilon_{Chop}(\lambda)$ . The following claims will hold:

1.  $\Pr[G_{F_{Chop}, S, P_{rom}, \mathcal{A}_{prf}}^{sim^*-ac-prf}(\lambda) | b = 1] - \Pr[G_0(\lambda)] \leq u(\lambda)^2/2|F_{Chop}.K(\lambda)| + P_{Chop}^{coll}(\lambda, u(\lambda), m(\lambda))$
2.  $\Pr[G_0(\lambda)] - \Pr[G_1(\lambda)] \leq \frac{(m(\lambda)-1)p(\lambda)}{\rho_{Chop}(\lambda)}$
3.  $\Pr[G_1(\lambda)] - \Pr[G_2(\lambda)] \leq 0$
4.  $\Pr[G_2(\lambda)] - \Pr[G_3(\lambda)] \leq u(\lambda) \cdot \varepsilon_{Chop}(\lambda)$
5.  $\Pr[G_3(\lambda)] = \Pr[\neg G_{F_{Chop}, S, P_{rom}, \mathcal{A}_{prf}}^{sim^*-ac-prf}(\lambda) | b = 0]$

Combining these (in)equalities gives the claimed bound.

**Claim 1.** Game  $G_0$  is defined in Fig. 36. It contains highlighted and boxed code. For notational simplicity, we assume the name of the users queried by  $\mathcal{A}_{prf}$  are  $1, \dots, u(\lambda)$ . Then we define  $\mathcal{K}$  to be the set of key tuples  $(K_1, \dots, K_{u(\lambda)})$  that contain no key collisions between users and not  $m(\lambda)$ -wise collisions between the

| Games $\boxed{G_0(\lambda)}, \boxed{G_1(\lambda)}, G_2(\lambda)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | $\text{Ev}(u, x)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\sigma_{\text{rom}} \leftarrow [\cdot]$<br>$\mathcal{K} \leftarrow \{(K_1, \dots, K_{u(\lambda)}) : \forall i \neq j, K_i \neq K_j \wedge \mu(\text{Chop}(1^\lambda, K_1), \dots, \text{Chop}(1^\lambda, K_{u(\lambda)})) < m(\lambda)\}$<br>For $i = 1, \dots, u(\lambda)$ : $K_i \leftarrow \$ \text{F}_{\text{Chop}}.K(\lambda)$<br>If $(K_1, \dots, K_{u(\lambda)}) \notin \mathcal{K}$ :<br>$\text{bad}_{\text{coll}} \leftarrow \text{true}; \boxed{(K_1, \dots, K_{u(\lambda)}) \leftarrow \$ \mathcal{K}}$<br>$b' \leftarrow \$ \mathcal{A}_{\text{prf}}^{\text{Ev}, \text{Exp}, \text{PPRIM}}(1^\lambda)$<br>Return $(b' = 1) \wedge \neg \text{bad}_{\text{coll}}$<br>$\text{PPRIM}(\text{Op}, (k, x), y)$<br>If $\text{Op} = \text{Ls}$ : $y \leftarrow \$ \text{F}.\text{Out}(\lambda)$<br>If $\exists u$ s.t. $\neg \text{corr}_u$ and $K_u = k$ and $T_u[x] \neq \perp$ :<br>$\text{bad}_{\text{guess}} \leftarrow \text{true}; \sigma_{\text{rom}}[(k, x)] \leftarrow T_u[x]$<br>If $\sigma_{\text{rom}}[(k, x)] = \perp$ : $\sigma_{\text{rom}}[(k, x)] \leftarrow y$<br>If $\text{Op} = \text{Ls}$ : Return $\sigma_{\text{rom}}[(k, x)]$<br>Return $\perp$ | If $x = \varepsilon$ : $T_u[x] \leftarrow \text{Chop}(1^\lambda, K_u)$<br>Else if $\text{corr}_u$ : $T_u[x] \leftarrow \$ \text{P}_{\text{rom}}.\text{Ls}(1^\lambda, (K_u, x) : \sigma_{\text{rom}})$<br>Else if $T_u[x] \neq \perp$ : $T_u[x] \leftarrow T_u[x]$ // No-op<br>Else if $\sigma_{\text{rom}}[(K_u, x)] \neq \perp$ :<br>$\text{bad}_{\text{guess}} \leftarrow \text{true}; T_u[x] \leftarrow \sigma_{\text{rom}}[(K_u, x)]$<br>Else $T_u[x] \leftarrow \$ \text{F}.\text{Out}(\lambda)$<br>Return $T_u[x]$ |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | $\text{EXP}(u)$<br>If $\neg \text{corr}_u$ :<br>For $x \neq \varepsilon$ s.t. $T_u[x] \neq \perp$ :<br>$\text{P}_{\text{rom}}.\text{Prog}(1^\lambda, (K_u, x), T_u[x] : \sigma_{\text{rom}})$<br>$\text{corr}_u \leftarrow \text{true}$<br>Return $K_u$                                                                                                                                                                                                                                                                  |

Fig. 36. Hybrid games  $G_0$ ,  $G_1$ , and  $G_2$  for proving security of  $\text{F}_{\text{Chop}}$ 

| Games $G_{2.5}(\lambda), G_3(\lambda)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | $\text{Ev}(u, x)$                                                                                                                                                                                                                                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\sigma_{\text{rom}} \leftarrow \$ \text{P}_{\text{rom}}.\text{Init}(1^\lambda)$<br>For $i = 1, \dots, u(\lambda)$ :<br>$K_i \leftarrow \$ \text{F}_{\text{Chop}}.K(\lambda)$<br>$\ell_i \leftarrow \$ \text{Chop}.\text{Out}(\lambda)$<br>$K_i \leftarrow \$ \text{Chop}^{-1}(1^\lambda, \ell_i)$<br>$b' \leftarrow \$ \mathcal{A}_{\text{prf}}^{\text{Ev}, \text{Exp}, \text{PPRIM}}(1^\lambda)$<br>Return $(b' = 1)$<br>$\text{PPRIM}(\text{Op}, (k, x), y)$<br>$y \leftarrow \$ \text{P}_{\text{rom}}.\text{Op}(1^\lambda, (k, x), y : \sigma_{\text{rom}})$<br>Return $y$ | If $x = \varepsilon$ : $T_u[x] \leftarrow \text{Chop}(1^\lambda, K_u)$<br>Else if $\text{corr}_u$ : $T_u[x] \leftarrow \$ \text{P}_{\text{rom}}.\text{Ls}(1^\lambda, (K_u, x) : \sigma_{\text{rom}})$<br>Else if $T_u[x] \neq \perp$ : $T_u[x] \leftarrow T_u[x]$ // No-op<br>Else $T_u[x] \leftarrow \$ \text{F}.\text{Out}(\lambda)$<br>Return $T_u[x]$ |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | $\text{EXP}(u)$<br>If $\neg \text{corr}_u$ :<br>For $x \neq \varepsilon$ s.t. $T_u[x] \neq \perp$ :<br>$\text{P}_{\text{rom}}.\text{Prog}(1^\lambda, (K_u, x), T_u[x] : \sigma_{\text{rom}})$<br>$\text{corr}_u \leftarrow \text{true}$<br>Return $K_u$                                                                                                   |

Fig. 37. Hybrid games  $G_{2.5}$  and  $G_3$  for proving security of  $\text{F}_{\text{Chop}}$ 

“leakages” of the keys. If the randomly sampled keys are not in  $\mathcal{K}$  we re-sample them from  $\mathcal{K}$  and set a bad flag with ensures that the game returns false.

Additionally, we’ve written out the code of PPRIM to explicitly sample the random function using  $\sigma_{\text{rom}}$  and Ev to sample its random outputs using  $T_u$  for unexposed users. We need these random tables to define a single consistent random function, so we maintain the invariant that  $T_u[x] = \sigma_{\text{rom}}[(K_u, x)]$  whenever both these values are non- $\perp$  for unexposed users. Because users are guaranteed to have distinct keys, we do not need to account for consistency between different users’  $T_u$  tables. When users are exposed, we program  $\sigma_{\text{rom}}[(K_u, x)]$  to equal  $T_u[x]$  wherever the latter is non- $\perp$  and thereafter use  $\text{P}_{\text{rom}}$  with  $\sigma_{\text{rom}}$  directly in Ev. These invariants ensure the behavior of these oracles matches the real world.

So we get  $\Pr[\text{G}_{\text{FChop}, \text{S}, \text{P}_{\text{rom}}, \mathcal{A}_{\text{prf}}}^{\text{sim}^* \text{-ac-prf}}(\lambda) | b = 1] - \Pr[G_0(\lambda)] \leq \Pr[G_0(\lambda) \text{ sets } \text{bad}_{\text{coll}}]$ . Then the claim follows from  $\Pr[\text{bad}_{\text{coll}}] \leq \binom{u(\lambda)}{2} / |\text{F}_{\text{Chop}}.K(\lambda)| + \text{P}_{\text{Chop}}^{\text{coll}}(\lambda, u(\lambda), m(\lambda))$  where these terms come, respectively, from the probability of two user’s sampling the same key and the probability of  $m(\lambda)$  users having the same key leakage.

**Claim 2.** Game  $G_1$  is also defined in Fig. 36. It contains boxed, but not highlighted code. It differs from  $G_0$  in removing code which occurs after  $\text{bad}_{\text{guess}}$  is set so  $\Pr[G_0(\lambda)] - \Pr[G_1(\lambda)] \leq \Pr[G_1 \text{ sets } \text{bad}_{\text{guess}}]$ . This bad flag corresponds to the checks that were keeping  $\sigma_{\text{rom}}$  and  $T_u$  consistent for unexposed users; it can only be

set if PPRIM is queried with a  $k$  equal to the key of an exposed user. (Note that  $P_{\text{rom}}.\text{Prog}$  and  $P_{\text{rom}}.\text{Ls}$  in EXP and EV can add values to  $\sigma_{\text{rom}}$ , but only using exposed users' keys which don't equal other users' keys.)

Suppose the adversary queries PPRIM with a  $k$  such that  $\text{Chop}(1^\lambda, k) = \ell$ . Define sets  $S \supseteq S' \supseteq S''$  by  $S = \{k : \text{Chop}(1^\lambda, k) = \ell\}$ ,  $S' = \{K_u : \text{Chop}(1^\lambda, K_u) = \ell\}$  and  $S'' = \{K_u : \text{Chop}(1^\lambda, K_u) = \ell \wedge \text{corr}_u\}$ . We want to bound the probability that  $k \in S' \setminus S''$ . From the perspective of the adversary, each key in  $S' \setminus S''$  is uniformly randomly distributed in  $S \setminus S''$  so we can bound this by  $(|S'| - |S''|)/(|S| - |S''|) \leq |S'|/|S| \leq (m(\lambda) - 1)/\rho_{\text{Chop}}(\lambda)$ . The first inequality follows because  $(x - n)/(y - n) \leq x/y$  whenever  $y \geq x$  and  $y > n \geq 0$ . The second inequality follows because the sampling of the keys ensures  $|S'| \leq m(\lambda) - 1$  and  $|S| \geq \rho_{\text{Chop}}(\lambda)$  by definition. A union bound over the  $p(\lambda)$  queries to PPRIM gives the claim.

**Claim 3.** Game  $G_2$  is also defined in Fig. 36. It contains neither boxed nor highlighted code. Because  $G_1$  always returns **false** when  $\text{bad}_{\text{coll}}$  was set, the removal of boxed code can only increase the probability  $G_2$  returns **true**, giving the claim.

**Claim 4.** Game  $G_{2.5}$  is defined in Fig. 37. It does not contain highlighted code. It was obtained from  $G_2$  by removing unused if statements in PPRIM and EV and by simplifying PPRIM to just call  $P_{\text{rom}}$ . This is an equivalent rewriting of  $G_2$ . Hence  $\Pr[G_2(\lambda)] = \Pr[G_{2.5}(\lambda)]$ .

Now  $G_3$  in the same figure contains the highlighted code. It differs from  $G_{2.5}$  only in that rather than sampling keys uniformly, we instead sample their “leakage”  $\ell_i$  at random and then use  $\text{Chop}^{-1}$ . Applying a  $\varepsilon_{\text{Chop}}(\lambda)$  bound for each key gives the claim.

**Claim 5.** Finally we claim  $G_3$  is identical to the ideal world with simulator  $S$ , except with flipped output. Essentially, the ideal world just delays sampling  $\ell_i$  until it is needed in EV or EXP and delays sampling  $K_i$  from  $\ell_i$  until it is needed in EXP. Correctness of EV uses in part that  $\text{Chop}(1^\lambda, \text{Chop}^{-1}(1^\lambda, \ell)) = \ell$ .  $\square$

## G.2 Security of Davies-Meyer Compression Function with Leakage

In this section, we will look at when the compression function is built from a blockcipher using the Davies-Mayer construction. We analyze this in the ideal cipher model as captured by  $P_{\text{icm}}$  defined below. Here we use the convention that setting  $\sigma_k^+[x] \leftarrow z$  also sets  $\sigma_k^-[z] \leftarrow x$  and vice versa. We define  $\bar{o} = -$  if  $o = +$  and vice versa.

$$\frac{P_{\text{icm}}.\text{Init}(1^\lambda)}{\text{Return } \sigma^+ = [\cdot]} \left| \begin{array}{l} P_{\text{icm}}.\text{Ls}(1^\lambda, (k, x, o) : \sigma) \quad // \quad o \in \{+, -\} \\ \text{If } \sigma_k^o[x] = \perp : \\ \quad \sigma_k^o[x] \leftarrow \mathcal{R}_\lambda \setminus \{\sigma_k^o[x'] : \sigma_k^o[x'] \neq \perp\} \\ \text{Return } \sigma_k^o[x] \end{array} \right| \frac{P_{\text{icm}}.\text{Prog}(1^\lambda, (k, x, o), z)}{\text{Return } \perp} \quad // \quad o \in \{+, -\} \left| \begin{array}{l} \text{If } \sigma_k^o[x] = \perp \text{ and } \sigma_k^{\bar{o}}[k, z] = \perp : \\ \quad \sigma_k^o[x] \leftarrow z \\ \text{Return } \perp \end{array} \right|$$

Here  $(k, x, +)$  indicates a forward invocation of the ideal cipher with key  $k$  and input  $x$  while  $(k, x, -)$  indicates a backwards query.

The Davies-Meyer construction DM is defined by  $\text{DM.Ev}^{P_{\text{icm}}}(1^\lambda, K, x) = P_{\text{icm}}(x, K, +) \oplus K$  with  $\text{DM.Inp} = \text{DM.Out} = \mathcal{R}$ . Note the key of DM is used as an input of  $P_{\text{icm}}$  and the input of DM is used as a key of  $P_{\text{icm}}$ .

**Theorem 9.** Let  $\text{Chop}(1^\lambda, \cdot) : \text{DM.K}(\lambda) \rightarrow \text{Chop.Out}(\lambda)$  and  $m(\lambda) \geq 1$ . Then we can build an efficient  $S$  such that

$$\text{Adv}_{\text{DM}_{\text{Chop}}, S, P, \mathcal{A}_{\text{prf}}}^{\text{sim}^* - \text{ac-prf}}(\lambda) \leq \frac{u(\lambda)(2q(\lambda) + u(\lambda))}{2|\text{DM}_{\text{Chop}}.\text{K}(\lambda)|} + P_{\text{Chop}}^{\text{coll}}(\lambda, u(\lambda), m(\lambda)) + \frac{(u(\lambda) + m(\lambda) - 1)p(\lambda)}{\rho_{\text{Chop}}(\lambda)} + u(\lambda) \cdot \varepsilon_{\text{Chop}}(\lambda)$$

holds for all  $\mathcal{A}_{\text{prf}}$  making  $p(\lambda)$  queries to PPRIM,  $q(\lambda)$  queries to EV, and querying at most  $u(\lambda)$  distinct users.

The dependence on  $up/\rho_{\text{Chop}}$  do not appear the bound claimed for IND-PRF security with leakage in [5]. They both arise in our proof from patching errors we identified in the original proof. This term is a meaningful degradation in security that obviates the “better than trivial” multi-user security desired of the proof. We discuss the source of the errors and whether this term might be improved after our proof.

*Proof.* Our proof is somewhat analogous to the proof of Theorem 8, but the use of an ideal cipher rather than a random oracle adds additional complexity because we just account for collisions with both the inputs and the outputs of the primitive. We define our simulator  $S$  as follows.

$$\begin{array}{l|l}
 \begin{array}{l}
 \boxed{\text{S.Init}(1^\lambda)} \\
 \boxed{K_{(\cdot)} \leftarrow [\cdot]} \\
 \boxed{\text{Return } K_{(\cdot)}} \\
 \boxed{\text{S.Ev}^{\text{PPRIM}}(1^\lambda, u, x : K)} \\
 y \leftarrow \text{DM}_{\text{Chop}}.\text{Ev}^{\text{P}}(1^\lambda, K_u, x) \\
 \boxed{\text{Return } y}
 \end{array} &
 \begin{array}{l}
 \boxed{\text{S.Exp}^{\text{PPRIM}}(1^\lambda, u, T_u : K)} \\
 \text{If } K_u = \perp: \\
 \quad \text{If } T_u[\varepsilon] = \perp: T_u[\varepsilon] \leftarrow \text{Chop.Out}(\lambda) \\
 \quad K_u \leftarrow \text{Chop}^{-1}(1^\lambda, T_u[\varepsilon]) \\
 \quad \text{For } x \neq \varepsilon \text{ s.t. } T_u[x] \neq \perp: \\
 \quad \quad \text{PPRIM}(\text{Prog}, (x, K_u, +), K_u \oplus T_u[x]) \\
 \boxed{\text{Return } K_u}
 \end{array}
 \end{array}$$

We construct a series of hybrids  $G_0$  to  $G_3$  switching from the real world of SIM\*-AC-PRF to the ideal world. Game  $G_0$  modifies the real world to sample keys so that no two users have the same key and no  $m$  users have keys with the same leakage. A birthday bound and  $\text{P}_{\text{Chop}}^{\text{coll}}$  bound how detectable this change is. Additionally, PPRIM and EV are rewritten to separately sample their outputs at random with appropriate checks to make sure they are consistent. Game  $G_1$  removes these consistency checks. Roughly, this modification is only detectable if the adversary queries an unexposed user's key  $K_u$  to PPRIM, which we can bound with  $(m(\lambda) - 1)p(\lambda)/\rho_{\text{Chop}}(\lambda)$  or it queries an output value  $z$  satisfying  $z \oplus \text{Ev}(u, x) = K_u$  which we can bound with  $u(\lambda)p(\lambda)/\rho_{\text{Chop}}(\lambda)$ , or EV samples a random output which has already been used as an output of the ideal cipher which we can bound with  $q(\lambda) \cdot (q(\lambda) + p(\lambda))/|\mathcal{R}_\lambda|$ . Game  $G_2$  returns to sampling all of the keys uniformly at random. Finally,  $G_3$  switches from sampling uniformly random keys to first sampling the “leakage” of the key, then sampling the key using  $\text{Chop}^{-1}$ . This makes it equivalent to the ideal world. This switch is bounded by  $u(\lambda) \cdot \varepsilon_{\text{Chop}}(\lambda)$ . The following claims will hold:

1.  $\Pr[\text{G}_{\text{DM}_{\text{Chop}}, \text{S}, \text{P}, \mathcal{A}_{\text{prf}}}^{\text{sim}^*-\text{ac-prf}}(\lambda) | b = 1] - \Pr[G_0(\lambda)] \leq u(\lambda)^2/2|\text{DM}_{\text{Chop}}.\text{K}(\lambda)| + \text{P}_{\text{Chop}}^{\text{coll}}(\lambda, u(\lambda), m(\lambda))$
2.  $\Pr[G_0(\lambda)] - \Pr[G_1(\lambda)] \leq (m(\lambda) - 1)p(\lambda)/\rho_{\text{Chop}}(\lambda) + u(\lambda)p(\lambda)/\rho_{\text{Chop}}(\lambda) + u(\lambda)q(\lambda)/|\text{DM}_{\text{Chop}}.\text{K}(\lambda)|$
3.  $\Pr[G_1(\lambda)] - \Pr[G_2(\lambda)] \leq 0$
4.  $\Pr[G_2(\lambda)] - \Pr[G_3(\lambda)] \leq u(\lambda) \cdot \varepsilon_{\text{Chop}}(\lambda)$
5.  $\Pr[G_3(\lambda)] = \Pr[\neg \text{G}_{\text{DM}_{\text{Chop}}, \text{S}, \text{P}, \mathcal{A}_{\text{prf}}}^{\text{sim}^*-\text{ac-prf}}(\lambda) | b = 0]$

Combining these (in)equalities gives the claimed bound.

**Claim 1.** Game  $G_0$  is defined in Fig. 38. It contains highlighted and boxed code. For notational simplicity, we assume the name of the users queried by  $\mathcal{A}_{\text{prf}}$  are  $1, \dots, u(\lambda)$ . Then we define  $\mathcal{K}$  to be the set of key tuples  $(K_1, \dots, K_{u(\lambda)})$  that contain no key collisions between users and not  $m(\lambda)$ -wise collisions between the “leakages” of the keys. If the randomly sampled keys are not in  $\mathcal{K}$  we re-sample them from  $\mathcal{K}$  and set a bad flag with ensures that the game returns false.

Additionally, we have divided the lazy sampling of the ideal cipher between state  $\sigma$  used in PPRIM and table  $V$  used in EV. Not that when a user is exposed, the values from  $V_x[u]$  are copied into  $\sigma$  and  $\sigma$  is henceforth always used for it. Together they (partially) define a keyed-permutation  $\Pi$  by

$$\Pi_x(k) = \begin{cases} \sigma_x^+[k], & \text{if } \sigma_x^+[k] \neq \perp \\ V_x[u], & \text{if } V_x[u] \neq \perp \text{ and } K = K_u \\ \perp, & \text{otherwise.} \end{cases}$$

For this to be well-defined the checks need to ensure that the first two cases above always give consistent answers. Inconsistent answers could arise from:

1.  $\sigma_x^+[k] \neq V_x[u]$  where  $K = K_u$  and  $\perp \notin \{\sigma_x^+[k], V_x[u]\}$
2.  $\sigma_x^+[k] = V_x[u]$  where  $K \neq K_u$  and  $\perp \notin \{\sigma_x^+[k], V_x[u]\}$
3.  $V_x[u] \neq V_x[v]$  where  $K_u = K_v$  and  $\perp \notin \{V_x[u], V_x[v]\}$
4.  $V_x[u] = V_x[v]$  where  $K_u \neq K_v$  and  $\perp \notin \{V_x[u], V_x[v]\}$ .

| Games $\boxed{\mathbf{G}_0(\lambda)}, \boxed{\mathbf{G}_1(\lambda)}, \mathbf{G}_2(\lambda)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | $\text{Ev}(u, x)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\sigma \leftarrow [\cdot]$<br>$\mathcal{K} \leftarrow \{(K_1, \dots, K_{u(\lambda)}) : \forall i \neq j, K_i \neq K_j \wedge \mu(\text{Chop}(1^\lambda, K_1), \dots, \text{Chop}(1^\lambda, K_{u(\lambda)})) < m(\lambda)\}$<br>For $i = 1, \dots, u(\lambda)$ : $K_i \leftarrow \$ \text{DM}_{\text{Chop} \cdot \mathbf{K}}(\lambda)$<br>If $(K_1, \dots, K_{u(\lambda)}) \notin \mathcal{K}$ :<br>$\text{bad}_{\text{coll}} \leftarrow \text{true}; \boxed{(K_1, \dots, K_{u(\lambda)}) \leftarrow \$ \mathcal{K}}$<br>$b' \leftarrow \$ \mathcal{A}_{\text{prf}}^{\text{Ev, Exp, PPRIM}}(1^\lambda)$<br>Return $(b' = 1) \wedge \neg \text{bad}_{\text{coll}}$<br>$\text{PPRIM}(\text{Ls}, (x, k, +))$<br>If $\exists u$ s.t. $\neg \text{corr}_u$ and $k = K_u$ :<br>$\text{bad}_{\text{guess}} \leftarrow \text{true}; \text{If } V_x[u] \neq \perp: \sigma_x^+[k] \leftarrow V_x[u]$<br>If $\sigma_x^+[k] = \perp: \sigma_x^+[k] \leftarrow \$ \mathcal{R}_\lambda \setminus \sigma.\text{Range}(x)$<br>If $\exists u$ s.t. $\neg \text{corr}_u$ and $V_x[u] = \sigma_x^+[k]$ :<br>$\text{bad}_{\text{guess2}} \leftarrow \text{true}; \sigma_x^+[k] \leftarrow \$ \mathcal{R}_\lambda \setminus \text{Range}(x)$<br>Return $\sigma_x^+[k]$<br>$\text{PPRIM}(\text{Prog}, (x, k, \circ), z)$<br>If $\circ = -$ : $(k, z) \leftarrow (z, k)$<br>If $\exists u$ s.t. $\neg \text{corr}_u$ and $k = K_u$ :<br>$\text{bad}_{\text{guess}} \leftarrow \text{true}; \text{If } V_x[u] \neq \perp: \sigma_x^+[k] \leftarrow V_x[u]$<br>If $\exists u$ s.t. $\neg \text{corr}_u$ and $V_x[u] = z$ :<br>$\text{bad}_{\text{guess2}} \leftarrow \text{true}; \sigma_x^-[z] \leftarrow K_u$<br>If $\sigma_x^+[k] = \perp$ and $\sigma_x^-[z] = \perp: \sigma_x^+[k] \leftarrow z$<br>Return $\perp$ | $\text{If } x = \varepsilon: \text{Return } \text{Chop}(1^\lambda, K_u)$<br>Else if $\text{corr}_u = \text{true}: V_x[u] \leftarrow \$ \text{P}_{\text{icm}}.\text{Ls}((x, K_u, +) : \sigma)$<br>Else if $V_x[u] \neq \perp: V_x[u] \leftarrow V_x[u]$ // No-op<br>Else if $\sigma_x^+[K_u] \neq \perp: \text{bad}_{\text{guess}} \leftarrow \text{true}; V_x[u] \leftarrow \sigma_x^+[K_u]$<br>Else $V_x[u] \leftarrow \$ \mathcal{R}_\lambda$<br>If $\exists v \neq u$ s.t. $V_x[v] = V_x[u]$ :<br>$\text{bad}_{\text{§}} \leftarrow \text{true}; V_x[u] \leftarrow \$ \mathcal{R}_\lambda \setminus \text{Range}(x)$<br>Else if $\exists k$ s.t. $\sigma_x^+[k] = V_x[u]$ :<br>$\text{bad}_{\text{guess2}} \leftarrow \text{true}; V_x[u] \leftarrow \$ \mathcal{R}_\lambda \setminus \text{Range}(x)$<br>$T_u[x] \leftarrow K_u \oplus V_x[u]$<br>Return $T_u[x]$<br>$\text{PPRIM}(\text{Ls}, (x, z, -))$<br>If $\exists u$ s.t. $\neg \text{corr}_u$ and $V_x[u] = z$ :<br>$\text{bad}_{\text{guess2}} \leftarrow \text{true}; \sigma_x^-[z] \leftarrow K_u$<br>If $\sigma_x^-[z] = \perp: \sigma_x^-[z] \leftarrow \$ \mathcal{R}_\lambda \setminus \sigma.\text{Domain}(x)$<br>If $\exists u$ s.t. $\neg \text{corr}_u$ and $\sigma_x^-[z] = K_u$ :<br>$\text{bad}_{\text{guess}} \leftarrow \text{true}$<br>If $V_x[u] \neq \perp: \sigma_x^-[z] \leftarrow \$ \mathcal{R}_\lambda \setminus \text{Domain}(x)$<br>Return $\sigma_x^-[z]$<br>$\text{EXP}(u)$<br>If $\neg \text{corr}_u$ : For all $x \neq \varepsilon$ s.t. $T_u[x] \neq \perp$ :<br>$\text{P}_{\text{icm}}.\text{Prog}(1^\lambda, (x, K_u, +), V_x[u] : \sigma)$<br>$\text{corr}_u \leftarrow \text{true}; \text{Return } K_u$ |
| <b>Definitions</b> // $x \neq \varepsilon$<br>$\text{Range}(x) = \{\sigma_x^+[k] : \sigma_x^+[k] \neq \perp\} \cup \{V_x[u] : V_x[u] \neq \perp\}$ $\sigma.\text{Range}(x) = \{\sigma_x^+[k] : \sigma_x^+[k] \neq \perp\}$<br>$\text{Domain}(x) = \{k : \sigma_x^+[k] \neq \perp\} \cup \{K_u : V_x[u] \neq \perp\}$ $\sigma.\text{Domain}(x) = \{k : \sigma_x^+[k] \neq \perp\}$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

**Fig. 38.** Hybrid games  $\mathbf{G}_0$ ,  $\mathbf{G}_1$ , and  $\mathbf{G}_2$  for proving security of  $\text{DM}_{\text{Chop}}$ 

Inconsistency (3) can never arise because we are sampling all user keys to be distinct. The consistency checks associated with  $\text{bad}_{\text{guess}}$  ensure that (1) will not occur. The consistency checks associated with  $\text{bad}_{\text{guess2}}$  ensure that (2) will not occur. The consistency checks associated with  $\text{bad}_{\text{§}}$  ensures that (4) will not occur.

So we get  $\Pr[\mathbf{G}_{\text{DMChop}, \text{S}, \text{P}, \mathcal{A}_{\text{prf}}}^{\text{sim*ac-prf}}(\lambda) | b = 1] - \Pr[\mathbf{G}_0(\lambda)] \leq \Pr[\mathbf{G}_0(\lambda) \text{ sets } \text{bad}_{\text{coll}}] \leq \binom{u(\lambda)}{2} / |\text{DM}_{\text{Chop} \cdot \mathbf{K}}(\lambda)| + \text{P}_{\text{Chop}}^{\text{coll}}(\lambda, u(\lambda), m(\lambda))$  where these terms come, respectively, from the probability of two user's sampling the same key and the probability of  $m(\lambda)$  users having the same key leakage.

**Claim 2.** Game  $\mathbf{G}_1$  is also defined in Fig. 38. It contains boxed, but not highlighted code. It differs from  $\mathbf{G}_0$  in removing code which occurs after  $\text{bad}_{\text{§}}$ ,  $\text{bad}_{\text{guess}}$ ,  $\text{bad}_{\text{guess2}}$  is set. corresponding to  $\sigma$  and  $V_x$  no longer jointly defining a keyed-permutation. We have  $\Pr[\mathbf{G}_0(\lambda)] - \Pr[\mathbf{G}_1(\lambda)] \leq \Pr[\mathbf{G}_1 \text{ sets } \text{bad}_{\text{§}} \vee \text{bad}_{\text{guess}} \vee \text{bad}_{\text{guess2}}]$ .

Table  $V_x$  stores at most  $u(\lambda)$  non- $\perp$  values. A union bound over these and the number of Ev queries gives  $\Pr[\mathbf{G}_1 \text{ sets } \text{bad}_{\text{§}}] \leq q(\lambda)u(\lambda)/|\mathcal{R}_\lambda| = q(\lambda)u(\lambda)/|\text{DM}_{\text{Chop} \cdot \mathbf{K}}(\lambda)|$ .

We will analyze the probability that an individual PPRIM query can set the bad flags and then perform a union bound over all queries. Note that in  $\mathbf{G}_1$ , until a user has been corrupted the view of the adversary only depends on its key via  $\text{Chop}(1^\lambda, K_u)$  values returned by evaluation.<sup>8</sup>

Consider  $\text{bad}_{\text{guess}}$  for a forward lazy sampling query  $\text{PPRIM}(\text{Ls}, (x, k, +))$  or programming query. Analogously to the analysis of  $\text{bad}_{\text{guess}}$  for Theorem 8, there are fewer than  $m$  keys  $K_u$  satisfying  $\text{Chop}(1^\lambda, K_u) =$

<sup>8</sup> While  $T_u[x] \leftarrow V_x[u] \oplus K_u$  in Ev, this does not cause a dependence because  $V_x[u]$  is uniformly random and not used elsewhere.



|                                                                               |                                                                                                                              |
|-------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Games $G_{2.5}(\lambda), G_3(\lambda)$                                        | $EV(u, x)$                                                                                                                   |
| $\sigma \leftarrow \text{P}_{\text{icm}}.\text{Init}(1^\lambda)$              | If $x = \varepsilon$ : $T_u[x] \leftarrow \text{Chop}(1^\lambda, K_u)$                                                       |
| For $i = 1, \dots, u(\lambda)$ :                                              | Else if $\text{corr}_u = \text{true}$ : $T_u[x] \leftarrow K_u \oplus \text{P}_{\text{icm}}.\text{Ls}((x, K_u, +) : \sigma)$ |
| $K_i \leftarrow \text{DM}_{\text{Chop}}.\text{K}(\lambda)$                    | Else if $T_u[x] \neq \perp$ : $T_u[x] \leftarrow T_u[x]$ // No-op                                                            |
| $\ell_i \leftarrow \text{Chop}.\text{Out}(\lambda)$                           | Else $T_u[x] \leftarrow \mathcal{R}_\lambda$                                                                                 |
| $K_i \leftarrow \text{Chop}^{-1}(1^\lambda, \ell_i)$                          | Return $T_u[x]$                                                                                                              |
| $b' \leftarrow \mathcal{A}_{\text{prf}}^{\text{Ev, Exp, PPRIM}}(1^\lambda)$   | $EXP(u)$                                                                                                                     |
| Return $(b' = 1)$                                                             | If $\neg \text{corr}_u$ :                                                                                                    |
| $\text{PPRIM}(\text{Op}, (k, x), y)$                                          | For all $x \neq \varepsilon$ s.t. $T_u[x] \neq \perp$ :                                                                      |
| $y \leftarrow \text{P}_{\text{icm}}.\text{Op}(1^\lambda, (k, x), y : \sigma)$ | $\text{P}_{\text{icm}}.\text{Prog}(1^\lambda, (x, K_u, +), K_u \oplus T_u[x] : \sigma)$                                      |
| Return $y$                                                                    | $\text{corr}_u \leftarrow \text{true}$                                                                                       |
|                                                                               | Return $K_u$                                                                                                                 |

**Fig. 39.** Hybrid games  $G_{2.5}$  and  $G_3$  for proving security of  $\text{DM}_{\text{Chop}}$

$\text{Chop}(1^\lambda, K)$  so by a union bound the probability this query sets  $\text{bad}_{\text{guess}}$  is at most  $(m(\lambda) - 1)/\rho_{\text{Chop}}(\lambda)$ . The same logic with  $\sigma_x^-[z]$  in place of  $k$  applies for an inverse query  $\text{PPRIM}(\text{Ls}, (x, z, -))$ . Note that  $\text{bad}_{\text{guess}}$  can only be set by EV if it was already set by a PPRIM query, so we do not additionally need to analyze this.

Consider  $\text{bad}_{\text{guess2}}$  for an inverse lazy sampling query  $\text{PPRIM}(\text{Ls}, (x, z, -))$  or programming query. If  $V_x[u] = z$ , then  $z \oplus T_u[x] = K_u$ . By a union bound over all  $u$ , the probability this query sets  $\text{bad}_{\text{guess2}}$  is at most  $u(\lambda)/\rho_{\text{Chop}}(\lambda)$ . The same logic with  $\sigma_x^+[k]$  in place of  $z$  applies for a forward query  $\text{PPRIM}(\text{Ls}, (x, k, +))$ . Technically, we only need to consider users previously queried via  $EV(u, x)$  for the above. But then each later  $EV(u, x)$  query has at most a  $1/|\mathcal{R}_\lambda| \leq \rho_{\text{Chop}}(\lambda)$  probability of  $\text{bad}_{\text{guess2}}$  from this earlier primitive query, so we can fold them together to still get  $u(\lambda)/\rho_{\text{Chop}}(\lambda)$  overall.

Since the adversary makes at most  $p$  queries to PPRIM, a union bound gives the claim that

$$\Pr[G_0(\lambda)] - \Pr[G_1(\lambda)] \leq \frac{(m(\lambda) - 1)p(\lambda)}{\rho_{\text{Chop}}(\lambda)} + \frac{u(\lambda)p(\lambda)}{\rho_{\text{Chop}}(\lambda)} + \frac{u(\lambda)q(\lambda)}{|\text{DM}_{\text{Chop}}.\text{K}(\lambda)|}.$$

**Claim 3.** Game  $G_2$  is also defined in Fig. 38. It contains neither boxed nor highlighted code. Because  $G_1$  always returns false when  $\text{bad}_{\text{coll}}$  was set, the removal of boxed code can only increase the probability  $G_2$  returns true, giving the claim.

**Claim 4.** Game  $G_{2.5}$  is defined in Fig. 39. It does not contain highlighted code. It was obtained from  $G_2$  by removing unused if statements in PPRIM and EV and by simplifying PPRIM to just call  $\text{P}_{\text{icm}}$ . This is an equivalent rewriting of  $G_2$ . Hence  $\Pr[G_2(\lambda)] = \Pr[G_{2.5}(\lambda)]$ .

Now  $G_3$  in the same figure contains the highlighted code. It differs from  $G_{2.5}$  only in that rather than sampling keys uniformly, we instead sample their “leakage”  $\ell_i$  at random and then use  $\text{Chop}^{-1}$ . Applying a  $\varepsilon_{\text{Chop}}(\lambda)$  bound for each key gives the claim.

**Claim 5.** Finally we claim  $G_3$  is identical to the ideal world with simulator  $S$ , except with flipped output. Essentially, the ideal world just delays sampling  $\ell_i$  until it is needed in EV or EXP and delays sampling  $K_i$  from  $\ell_i$  until it is needed in EXP. Correctness of EV uses in part that  $\text{Chop}(1^\lambda, \text{Chop}^{-1}(1^\lambda, \ell)) = \ell$ .  $\square$

### G.3 Proof Errors and Improving Concrete Security for Truncation

Bellare, et al. [5, Theorem 10.1, [ePrint version](#)] prove IND-PRF security of DM under key-leakage in the (non-programmable) ideal cipher model. The given proof is incorrect. It falsely claims that its game  $G_1$  is equivalent to the real world of the IND-PRF game.

This game does not properly capture all of the four possible inconsistencies we discuss in Claim 1 of our proof. The consistency checks in  $G_1$ ’s PRF oracle cover inconsistencies (1), (3), and (4) in that order but

not (2). The consistency checks in  $G_1$ 's primitive oracle for forward queries covers inconsistency (1) but not (2). The consistency checks in  $G_1$ 's primitive oracle for backwards queries covers inconsistency (2) but not (1), though its code for avoiding (2) is incorrect as is the oracles return statement. Inconsistencies (3) and (4) do not apply to primitive queries.

The  $mp/\rho_{\text{Chop}}$  term in the proofs bound is claimed to cover (1) from the PRF and forward primitive queries and well as (2) from the backwards primitive queries. The last of these is seemingly incorrect, but the term should instead cover the missing (1) from these queries.

The  $up/\rho_{\text{Chop}}$  term in our bound arises from addressing inconsistency (2). Ideally we would like it to be an  $mp$  term, but the type of analysis used for (1) does not apply. We can think of the analysis in (1) as using the fact that for a given  $k$  and unexposed users' key  $K_u$  we have

$$\Pr[k = K_u] \leq \begin{cases} 1/\rho_{\text{Chop}}, & \text{if } \text{Chop}(1^\lambda, K_u) = \text{Chop}(1^\lambda, k) \\ 0, & \text{otherwise.} \end{cases}$$

There are fewer than  $m$  keys satisfying  $\text{Chop}(1^\lambda, K_u) = \text{Chop}(1^\lambda, k)$  and so a union bound gives  $\Pr[k \in \{K_1, \dots, K_{u(\lambda)}\}] \leq (m(\lambda) - 1)/\rho_{\text{Chop}}(\lambda)$ .

On the other hand, to analyze (2) for a given  $z$  and unexposed user  $u$  we have

$$\Pr[z = V_x[u] = T_u[x] \oplus K_u] \leq \begin{cases} 1/\rho_{\text{Chop}}, & \text{if } \text{Chop}(1^\lambda, K_u) = \text{Chop}(1^\lambda, T_u[x] \oplus z) \\ 0, & \text{otherwise.} \end{cases}$$

In our proof we upper bound how many users can satisfy the first case of this by  $u(\lambda)$ . This gives  $\Pr[z \in \{T_1[x] \oplus K_1, \dots\}] \leq u(\lambda)/\rho_{\text{Chop}}(\lambda)$ . We do not know a good general argument for improving this, though can do so when  $\text{Chop}(1^\lambda, \cdot)$  is truncation.

The core observation is that truncation is a homomorphism over  $\oplus$ , so the first case implies that  $\text{Chop}(1^\lambda, K_u \oplus T_u[x]) = \text{Chop}(1^\lambda, z)$ . Consider a fixed  $x$ . Define  $K_{u,x} = T_u[x] \oplus K_u$  for each user. Then  $\mu(\text{Chop}(1^\lambda, K_{1,x}), \dots, \text{Chop}(1^\lambda, K_{u(\lambda),x}))$  upper bounds the number of users that can satisfy the first case for any  $z$ . Because each  $T_u[x]$  is uniform and independent, each  $K_{1,x}$  is uniform and independent. Thus the probability there are at least  $m$  unexposed users satisfying the first case can be bounded by  $P_{\text{Chop}}^{\text{coll}}(u, m)$ . Applying a union bound over the at most  $q$  values of  $x$  queried to EV, the probability of their being an  $x$  with at least  $m$  unexposed users satisfying the first case is bounded by  $q \cdot P_{\text{Chop}}^{\text{coll}}(\lambda, u, m)$ . Consequently, in Theorem 9 applied to truncation we can replace the  $up/\rho_{\text{Chop}}$  term in our bound with  $q(\lambda) \cdot P_{\text{Chop}}^{\text{coll}}(u, m) + (m-1)p/\rho_{\text{Chop}}$ , a significant improvement. For example, with the parameters Bellare, et al. consider (truncating from  $c$  to  $r$  bits and setting  $m = 2u/2^r + 3cr$ ) we have  $P_{\text{Chop}}^{\text{coll}}(u, m) = e^{-c} < 2^{-c} = |\text{DM}_{\text{Chop}} \cdot \mathbf{K}|$  so these two terms are upper-bounded by other terms in our theorem.